

Data Wrangling with R

Aravind Hebbali

2026-10-02

Table of contents

Preface	10
Structure of the book	11
The pipe <code> ></code> in 30 seconds	11
Software information	11
About the Author	13
1 Import Data - Basics	14
1.1 Introduction	14
1.2 Delimiters	14
1.2.1 Comma Separated Values	14
1.2.2 Semi Colon Separated Values	15
1.2.3 Space Separated Values	15
1.2.4 Tab Separated Values	15
1.3 Read Data	15
1.4 Column Names	17
1.5 Skip Lines	19
1.6 Maximum Lines	20
1.7 Column Types	21
1.8 Select Columns	23
1.9 Beyond CSV: parquet and quiet reads	24
1.10 Summary	25
2 Import Data - Advanced	26
2.1 Introduction	26
2.2 List Sheets	26
2.3 Read Sheet	26
2.3.1 Case 1: Specify the sheet number	27
2.3.2 Case 2: Specify the sheet name	27
2.4 Read Specific Cells	27
2.4.1 Method 1	28
2.4.2 Method 2	29
2.4.3 Method 3	29
2.4.4 Method 1	31
2.4.5 Method 2	31
2.4.6 Method 3	31
2.5 Read Specific Rows	32
2.6 Read Single Column	32

2.7	Read Multiple Columns	32
2.7.1	Summary	34
2.8	Statistical Softwares	34
2.8.1	STATA	34
2.8.2	SPSS	35
2.8.3	SAS	35
2.9	Summary	36
3	Pipe Operator	37
3.1	Introduction	37
3.2	Pipes	37
3.2.1	> vs %>% at a glance	37
3.3	Data	39
3.3.1	Data Dictionary	40
3.4	head()	40
3.4.1	Using Pipe	40
3.5	Square Root	41
3.6	Visualization	42
3.6.1	Using pipe	43
3.7	Correlation	44
3.8	Regression	45
3.8.1	Using pipe	46
3.9	String Manipulation	47
3.10	Data Extraction	48
3.10.1	Extract Column	48
3.10.2	Extract List Element	50
3.11	Arithmetic Operations	51
3.11.1	Addition	52
3.11.2	Multiplication	52
3.11.3	Division	52
3.11.4	Power	53
3.12	Logical Operators	53
3.12.1	Greater Than	54
3.12.2	Weakly Greater Than	54
4	dplyr Basics	55
4.1	Introduction	55
4.2	dplyr Verbs	56
4.3	Data	56
4.3.1	Data Dictionary	56
4.4	Case Study	57
4.5	Average Order Value	57
4.6	AOV by Devices	58
4.7	Syntax	58
4.8	Filter Rows	58
4.8.1	Case Study	61

4.9	Select Columns	61
4.9.1	Case Study	64
4.10	Grouping Data	65
4.10.1	Case Study	65
4.11	Summarise Data	66
4.11.1	Case Study	69
4.12	Create Columns	70
4.12.1	Case Study	71
4.13	Arrange Data	72
4.13.1	Case Study	74
4.14	AOV by Devices	74
4.15	Your Turn	76
	Try it live	76
5	Joining Tables using dplyr	77
5.1	Introduction	77
5.2	Case Study	77
5.2.1	Data: Orders	78
5.2.2	Data: Customers	78
5.3	Example Data	79
5.4	Specifying keys with <code>join_by()</code>	79
5.5	Inner Join	80
5.5.1	Case Study: Details of customers who have placed orders and their order details	80
5.6	Left Join	80
5.7	Case Study: Details of customers and their orders irrespective of whether a customer has	81
5.8	Right Join	81
5.8.1	Case Study: Customer details for each order	82
5.9	Semi Join	82
5.9.1	Case Study: Details of customers who have placed orders	83
5.10	Anti Join	83
5.10.1	Case Study: Details of customers who have not placed orders	84
5.10.2	Audit pattern: check mismatches in both directions	84
5.11	Full Join	85
5.11.1	Case Study: Details of all customers and all orders	86
5.12	Cross Join	86
	Try it live	87
6	dplyr Helpers	88
6.1	Introduction	88
6.2	Case Study	88
6.2.1	Data	88
6.2.2	Data Dictionary	89
6.3	Data Sanitization	89
6.4	Rename Columns	90

6.5	Data Tabulation	91
6.6	Sampling Data	94
6.7	Data Extraction	96
6.7.1	Sample Data	96
6.8	Between	98
6.9	Case When	99
	Try it live	99
7	tibbles	101
7.1	Introduction	101
7.2	Creating tibbles	101
7.3	tibble features	102
7.3.1	never changes input's types	102
7.3.2	never adjusts variable names	103
7.3.3	never prints all rows	103
7.3.4	never recycles vector of length greater than 1	103
7.4	Membership Testing	104
7.5	Tibble	104
7.6	Column Names	105
7.7	Add Rows	105
7.8	Add Columns	106
7.9	Rownames	106
7.9.1	Remove Rownames	107
7.9.2	Rownames to Column	107
7.9.3	Column to Rownames	108
7.10	Glimpse	109
7.11	Check Column	109
7.12	Summary	110
7.12.1	Creating tibbles	110
7.12.2	Modifying tibbles	110
7.12.3	Testing tibbles	110
	Try it live	110
8	Tidying Data with tidyr	111
8.1	Introduction	111
8.2	Data	111
8.3	Long format with <code>pivot_longer()</code>	112
8.4	Wide format with <code>pivot_wider()</code>	113
8.5	Split columns with <code>separate_wider_delim()</code>	114
8.6	Combine columns with <code>unite()</code>	114
8.7	Drop missing values with <code>drop_na()</code>	115
8.8	Complete combinations with <code>complete()</code>	115
8.9	Fill down labels with <code>fill()</code>	116
8.10	Your Turn	117
	Try it live	117

9	Hacking Strings	118
9.1	Introduction	118
9.2	Case Study	118
9.2.1	Data	118
9.2.2	Data Dictionary	119
9.3	Overview	119
9.4	Extract domain name from email ids	130
9.4.1	Steps	130
9.5	Extract Domain Extension	133
9.6	Extract image type from URL	133
9.6.1	Steps	133
9.7	Extract Image Dimesion from URL	136
9.7.1	Steps	136
9.8	Extract HTTP Protocol from URL	138
9.8.1	Steps	138
9.9	Extract file type	139
9.9.1	Steps	139
	Try it live	141
10	Date & Time: Parsing and Components	142
10.1	Introduction	142
10.1.1	Date	142
10.1.2	Time	142
10.1.3	AM or PM?	143
10.1.4	Leap Year	143
10.1.5	Summary	143
10.1.6	Your Turn	144
10.2	Case Study	144
10.2.1	Data	144
10.2.2	Data Dictionary	144
10.3	Date & Time Classes	145
10.3.1	Introduction	145
10.3.2	Date	146
10.3.3	ISO 8601	149
10.3.4	POSIX	150
10.3.5	Your Turn	153
10.4	Date & Time Formats	154
10.4.1	Introduction	154
10.4.2	Conversion Specifications	154
10.4.3	Guess Format	156
10.4.4	Your Turn	157
10.5	Parse Date & Time	157
10.5.1	Your Turn	161
10.6	Date & Time Components	162
10.6.1	Year	162
10.6.2	Month	163

10.6.3	Week	163
10.6.4	Day	164
10.6.5	Days in Month	165
10.6.6	Hour, Minute & Seconds	166
10.6.7	Quarter & Semester	167
10.6.8	Case Study	168
10.6.9	Your Turn	170
10.7	Create, Update & Verify	170
10.7.1	Create	170
10.7.2	Update	171
10.7.3	Date Sequence	171
10.7.4	Verify Type	174
10.7.5	Your Turn	174
	Try it live	174
11	Date & Time: Arithmetic and Time Zones	176
11.1	Date Arithmetic	176
11.1.1	Introduction	176
11.1.2	Shift Date	177
11.1.3	Case Study	177
11.1.4	Your Turn	179
11.2	Time Zones	179
11.2.1	Introduction	179
11.2.2	Time Zones	180
11.2.3	Daylight Savings	182
11.2.4	Your Turn	183
11.3	Intervals, Duration & Period	183
11.3.1	Interval	183
11.3.2	Within	188
11.3.3	Duration	189
11.3.4	Period	192
11.4	Others	194
11.4.1	Rounding Dates	195
11.4.2	Rollback	196
11.4.3	Your Turn	196
	Try it live	196
12	Categorical Data Fundamentals	198
12.1	Introduction	198
12.1.1	Data Types	198
12.1.2	Categorical Data	200
12.1.3	Dichotomous	200
12.1.4	Polychotomous	201
12.1.5	Ordinal	202
12.1.6	Nominal	202
12.1.7	Summary	203

12.1.8	Your Turn...	203
12.2	Case Study	204
12.2.1	Data	205
12.2.2	Data Dictionary	206
12.3	Factors	207
12.3.1	Introduction	207
12.3.2	Membership Testing	208
12.3.3	Coercion	208
12.3.4	Factor Function	209
12.3.5	Modify Labels	210
12.3.6	Missing Values	210
12.3.7	Ordered Factors	211
12.3.8	Modify Order of Levels	212
12.3.9	Key Functions	213
12.3.10	Summary	213
12.3.11	Your Turn...	214
12.4	Summarize	215
12.4.1	Introduction	215
12.4.2	Number of Categories	215
12.4.3	Category Names	215
12.4.4	Names & Counts	216
12.4.5	Tables	217
12.4.6	Contingency Table	227
12.4.7	Key Functions	230
12.4.8	Your Turn...	230
13	Factor Manipulation with forcats	231
13.1	Data Manipulation	231
13.1.1	Introduction	231
13.1.2	Your Turn...	233
13.1.3	Change Value of Levels	233
13.1.4	Key Functions	241
13.1.5	Your Turn..	242
13.1.6	Add / Remove Levels	242
13.1.7	Key Functions	245
13.1.8	Change Order of Levels	245
13.1.9	Key Functions	250
13.1.10	Your Turn...	251
13.2	Data Visualization	251
13.2.1	Introduction	251
13.2.2	Bar Plot	252
13.2.3	Grouped Bar Plot	252
13.2.4	Stacked Bar Plot	253
13.2.5	Proportional Bar Plot	254
13.2.6	Mosaic Plot	255
13.2.7	Pie Chart	255

13.2.8 Donut Chart	257
13.2.9 Summary	258
13.2.10 Your Turn...	258
13.3 Useful R Packages	265
13.3.1 Summarize Variables	265
13.3.2 Summarize Data Set	267
Try it live	271
14 Exercise Solutions	272
14.1 Introduction	272
14.2 dplyr Basics	272
14.3 Open stubs	273
Appendices	274
A Arrow, Parquet and DuckDB	274
A.1 Introduction	274
A.2 Parquet with arrow	274
A.3 DuckDB with duckplyr	275
B Legacy magrittr Operators	277
C From pandas to dplyr	278
C.1 Introduction	278
C.2 Verb map	278
C.3 Worked pair: grouped means	279
Try it live	280

Preface



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Structure of the book

Chapters `@import-data-in-r-basics` and `@import-data-in-r-advanced` focus on reading data from flat/delimited files and spreadsheets. Chapters `@dplyr-basics`, `@joining-tables-in-r-dplyr` and `@dplyr-helper-functions` focus on wrangling data using the **dplyr** package, chaining verbs with the base R pipe `|>` (see the primer below). Chapter `@r-pipe-magrittr` covers the pipe mechanic in detail, including the `_` placeholder, anonymous functions with `\(x)`, and reading older magrittr code. Chapter `@tibbles-in-r` explores `tibble()`, an alternative for `data.frame()`. Chapter `@tidying-data-with-tidyr` reshapes data with the **tidyr** package (pivots, splitting/combining columns, missing-value helpers). Chapters `@strings-in-r`, `@date-and-time-in-r` and `@categorical-data-in-r` explore ways to handle text, date/time and categorical data.

The pipe `|>` in 30 seconds

Every example from `@dplyr-basics` onward chains verbs with the base R pipe `|>`, available in R 4.1 and later with no extra package:

- `|>` forwards the value on its left into the **first argument** of the function on its right, so `ecom |> filter(purchase)` means `filter(ecom, purchase)`.
- Every **dplyr** verb takes a data frame (or tibble) first and returns one, so a chain like `ecom |> filter(purchase) |> summarise(...)` reads top to bottom as “take `ecom`, keep purchasers, then summarise”.

Tip

If you inherit code using the older magrittr pipe `%>`, read it the same way: for first-argument forwarding it behaves identically to `|>`. The differences (`_` placeholder, `\(x)` functions, exposition/assignment variants) are covered in `@r-pipe-magrittr`.

Software information

This book requires R $\geq$ 4.1 (for the native pipe `|>`) and dplyr $\geq$ 1.1.0 (for `.by`, `slice_*()` and `join_by()`); package versions are pinned in `renv.lock`. The R session information when compiling this book is shown below:

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.5 LTS
```

```

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p0.3.26.so; LAPACK version 3.6.0

locale:
 [1] LC_CTYPE=C.UTF-8      LC_NUMERIC=C          LC_TIME=C.UTF-8
 [4] LC_COLLATE=C.UTF-8   LC_MONETARY=C.UTF-8  LC_MESSAGES=C.UTF-8
 [7] LC_PAPER=C.UTF-8     LC_NAME=C            LC_ADDRESS=C
[10] LC_TELEPHONE=C       LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C

time zone: UTC
tzcode source: system (glibc)

attached base packages:
[1] stats      graphics  grDevices datasets  utils      methods    base

loaded via a namespace (and not attached):
 [1] compiler_4.5.3 fastmap_1.2.0 cli_3.6.6      htmltools_0.5.9
 [5] tools_4.5.3    ote1_0.2.0    yaml_2.3.12    rmarkdown_2.32
 [9] knitr_1.52     jsonlite_2.0.0 xfun_0.61      digest_0.6.39
[13] rlang_1.3.0    renv_1.2.4     evaluate_1.0.5

```

We do not add prompts (`>` and `+`) to R source code in this book, and we comment out the text output with two hashes `##` by default, as you can see from the R session information above. This is for your convenience when you want to copy and run the code (the text output will be ignored since it is commented out). Package names are in bold text (e.g., **rmarkdown**), and function names are followed by parentheses (e.g., **bookdown::render_book()**). The double-colon operator `::` means accessing an object from a package.

About the Author

[Aravind Hebbali](#) is the founder of [Rsquared Academy](#). He earned his Masters in Economics from [Madras School of Economics](#). As an active R user, he has authored several R packages such as

- [olsrr](#)
- [rfm](#)
- [descriptr](#)
- [blorr](#)
- [xploreerr](#)

In 2015, he founded [Rsquared Academy](#), a free and open source education initiative with focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on [GitHub](#).

1 Import Data - Basics

1.1 Introduction

In this chapter, we will learn to:

- read data from flat or delimited files
- handle column names/header
- skip text/info present before data
- specify column/variable types
- read specific columns/variables

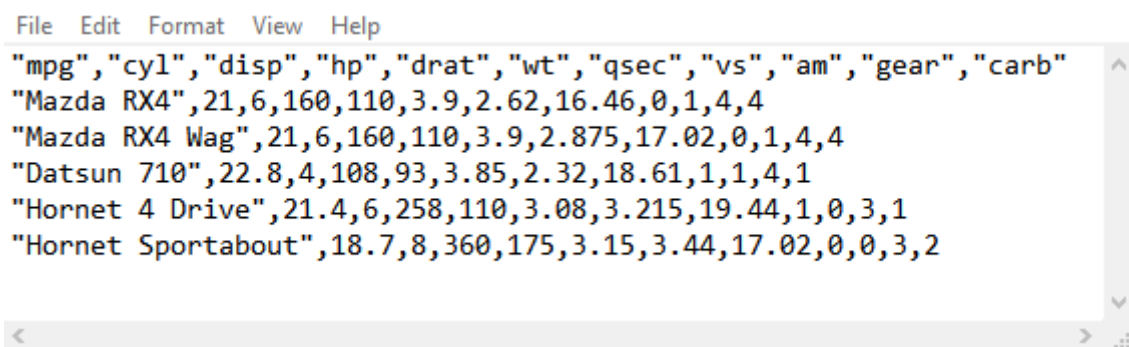
We will use the following R packages:

```
library(readr)
```

1.2 Delimiters

Before we start reading data from files, let us take a quick look at the different types of delimiters we have to deal with while reading or importing data. In general, it is a good practice to take a quick look at as you will clearly know the delimiter used in the file.

1.2.1 Comma Separated Values



```
File Edit Format View Help
"mpg", "cyl", "disp", "hp", "drat", "wt", "qsec", "vs", "am", "gear", "carb"
"Mazda RX4", 21, 6, 160, 110, 3.9, 2.62, 16.46, 0, 1, 4, 4
"Mazda RX4 Wag", 21, 6, 160, 110, 3.9, 2.875, 17.02, 0, 1, 4, 4
"Datsun 710", 22.8, 4, 108, 93, 3.85, 2.32, 18.61, 1, 1, 4, 1
"Hornet 4 Drive", 21.4, 6, 258, 110, 3.08, 3.215, 19.44, 1, 0, 3, 1
"Hornet Sportabout", 18.7, 8, 360, 175, 3.15, 3.44, 17.02, 0, 0, 3, 2
```

1.2.2 Semi Colon Separated Values

```
File Edit Format View Help
"mpg";"cyl";"displ";"hp";"drat";"wt";"qsec";"vs";"am";"gear";"carb"
"Mazda RX4";21;6;160;110;3.9;2.62;16.46;0;1;4;4
"Mazda RX4 Wag";21;6;160;110;3.9;2.875;17.02;0;1;4;4
"Datsun 710";22.8;4;108;93;3.85;2.32;18.61;1;1;4;1
"Hornet 4 Drive";21.4;6;258;110;3.08;3.215;19.44;1;0;3;1
"Hornet Sportabout";18.7;8;360;175;3.15;3.44;17.02;0;0;3;2
```

1.2.3 Space Separated Values

```
File Edit Format View Help
"mpg" "cyl" "displ" "hp" "drat" "wt" "qsec" "vs" "am" "gear" "carb"
"Mazda RX4" 21 6 160 110 3.9 2.62 16.46 0 1 4 4
"Mazda RX4 Wag" 21 6 160 110 3.9 2.875 17.02 0 1 4 4
"Datsun 710" 22.8 4 108 93 3.85 2.32 18.61 1 1 4 1
"Hornet 4 Drive" 21.4 6 258 110 3.08 3.215 19.44 1 0 3 1
"Hornet Sportabout" 18.7 8 360 175 3.15 3.44 17.02 0 0 3 2
```

1.2.4 Tab Separated Values

```
File Edit Format View Help
"mpg"    "cyl"    "displ"  "hp"    "drat"   "wt"    "qsec"   "vs"    "am"    "gear"   "carb"
"Mazda RX4"      21      6      160     110     3.9     2.62    16.46    0       1       4       4
"Mazda RX4 Wag"  21      6      160     110     3.9     2.875   17.02    0       1       4       4
"Datsun 710"     22.8    4      108     93      3.85    2.32    18.61    1       1       4       1
"Hornet 4 Drive" 21.4    6      258     110     3.08    3.215   19.44    1       0       3       1
"Hornet Sportabout" 18.7    8      360     175     3.15    3.44    17.02    0       0       3       2
```

1.3 Read Data

Let us begin by reading data from a csv file using `read_csv()`.

```
read_csv('hsb2.csv')
```

```
# A tibble: 200 x 11
      id female  race  ses schtyp  prog  read write  math science socst
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    70      0    4    1     1     1    57    52    41     47    57
2   121      1    4    2     1     3    68    59    53     63    61
3    86      0    4    3     1     1    44    33    54     58    31
4   141      0    4    3     1     3    63    44    47     53    56
5   172      0    4    2     1     2    47    52    57     53    61
6   113      0    4    2     1     2    44    52    51     63    61
7    50      0    3    2     1     1    50    59    42     53    61
8    11      0    1    2     1     2    34    46    45     39    36
9    84      0    4    2     1     1    63    57    54     58    51
10   48      0    3    2     1     2    57    55    52     50    51
# i 190 more rows
```

Great! If you see the above output, you have successfully read data into R. If you see an error message (which most of us see when we are trying to read data for the first time), follow the below instructions:

- check the separator in the file and ensure it is a **comma**
- check the file name
- check the file path i.e. location of the file
- ensure that the file name or path is enclosed in single or double quotes

When you read data using `readr`, it will display the data type detected for each column/variable in the data set. If you want to check the data types before reading the data, use `spec_csv()`. We will learn to specify the column types in the next section.

```
spec_csv('hsb2.csv')
```

```
cols(
  id = col_double(),
  female = col_double(),
  race = col_double(),
  ses = col_double(),
  schtyp = col_double(),
  prog = col_double(),
  read = col_double(),
  write = col_double(),
  math = col_double(),
  science = col_double(),
  socst = col_double()
)
```

1.4 Column Names

In some cases, files do not include column names or headers. If we do not indicate the absence of column names, `readr` will treat the first row from the data as the column name. Like we said before, it is a good practice to take a quick look at the data to check for the presence/absence of column names.

```
File Edit Format View Help
"mpg", "cyl", "disp", "hp", "drat", "wt", "qsec", "vs", "am", "gear", "carb"
"Mazda RX4", 21, 6, 160, 110, 3.9, 2.62, 16.46, 0, 1, 4, 4
"Mazda RX4 Wag", 21, 6, 160, 110, 3.9, 2.875, 17.02, 0, 1, 4, 4
"Datsun 710", 22.8, 4, 108, 93, 3.85, 2.32, 18.61, 1, 1, 4, 1
"Hornet 4 Drive", 21.4, 6, 258, 110, 3.08, 3.215, 19.44, 1, 0, 3, 1
"Hornet Sportabout", 18.7, 8, 360, 175, 3.15, 3.44, 17.02, 0, 0, 3, 2
```

```
File Edit Format View Help
"Mazda RX4", 21, 6, 160, 110, 3.9, 2.62, 16.46, 0, 1, 4, 4
"Mazda RX4 Wag", 21, 6, 160, 110, 3.9, 2.875, 17.02, 0, 1, 4, 4
"Datsun 710", 22.8, 4, 108, 93, 3.85, 2.32, 18.61, 1, 1, 4, 1
"Hornet 4 Drive", 21.4, 6, 258, 110, 3.08, 3.215, 19.44, 1, 0, 3, 1
"Hornet Sportabout", 18.7, 8, 360, 175, 3.15, 3.44, 17.02, 0, 0, 3, 2
```

We will first read the data set without indicating the presence or absence of column names.

```
read_csv('hsb3.csv')
```

```
# A tibble: 199 x 11
  `70`  `0`  `4` `1...4` `1...5` `1...6` `57...7` `52` `41` `47`
  <dbl> <dbl> <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl> <dbl> <dbl>
1   121     1     4       2       1       3       68    59    53    63
2    86     0     4       3       1       1       44    33    54    58
3   141     0     4       3       1       3       63    44    47    53
4   172     0     4       2       1       2       47    52    57    53
5   113     0     4       2       1       2       44    52    51    63
6    50     0     3       2       1       1       50    59    42    53
7    11     0     1       2       1       2       34    46    45    39
8    84     0     4       2       1       1       63    57    54    58
9    48     0     3       2       1       2       57    55    52    50
```

```

10    75    0    4    2    1    3    60    46    51    53
# i 189 more rows
# i 1 more variable: `57...11` <dbl>

```

As you can see, in the absence of column names, `readr` has converted the first row of the data into the column names. As a result, the data is not read properly and there are lots of missing values and warnings. If the column names are absent (i.e. the column names are provided in a separate file), use the `col_names` argument and set it to `FALSE`. Now `readr` will not convert the first row of data into column name and instead it will generate new column names.

```
read_csv('hsb3.csv', col_names = FALSE)
```

```

# A tibble: 200 x 11
      X1     X2     X3     X4     X5     X6     X7     X8     X9     X10    X11
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     70     0     4     1     1     1    57    52    41    47    57
2    121     1     4     2     1     3    68    59    53    63    61
3     86     0     4     3     1     1    44    33    54    58    31
4    141     0     4     3     1     3    63    44    47    53    56
5    172     0     4     2     1     2    47    52    57    53    61
6    113     0     4     2     1     2    44    52    51    63    61
7     50     0     3     2     1     1    50    59    42    53    61
8     11     0     1     2     1     2    34    46    45    39    36
9     84     0     4     2     1     1    63    57    54    58    51
10    48     0     3     2     1     2    57    55    52    50    51
# i 190 more rows

```

We may not always want to use the column names generated by `readr` and instead specify new column names. In such cases, we can use `col_names` to supply column names as shown in the below example. Let us reread `hsb3` and specify column names.

```

cnames <- c("id", "gender", "race", "socio_economic_status", "school_type", "program", "read", "write")
read_csv('hsb3.csv', col_names = cnames)

```

```

# A tibble: 200 x 11
      id gender  race socio_economic_status school_type program  read write
  <dbl> <dbl> <dbl>          <dbl>          <dbl>    <dbl> <dbl> <dbl>
1     70     0     4              1              1         1    57    52
2    121     1     4              2              1         3    68    59
3     86     0     4              3              1         1    44    33
4    141     0     4              3              1         3    63    44
5    172     0     4              2              1         2    47    52
6    113     0     4              2              1         2    44    52

```

```

7      50      0      3      2      1      1      50      59
8      11      0      1      2      1      2      34      46
9      84      0      4      2      1      1      63      57
10     48      0      3      2      1      2      57      55
# i 190 more rows
# i 3 more variables: math <dbl>, science <dbl>, socst <dbl>

```

1.5 Skip Lines

In certain files, you will find information related to the data such as:

- the data source
- column names
- column description
- copyright etc.

The data will appear after/below such text/information. While reading data from such files, we need to skip all the rows where the text is present. If we do not skip them, `readr` will consider them as part of the data.

```

File Edit Format View Help
"The data was extracted from the 1974 Motor Trend US magazine, and comprises fuel consumption and 10 aspects of automobile design
"
A data frame with 32 observations on 11 variables.
mpg, cyl, disp, hp, drat, wt, qsec, vs, am, gear, carb
Henderson and Velleman (1981), Building multiple regression models interactively. Biometrics, 37, 391-411.
mpg,cyl,disp, hp, drat, wt, qsec, vs, am, gear, carb
21.6,160,110,3.9,2.62,16.46,0,1,4,4
21.6,160,110,3.9,2.875,17.02,0,1,4,4
22.8,4,108,93,3.85,2.32,18.61,1,1,4,1
21.4,6,258,110,3.08,3.215,19.44,1,0,3,1
18.7,8,360,175,3.15,3.44,17.02,0,0,3,2
18.1,6,225,105,2.76,3.46,20.22,1,0,3,1
14.3,8,360,245,3.21,3.57,15.84,0,0,3,4

```

Let us read the data without skipping any lines/rows and observe the result.

```
read_csv('hsb4.csv')
```

```

# A tibble: 203 x 1
  `# A dataset containing demographic information and standardized`
  <chr>
1 # test scores of high school students.
2 # http://www.ats.ucla.edu/stat/spss/whatstat/whatstat.htm
3 id,female,race,ses,schtyp,prog,read,write,math,science,socst

```

```

4 70,0,4,1,1,1,57,52,41,47,57
5 121,1,4,2,1,3,68,59,53,63,61
6 86,0,4,3,1,1,44,33,54,58,31
7 141,0,4,3,1,3,63,44,47,53,56
8 172,0,4,2,1,2,47,52,57,53,61
9 113,0,4,2,1,2,44,52,51,63,61
10 50,0,3,2,1,1,50,59,42,53,61
# i 193 more rows

```

Use `skip` argument to indicate the number of lines/rows to be skipped while reading data from a file. For example, if the file has contents other than data in the first few lines, we need to skip them before reading the data. In the below example, we will skip the first 3 lines as they contain information about the data set which we do not need.

```
read_csv('hsb4.csv', skip = 3)
```

```

# A tibble: 200 x 11
      id female  race  ses schtyp  prog  read write  math science socst
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    70      0     4     1     1     1    57    52    41     47     57
2   121      1     4     2     1     3    68    59    53     63     61
3    86      0     4     3     1     1    44    33    54     58     31
4   141      0     4     3     1     3    63    44    47     53     56
5   172      0     4     2     1     2    47    52    57     53     61
6   113      0     4     2     1     2    44    52    51     63     61
7    50      0     3     2     1     1    50    59    42     53     61
8    11      0     1     2     1     2    34    46    45     39     36
9    84      0     4     2     1     1    63    57    54     58     51
10   48      0     3     2     1     2    57    55    52     50     51
# i 190 more rows

```

1.6 Maximum Lines

Suppose the data file contains several thousands of rows of data and we do not want to read all of it. What can we do in such cases? `readr` allows us to specify the maximum number of rows to be read using the `n_max` argument. Suppose we want to read only 100 rows of data from a file, we can set `n_max` equal to 100. In the next example, we will read the first 120 rows from the `hsb2` file. If you observe the last row in the output, it says `# ... with 110 more rows`, indicating that only 120 rows of data has been read from the file.

```
read_csv('hsb2.csv', n_max = 120)
```

```
# A tibble: 120 x 11
      id female  race  ses schtyp  prog  read write  math science socst
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    70     0     4     1     1     1    57    52    41     47    57
2   121     1     4     2     1     3    68    59    53     63    61
3    86     0     4     3     1     1    44    33    54     58    31
4   141     0     4     3     1     3    63    44    47     53    56
5   172     0     4     2     1     2    47    52    57     53    61
6   113     0     4     2     1     2    44    52    51     63    61
7    50     0     3     2     1     1    50    59    42     53    61
8    11     0     1     2     1     2    34    46    45     39    36
9    84     0     4     2     1     1    63    57    54     58    51
10   48     0     3     2     1     2    57    55    52     50    51
# i 110 more rows
```

1.7 Column Types

If you have observed carefully, when you read data using `readr`, it displays the column names and column types followed by the first 10 rows of data. `readr` determines the data type for each column based on the first 1000 rows of data. The data can be of the following types:

- integer
- double (decimal point)
- logical (TRUE/FALSE)
- character (text/string)
- factor (categorical/qualitative)
- date/time

Data Type	Function
Integer	<code>col_integer()</code>
Double	<code>col_double()</code>
Logical	<code>col_logical()</code>
Categorical	<code>col_factor()</code>
Character	<code>col_character()</code>
Date/Time	<code>col_datetime()</code> , <code>col_date()</code> , <code>col_time()</code>
Skip	<code>col_skip()</code>

Before you read data from a file, use `spec_csv()` to see the data types as determined by `readr`. If it determines the data types correctly, you can go ahead and read the data else we will have to specify the data types and we will have to do that for all the columns we want to read and not just for those columns whose data type was wrongly determined by `readr`.

To specify the data types, we will use the `col_types` argument and supply it a list of data types. The data types can be specified using:

- `col_integer()`
- `col_double()`
- `col_factor()`
- `col_logical()`
- `col_character()`
- `col_date()`
- `col_time()`
- `col_datetime()`

While specifying the data types we also need to specify the categories of the categorical/qualitative variable. To do that, we use the `levels` argument within `col_factor()`. Let us read data from the `hsb2.csv` file to understand data type specification.

```
read_csv('hsb2.csv', col_types = list(
  col_integer(), col_factor(levels = c("0", "1")),
  col_factor(levels = c("1", "2", "3", "4")), col_factor(levels = c("1", "2", "3")),
  col_factor(levels = c("1", "2")), col_factor(levels = c("1", "2", "3")),
```

```
col_integer(), col_integer(), col_integer(), col_integer(),
col_integer())
)
```

```
# A tibble: 200 x 11
```

```
      id female race  ses  schtyp prog  read write  math science socst
  <int> <fct>  <fct> <fct> <fct> <fct> <int> <int> <int>    <int> <int>
1     70 0      4    1    1    1     57   52   41      47   57
2    121 1      4    2    1    3     68   59   53      63   61
3     86 0      4    3    1    1     44   33   54      58   31
4    141 0      4    3    1    3     63   44   47      53   56
5    172 0      4    2    1    2     47   52   57      53   61
6    113 0      4    2    1    2     44   52   51      63   61
7     50 0      3    2    1    1     50   59   42      53   61
8     11 0      1    2    1    2     34   46   45      39   36
9     84 0      4    2    1    1     63   57   54      58   51
10    48 0      3    2    1    2     57   55   52      50   51
# i 190 more rows
```

If we do not specify the data type for all columns, **readr** will return an error which leads to the following questions:

1. What if I want to skip a few columns?
2. What if I want to read certain columns only?

1.8 Select Columns

Objective	Function
Specify column data types	<code>col_types()</code>
Skip column	<code>col_skip()</code>
Read specific columns	<code>cols_only()</code>

For the first scenario, we can use `col_skip()` i.e. instead of specifying the data type, we indicate to `readr` to skip that particular column while reading the data.

In case of the second scenario, we will use `cols_only()` to specify the columns to be read i.e. instead of using `list()` to supply the data types, we will use `cols_only()` and provide the following details:

- column name
- column type using `col_types` argument

```
read_csv('hsb2.csv', col_types = cols_only(id = col_integer(),
  prog = col_factor(levels = c("1", "2", "3")), read = col_integer())
)
```

```
# A tibble: 200 x 3
      id prog  read
  <int> <fct> <int>
1     70 1      57
2    121 3      68
3     86 1      44
4    141 3      63
5    172 2      47
6    113 2      44
7     50 1      50
8     11 2      34
9     84 1      63
10    48 2      57
# i 190 more rows
```

If you have a data set with 10 columns and plan to skip only a couple of columns, use `col_skip()` instead if you plan to read only a couple of columns, use `cols_only()`.

1.9 Beyond CSV: parquet and quiet reads

Two habits scale beyond this chapter. First, `read_csv()` announces its guessed column types on every read; once you trust the spec, silence it with `show_col_types = FALSE`. Second, remote files re-download on every knit — cache them locally with `download.file()` once and read from disk afterwards. And when a CSV grows past comfortable memory (the 35 MB `analytics_raw.csv` in later chapters is the first warning sign), switch the format, not the workflow: `arrow::read_parquet()` returns the same tibble the dplyr verbs expect. See [@arrow-duckdb](#) for the full quickstart.

1.10 Summary

Type	readr	Base R
comma	<code>read_csv()</code>	<code>read.csv()</code>
semicolon	<code>read_csv2()</code>	<code>read.csv2()</code>
tab	<code>read_tsv()</code>	<code>read.delim() / read.table()</code>
space	<code>read_table()</code>	<code>read.table()</code>
multiple spaces	<code>read_table2()</code>	<code>read.table()</code>
any delimiter	<code>read_delim()</code>	<code>read.delim()</code>

2 Import Data - Advanced

2.1 Introduction

In this chapter, we will:

- list sheets in an excel file
- read data from an excel sheet
- read specific cells from an excel sheet
- read specific rows
- read specific columns
- read data from - SAS - SPSS - STATA

We will use the following R packages:

```
library(readxl)
library(haven)
```

2.2 List Sheets

An excel file may contain several sheets. Let us see how many sheets are present in `sample.xls` file and their respective names using `excel_sheets()`.

```
excel_sheets('sample.xls')
```

```
[1] "ecom"
```

2.3 Read Sheet

Now that we know the number of sheets and their names, let us read data from the `ecom` sheet of the `sample.xls` file using `read_excel()`. We will specify the file name, and the sheet name or sheet number.

2.3.1 Case 1: Specify the sheet number

```
read_excel('sample.xls', sheet = 1)
```

```
# A tibble: 7 x 5
  channel      users new_users sessions bounce_rate
  <chr>      <dbl>   <dbl>   <dbl> <chr>
1 Organic Search 43296   40238   50810 48.72%
2 Direct        12916   12311   16419 49.27%
3 Referral      10983    7636   18105 22.26%
4 Social        10346   10029   11101 61.92%
5 Display        5564    4790    7220 83.30%
6 Paid Search   2687     2205    3438 38.02%
7 Affiliates    1773     1585    2167 55.75%
```

2.3.2 Case 2: Specify the sheet name

```
read_excel('sample.xls', sheet = 'ecom')
```

```
# A tibble: 7 x 5
  channel      users new_users sessions bounce_rate
  <chr>      <dbl>   <dbl>   <dbl> <chr>
1 Organic Search 43296   40238   50810 48.72%
2 Direct        12916   12311   16419 49.27%
3 Referral      10983    7636   18105 22.26%
4 Social        10346   10029   11101 61.92%
5 Display        5564    4790    7220 83.30%
6 Paid Search   2687     2205    3438 38.02%
7 Affiliates    1773     1585    2167 55.75%
```

Notice when you use the sheet name, the name should be enclosed in single/double quotes.

2.4 Read Specific Cells

You may not always want to read all the columns or rows from the excel sheet. In such cases, you can specify the cells from which the data must be read which can be achieved using the `range` argument. So how do we specify the cells from which to read data? There are different ways of specifying the cell range and we will look at them one by one:

2.4.1 Method 1

	A	B	C	D	E
1	channel	users	new_users	sessions	bounce_rate
2	Organic Search	43296	40238	50810	48.72%
3	Direct	12916	12311	16419	49.27%
4	Referral	10983	7636	18105	22.26%
5	Social	10346	10029	11101	61.92%
6	Display	5564	4790	7220	83.30%
7	Paid Search	2687	2205	3438	38.02%
8	Affiliates	1773	1585	2167	55.75%
9					

→ range(B1:C4) →

B	C
users	new_users
43296	40238
12916	12311
10983	7636

The first method uses the cell names along with `:` to specify the cell range. For example, to read data from first 4 rows of columns **B** and **C**, we will specify the range as "B1:C4".

```
read_excel('sample.xls', sheet = 1, range = "B1:C4")
```

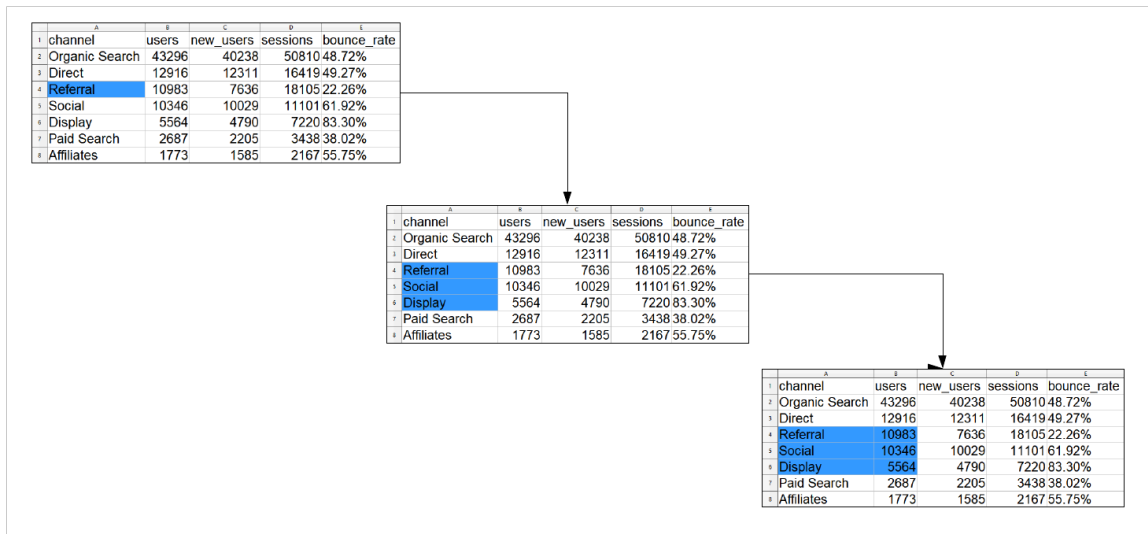
```
# A tibble: 3 x 2
  users new_users
  <dbl>   <dbl>
1 43296   40238
2 12916   12311
3 10983    7636
```

To read data from first 5 rows of columns **A**, **B** and **C**, we will specify the range as "A1:C5".

```
read_excel('sample.xls', sheet = 1, range = "A1:C5")
```

```
# A tibble: 4 x 3
  channel      users new_users
  <chr>      <dbl>   <dbl>
1 Organic Search 43296   40238
2 Direct        12916   12311
3 Referral      10983    7636
4 Social        10346   10029
```

2.4.2 Method 2



In the second method, we start from a particular cell and specify the number of rows and columns to be covered keeping the initial cell as anchorage. In the below example, we want to read 3 rows and 2 columns starting from the cell A4.

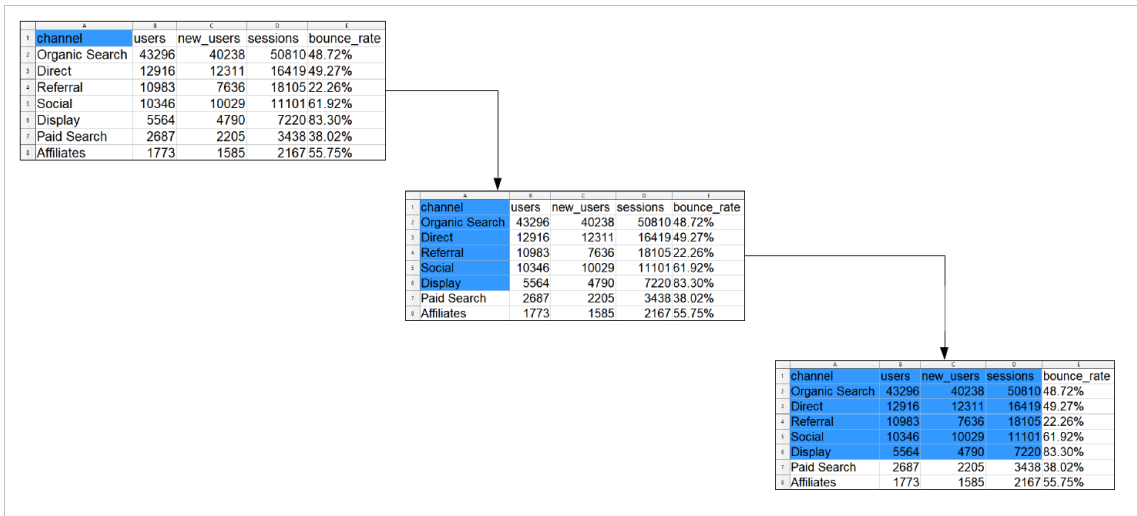
```
read_excel('sample.xls', sheet = 1, col_names = FALSE,  
           range = anchored("A4", dim = c(3, 2)))
```

```
# A tibble: 3 x 2  
  ...1    ...2  
  <chr>   <dbl>  
1 Referral 10983  
2 Social   10346  
3 Display   5564
```

2.4.3 Method 3

In this method, we use the `cell_limit()` and specify the location of two ends of a rectangle covering the cells we want to read. For example, to read data from the first 6 rows and 4 columns, we will specify the range as following:

- start from the first row of the first column
- cover all cells upto the 6th row of the 4th column



```
read_excel('sample.xls', sheet = 1,
  range = cell_limits(c(1, 1), c(6, 4)))
```

```
# A tibble: 5 x 4
  channel      users new_users sessions
  <chr>      <dbl>   <dbl>   <dbl>
1 Organic Search 43296   40238   50810
2 Direct        12916   12311   16419
3 Referral      10983    7636   18105
4 Social        10346   10029   11101
5 Display        5564    4790    7220
```

You can use NA to indicate the first and last row/column. For example, to read data from all the rows from the second column onwards:

```
read_excel('sample.xls', sheet = 1,
  range = cell_limits(c(1, 2), c(NA, NA)))
```

```
# A tibble: 7 x 4
  users new_users sessions bounce_rate
  <dbl>   <dbl>   <dbl> <chr>
1 43296   40238   50810 48.72%
2 12916   12311   16419 49.27%
3 10983    7636   18105 22.26%
4 10346   10029   11101 61.92%
5 5564    4790    7220 83.30%
6 2687    2205    3438 38.02%
7 1773    1585    2167 55.75%
```

Let us quickly look at how we will specify range of cells using the above 3 methods when we want to read data from the first 4 rows of columns B and C:

2.4.4 Method 1

```
read_excel('sample.xls', sheet = 1,  
           range = "B1:C4")
```

```
# A tibble: 3 x 2  
  users new_users  
  <dbl>   <dbl>  
1 43296   40238  
2 12916   12311  
3 10983    7636
```

2.4.5 Method 2

```
read_excel('sample.xls', sheet = 1,  
           range = anchored("B1", dim = c(4, 2)))
```

```
# A tibble: 3 x 2  
  users new_users  
  <dbl>   <dbl>  
1 43296   40238  
2 12916   12311  
3 10983    7636
```

2.4.6 Method 3

```
read_excel('sample.xls', sheet = 1,  
           range = cell_limits(c(1, 2), c(4, 3)))
```

```
# A tibble: 3 x 2  
  users new_users  
  <dbl>   <dbl>  
1 43296   40238  
2 12916   12311  
3 10983    7636
```

2.5 Read Specific Rows

When you want to read a subset of rows from the data, use `cell_rows()` and specify the row numbers or the range. In the below example, we want to read the first 4 rows of data from the file.

```
read_excel('sample.xls', sheet = 1, range = cell_rows(1:4))
```

```
# A tibble: 3 x 5
  channel      users new_users sessions bounce_rate
  <chr>      <dbl>   <dbl>   <dbl> <chr>
1 Organic Search 43296   40238   50810 48.72%
2 Direct        12916   12311   16419 49.27%
3 Referral      10983    7636   18105 22.26%
```

2.6 Read Single Column

If you want to read a single column from the data, use `cell_cols()` and specify the column number. In the below example, we read the second column from the `sample.xls` file.

```
read_excel('sample.xls', sheet = 1, range = cell_cols(2))
```

```
# A tibble: 7 x 1
  users
  <dbl>
1 43296
2 12916
3 10983
4 10346
5  5564
6  2687
7  1773
```

2.7 Read Multiple Columns

In case of multiple columns, we need to specify the column numbers or the column range. In the below example, we want to read the 2nd, 4th and 6th column from the `sample.xls` file.

```
read_excel('sample.xls', sheet = 1, range = cell_cols(c(2, 4, 6)))
```

```
# A tibble: 7 x 5
  users new_users sessions bounce_rate ...5
  <dbl>   <dbl>   <dbl> <chr>    <lgl>
1 43296   40238   50810 48.72%   NA
2 12916   12311   16419 49.27%   NA
3 10983    7636   18105 22.26%   NA
4 10346   10029   11101 61.92%   NA
5  5564    4790    7220 83.30%   NA
6  2687    2205    3438 38.02%   NA
7  1773    1585    2167 55.75%   NA
```

In the next example, we want to read data from the 2nd column upto and including the 6th column.

```
read_excel('sample.xls', sheet = 1, range = cell_cols(c(2:6)))
```

```
# A tibble: 7 x 5
  users new_users sessions bounce_rate ...5
  <dbl>   <dbl>   <dbl> <chr>    <lgl>
1 43296   40238   50810 48.72%   NA
2 12916   12311   16419 49.27%   NA
3 10983    7636   18105 22.26%   NA
4 10346   10029   11101 61.92%   NA
5  5564    4790    7220 83.30%   NA
6  2687    2205    3438 38.02%   NA
7  1773    1585    2167 55.75%   NA
```

2.7.1 Summary

Function	Description
<code>anchored()</code>	Range of cells
<code>cell_limits()</code>	Range of cells
<code>cell_cols()</code>	Columns
<code>cell_rows()</code>	Rows

2.8 Statistical Softwares

We will use the [haven](#) package to read data from files of other statistical softwares such as:

- SAS
- SPSS
- STATA

2.8.1 STATA

```
read_stata('airline.dta')
```

```
# A tibble: 32 x 6
  year      y      w      r      l      k
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  1948  1.21 0.243 0.145  1.41 0.612
2  1949  1.35 0.260 0.218  1.38 0.559
3  1950  1.57 0.278 0.316  1.39 0.573
```

```

4 1951 1.95 0.297 0.394 1.55 0.564
5 1952 2.27 0.310 0.356 1.80 0.574
6 1953 2.73 0.322 0.359 1.93 0.711
7 1954 3.03 0.335 0.403 1.96 0.776
8 1955 3.56 0.350 0.396 2.12 0.827
9 1956 3.98 0.361 0.382 2.43 0.800
10 1957 4.42 0.379 0.305 2.71 0.921
# i 22 more rows

```

2.8.2 SPSS

```
read_spss('employee.sav')
```

```

# A tibble: 474 x 9
      id gender      educ      jobcat salary salbegin jobtime prevexp minority
  <dbl> <chr+lbl> <dbl+lbl> <dbl+l> <dbl+> <dbl+lb> <dbl+l> <dbl+lbl> <dbl+lb>
1     1 m [Male] 15 [15] 3 [Man~ 57000 27000 98 144 0 [No]
2     2 m [Male] 16 [16] 1 [Cle~ 40200 18750 98 36 0 [No]
3     3 f [Female] 12 [12] 1 [Cle~ 21450 12000 98 381 0 [No]
4     4 f [Female] 8 [8] 1 [Cle~ 21900 13200 98 190 0 [No]
5     5 m [Male] 15 [15] 1 [Cle~ 45000 21000 98 138 0 [No]
6     6 m [Male] 15 [15] 1 [Cle~ 32100 13500 98 67 0 [No]
7     7 m [Male] 15 [15] 1 [Cle~ 36000 18750 98 114 0 [No]
8     8 f [Female] 12 [12] 1 [Cle~ 21900 9750 98 0 [mis~ 0 [No]
9     9 f [Female] 15 [15] 1 [Cle~ 27900 12750 98 115 0 [No]
10    10 f [Female] 12 [12] 1 [Cle~ 24000 13500 98 244 0 [No]
# i 464 more rows

```

2.8.3 SAS

```
read_sas('airline.sas7bdat')
```

```

# A tibble: 32 x 6
  YEAR      Y      W      R      L      K
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 1948 1.21 0.243 0.145 1.41 0.612
2 1949 1.35 0.260 0.218 1.38 0.559
3 1950 1.57 0.278 0.316 1.39 0.573
4 1951 1.95 0.297 0.394 1.55 0.564
5 1952 2.27 0.310 0.356 1.80 0.574
6 1953 2.73 0.322 0.359 1.93 0.711

```

```

7  1954  3.03 0.335 0.403  1.96 0.776
8  1955  3.56 0.350 0.396  2.12 0.827
9  1956  3.98 0.361 0.382  2.43 0.800
10 1957  4.42 0.379 0.305  2.71 0.921
# i 22 more rows

```

2.9 Summary

File Type	readr	foreign/sas7bdat
excel	<code>read_excel()</code>	
sas	<code>read_sas()</code>	<code>read.sas7bdat()</code>
spss	<code>read_sav()</code> / <code>read_spss()</code>	<code>read.spss()</code>
stata	<code>read_dta()</code> / <code>read_stata()</code>	<code>read.dta()</code>

3 Pipe Operator

3.1 Introduction

R code contain a lot of parentheses in case of a sequence of multiple operations. When you are dealing with complex code, it results in nested function calls which are hard to read and maintain. The [magrittr](#) package by [Stefan Milton Bache](#) provides pipes enabling us to write R code that is readable.

Pipes allow us to clearly express a sequence of multiple operations by:

- structuring operations from left to right
- avoiding
 - nested function calls
 - intermediate steps
 - overwriting of original data
- minimizing creation of local variables

3.2 Pipes

The examples in this chapter use the base R pipe `|>`, available in R 4.1 and later with no extra package. It forwards the value on its left into the first argument of the function on its right, letting us read a sequence of operations from left to right (or top to bottom).

i Legacy note

Older code — including the first edition of this chapter — used the `magrittr` pipe `%>%` and its companions documented in `?magrittr::operators`. Those operators are kept only for reading inherited code; every example below uses `|>` together with `pull()`/`pluck()` and base assignment instead (see the data-extraction section).

3.2.1 `|>` vs `%>%` at a glance

Topic	Base pipe <code> ></code> (this book)	magrittr <code>%>%</code> (inherited code)
Package	None — syntax in R $\geq$ 4.1	magrittr (loaded with the tidyverse)
How it works	Implemented in the R parser; no runtime pipe object	A regular function call, with a small overhead per step
Default	Forwards into the first argument	Forwards into the first argument
Other argument	Placeholder <code>_</code> : <code>mtcars > lm(mpg ~ disp, data = _)</code>	Dot <code>.</code> : <code>mtcars %>% lm(mpg ~ disp, data = .)</code>
Inline function	<code>\(x): 1:5 > \(x) x^2() > sum()</code>	. shorthand: <code>1:5 %>% (function(x) x^2) %>% sum()</code>

For first-argument chains — every example in this book — the two pipes read identically, so inherited `%>%` code can be read as `|>`:

```
# placeholder: pipe into a non-first argument
mtcars |>
  lm(mpg ~ disp, data = _) |>
  coef()
```

```
(Intercept)      disp
29.59985476 -0.04121512
```

```
# shorthand function of one argument, then continue the chain
1:5 |>
  \(x) x^2() |>
  sum()
```

```
[1] 55
```

We will use the following R packages:

```
library(magrittr)
library(readr)
library(dplyr)
library(stringr)
library(purrr)
```

3.3 Data

```
ecom <-  
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',  
    col_types = cols_only(  
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),  
      n_pages = col_double(), duration = col_double(), purchase = col_logical()  
    )  
  )  
  
ecom
```

```
# A tibble: 1,000 x 4  
  referrer n_pages duration purchase  
  <fct>      <dbl>    <dbl> <lgl>  
1 google         1      693 FALSE  
2 yahoo          1      459 FALSE  
3 direct         1      996 FALSE  
4 bing          18      468 TRUE  
5 yahoo          1      955 FALSE  
6 yahoo          5      135 FALSE  
7 yahoo          1       75 FALSE  
8 direct         1      908 FALSE  
9 bing          19      209 FALSE  
10 google         1      208 FALSE  
# i 990 more rows
```

We will create a smaller data set from the above data to be used in some examples:

```
ecom_mini <- slice_sample(ecom, n = 10)  
ecom_mini
```

```
# A tibble: 10 x 4  
  referrer n_pages duration purchase  
  <fct>      <dbl>    <dbl> <lgl>  
1 google         18      252 TRUE  
2 social          2       60 FALSE  
3 bing            1      751 FALSE  
4 google         10      240 FALSE  
5 yahoo           7      168 FALSE  
6 yahoo          20      200 FALSE  
7 google          1      934 FALSE  
8 direct          9      135 FALSE  
9 direct          1      433 FALSE  
10 direct         18      540 FALSE
```

3.3.1 Data Dictionary

- referrer: referrer website/search engine
- n_pages: number of pages visited
- duration: time spent on the website (in seconds)
- purchase: whether visitor purchased

3.4 head()

Let us start with a simple example. You must be aware of `head()`. If not, do not worry. It returns the first few observations/rows of data. We can specify the number of observations it should return as well. Let us use it to view the first 10 rows of our data set.

```
head(ecom, 10)
```

```
# A tibble: 10 x 4
  referrer n_pages duration purchase
  <fct>      <dbl>    <dbl> <lgl>
1 google         1      693 FALSE
2 yahoo          1      459 FALSE
3 direct         1      996 FALSE
4 bing          18      468  TRUE
5 yahoo          1      955 FALSE
6 yahoo          5      135 FALSE
7 yahoo          1       75 FALSE
8 direct         1      908 FALSE
9 bing          19      209 FALSE
10 google         1      208 FALSE
```

3.4.1 Using Pipe

Now let us do the same but with `|>`.

```
ecom |> head(10)
```

```
# A tibble: 10 x 4
  referrer n_pages duration purchase
  <fct>      <dbl>    <dbl> <lgl>
1 google         1      693 FALSE
2 yahoo          1      459 FALSE
3 direct         1      996 FALSE
4 bing          18      468  TRUE
```

5	yahoo	1	955	FALSE
6	yahoo	5	135	FALSE
7	yahoo	1	75	FALSE
8	direct	1	908	FALSE
9	bing	19	209	FALSE
10	google	1	208	FALSE

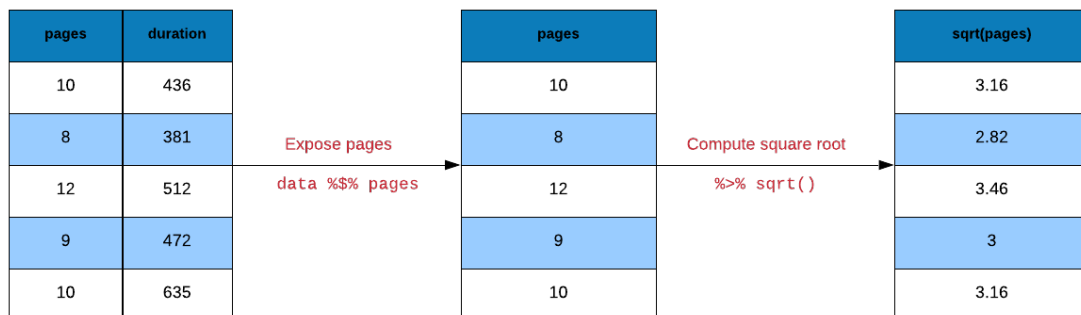
3.5 Square Root

Time to try a slightly more challenging example. We want the square root of `n_pages` column from the data set.

```
y <- sqrt(ecom_mini$n_pages)
```

Let us break down the above computation into small steps:

- select/expose the `n_pages` column from `ecom` data
- compute the square root
- assign the first few observations to `y`



Let us reproduce `y` using pipes.

```
# extract the n_pages column and assign it to y
y <-
  ecom_mini |>
  pull(n_pages)

# compute the square root of y and assign it back to y
y <- sqrt(y)
```

Another way to compute the square root of `y` is shown below.

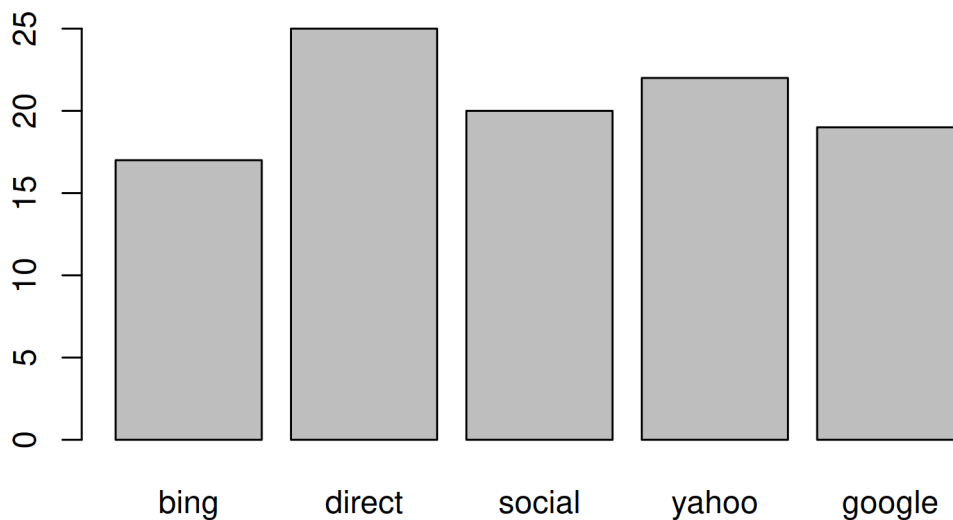
```
y <-  
ecom_mini |>  
pull(n_pages) |>  
sqrt()
```

3.6 Visualization

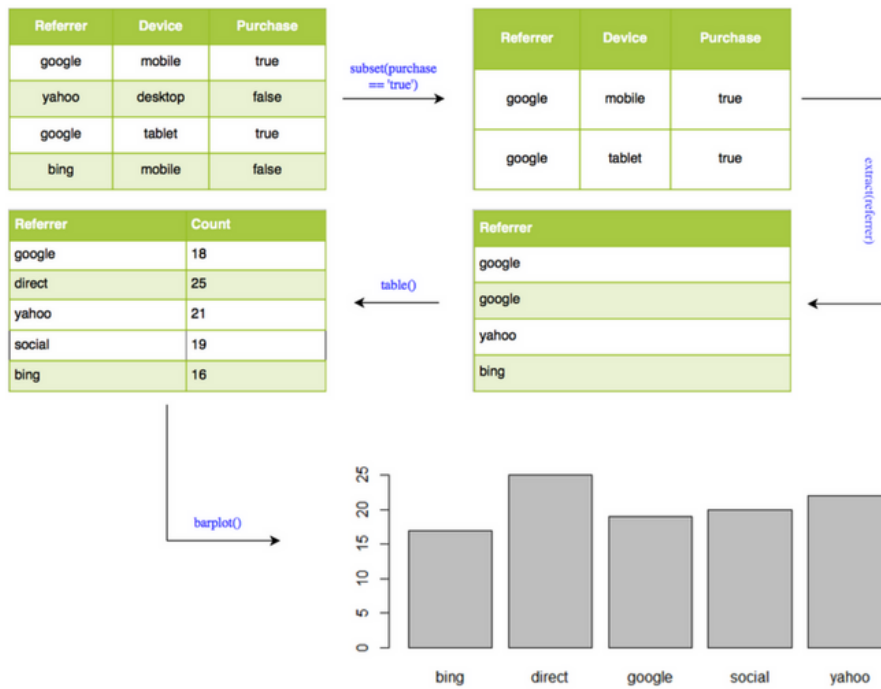
Let us look at a data visualization example. We will create a bar plot to visualize the frequency of different referrer types that drove purchasers to the website. Let us look at the steps involved in creating the bar plot:

- extract rows where purchase is TRUE
- select/expose **referrer** column
- tabulate referrer data using `table()`
- use the tabulated data to create bar plot using `barplot()`

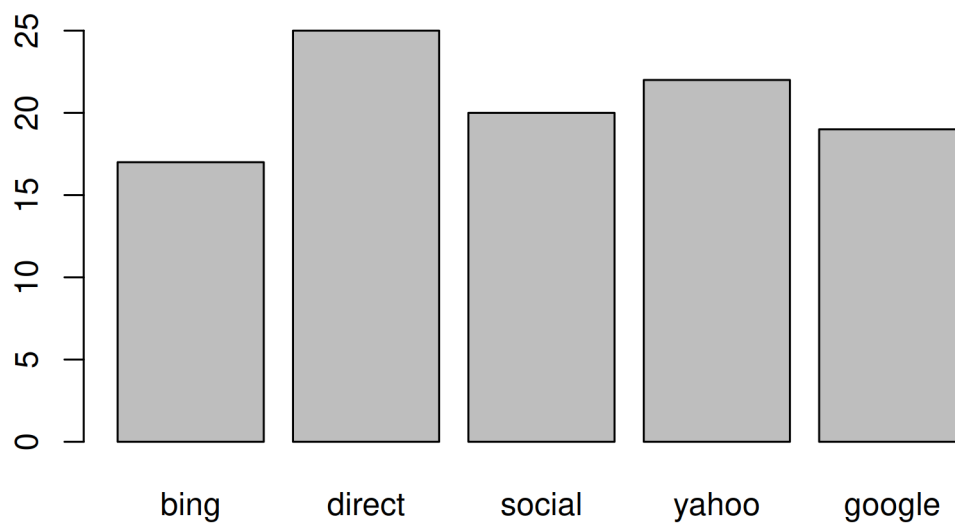
```
barplot(table(subset(ecom, purchase)$referrer))
```



3.6.1 Using pipe



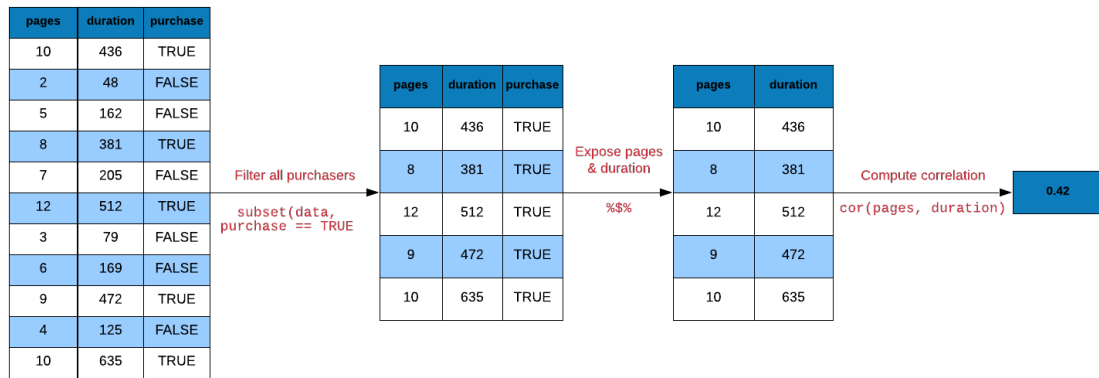
```
ecom |>
  subset(purchase) |>
  pull('referrer') |>
  table() |>
  barplot()
```



3.7 Correlation

Correlation is a statistical measure that indicates the extent to which two or more variables fluctuate together. In R, correlation is computed using `cor()`. Let us look at the correlation between the number of pages browsed and time spent on the site for visitors who purchased some product. Below are the steps for computing correlation:

- extract rows where purchase is TRUE
- select/expose `n_pages` and `duration` columns
- use `cor()` to compute the correlation



```
# without pipe
ecom1 <- subset(ecom, purchase)
cor(ecom1$n_pages, ecom1$duration)
```

```
[1] 0.4290905
```

```
# with pipe
ecom |>
  filter(purchase) |>
  summarise(cor = cor(n_pages, duration))
```

```
# A tibble: 1 x 1
  cor
  <dbl>
1 0.429
```

3.8 Regression

Let us look at a regression example. We regress time spent on the site on number of pages visited. Below are the steps involved in running the regression:

- use `duration` and `n_pages` columns from `ecom` data
- pass the above data to `lm()`
- pass the output from `lm()` to `summary()`

```
summary(lm(duration ~ n_pages, data = ecom))
```

```
Call:
lm(formula = duration ~ n_pages, data = ecom)

Residuals:
    Min       1Q   Median       3Q      Max
-386.45 -213.03  -38.93  179.31  602.55

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  404.803     11.323   35.750 < 2e-16 ***
n_pages       -8.355       1.296   -6.449 1.76e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 263.3 on 998 degrees of freedom
Multiple R-squared:  0.04, Adjusted R-squared:  0.03904
F-statistic: 41.58 on 1 and 998 DF, p-value: 1.756e-10
```

3.8.1 Using pipe

```
ecom |>
  with(lm(duration ~ n_pages)) |>
  summary()
```

```
Call:
lm(formula = duration ~ n_pages)

Residuals:
    Min       1Q   Median       3Q      Max
-386.45 -213.03  -38.93  179.31  602.55

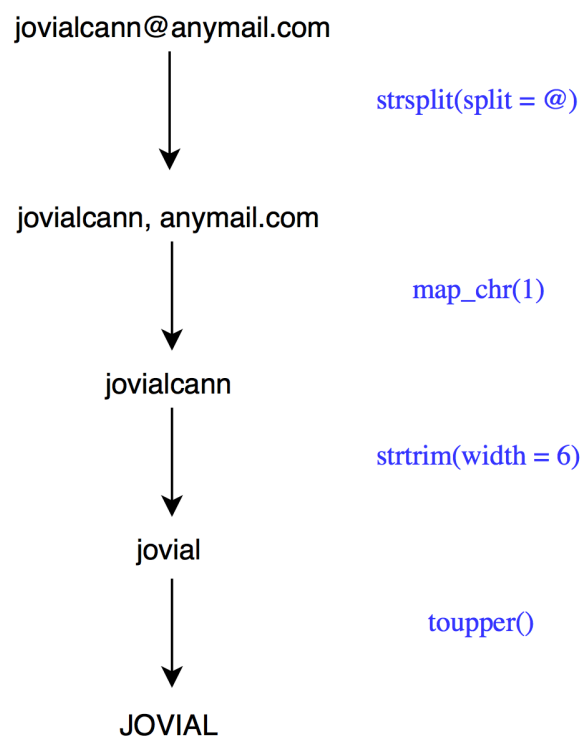
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  404.803     11.323   35.750 < 2e-16 ***
n_pages       -8.355       1.296   -6.449 1.76e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 263.3 on 998 degrees of freedom
Multiple R-squared:  0.04, Adjusted R-squared:  0.03904
F-statistic: 41.58 on 1 and 998 DF, p-value: 1.756e-10
```

3.9 String Manipulation

We want to extract the first name (jovial) from the below email id and convert it to upper case. Below are the steps to achieve this:

- split the email id using the pattern @ using `str_split()`
- extract the first element of the resulting list and the first element of the character vector using `pluck()`
- extract the first six characters using `str_sub()`
- convert to upper case using `str_to_upper()`



```
email <- 'jovialcann@anymail.com'

# without pipe
str_to_upper(str_sub(str_split(email, '@')[[1]][1], start = 1, end = 6))
```

```
[1] "JOVIAL"
```

```
# with pipe
email |>
  str_split(pattern = '@') |>
  pluck(1, 1) |>
```

```
str_sub(start = 1, end = 6) |>
str_to_upper()
```

```
[1] "JOVIAL"
```

Another method that uses `map_chr()` from the `purrr` package.

```
email |>
  str_split(pattern = '@') |>
  map_chr(1) |>
  str_sub(start = 1, end = 6) |>
  str_to_upper()
```

```
[1] "JOVIAL"
```

3.10 Data Extraction

Let us turn our attention towards data extraction. `dplyr` and `purrr` provide `pull()` and `pluck()` as readable alternatives to `$`, `[` and `[[`:

- `pull()` extracts a column as a vector (like `$`)
- `pluck()` extracts an element from a list or vector (like `[[`)

3.10.1 Extract Column

To extract a specific column using the column name, we mention the name of the column in single/double quotes within `[` or `[[`. In case of `$`, we do not use quotes.

```
# base
ecom_mini['n_pages']
```

```
# A tibble: 10 x 1
  n_pages
  <dbl>
1      18
2       2
3       1
4      10
5       7
6      20
7       1
```

8	9
9	1
10	18

```
# tidyverse
select(ecom_mini, n_pages)
```

```
# A tibble: 10 x 1
  n_pages
  <dbl>
1      18
2       2
3       1
4      10
5       7
6      20
7       1
8       9
9       1
10      18
```

We can extract columns using their index position. Keep in mind that index position starts from 1 in R. In the below example, we show how to extract `n_pages` column but instead of using the column name, we use the column position.

```
# base
ecom_mini[2]
```

```
# A tibble: 10 x 1
  n_pages
  <dbl>
1      18
2       2
3       1
4      10
5       7
6      20
7       1
8       9
9       1
10      18
```

```
# tidyverse
select(ecom_mini, 2)
```

```
# A tibble: 10 x 1
  n_pages
  <dbl>
1      18
2       2
3       1
4      10
5       7
6      20
7       1
8       9
9       1
10     18
```

One important differentiator between `[]` and `[[` is that `[[` will return an atomic vector and not a `data.frame`. `$` will also return an atomic vector. In the tidyverse, we use `pull()` in place of `$`.

```
# base
ecom_mini$n_pages
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

```
# tidyverse
pull(ecom_mini, n_pages)
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

3.10.2 Extract List Element

Let us convert `ecom_mini` into a list using `as.list()` as shown below:

```
ecom_list <- as.list(ecom_mini)
```

To extract elements of a list, we use `pluck()`. It is the tidyverse alternative for `[[`.

```
# base
ecom_list[['n_pages']]
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

```
# tidyverse  
pluck(ecom_list, 'n_pages')
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

```
# base  
ecom_list[[1]]
```

```
[1] google social bing  google yahoo  yahoo  google direct direct direct  
Levels: bing direct social yahoo google
```

```
# tidyverse  
pluck(ecom_list, 1)
```

```
[1] google social bing  google yahoo  yahoo  google direct direct direct  
Levels: bing direct social yahoo google
```

We can extract the elements of a list by name using `pluck()` as well.

```
# base  
ecom_list$n_pages
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

```
# tidyverse  
pluck(ecom_list, "n_pages")
```

```
[1] 18  2  1 10  7 20  1  9  1 18
```

3.11 Arithmetic Operations

`magrittr` offer alternatives for arithmetic operations as well. We will look at a few examples below.

- `add()`
- `subtract()`
- `multiply_by()`
- `multiply_by_matrix()`
- `divide_by()`
- `divide_by_int()`
- `mod()`
- `raise_to_power()`

3.11.1 Addition

```
1:10 + 1
```

```
[1]  2  3  4  5  6  7  8  9 10 11
```

```
add(1:10, 1)
```

```
[1]  2  3  4  5  6  7  8  9 10 11
```

```
`+`(1:10, 1)
```

```
[1]  2  3  4  5  6  7  8  9 10 11
```

3.11.2 Multiplication

```
1:10 * 3
```

```
[1]  3  6  9 12 15 18 21 24 27 30
```

```
multiply_by(1:10, 3)
```

```
[1]  3  6  9 12 15 18 21 24 27 30
```

```
`*`(1:10, 3)
```

```
[1]  3  6  9 12 15 18 21 24 27 30
```

3.11.3 Division

```
1:10 / 2
```

```
[1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

```
divide_by(1:10, 2)
```

```
[1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

```
`/`(1:10, 2)
```

```
[1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

3.11.4 Power

```
1:10 ^ 2
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
raise_to_power(1:10, 2)
```

```
[1] 1 4 9 16 25 36 49 64 81 100
```

```
`^^`(1:10, 2)
```

```
[1] 1 4 9 16 25 36 49 64 81 100
```

3.12 Logical Operators

There are alternatives for logical operators as well. We will look at a few examples below.

- `and()`
- `or()`
- `equals()`
- `not()`
- `is_greater_than()`
- `is_weakly_greater_than()`
- `is_less_than()`
- `is_weakly_less_than()`

3.12.1 Greater Than

```
1:10 > 5
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
```

```
is_greater_than(1:10, 5)
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
```

```
`>`(1:10, 5)
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
```

3.12.2 Weakly Greater Than

```
1:10 >= 5
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
is_weakly_greater_than(1:10, 5)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
`>=`(1:10, 5)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

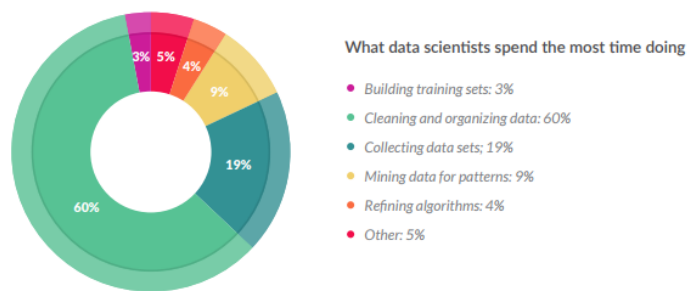
4 dplyr Basics

4.1 Introduction

According to a [survey](#) by [CrowdFlower](#), data scientists spend most of their time cleaning and manipulating data rather than mining or modeling them for insights. As such, it becomes important to have tools that make data manipulation faster and easier. In today's chapter, we introduce you to [dplyr](#), a grammar of data manipulation.

How a Data Scientist Spends Their Day

Here's where the popular view of data scientists diverges pretty significantly from reality. Generally, we think of data scientists building algorithms, exploring data, and doing predictive analysis. That's actually not what they spend most of their time doing, however.



As you can see from the chart above, 3 out of every 5 data scientists we surveyed actually spend the most time cleaning and organizing data. You may have heard this referred to as "data wrangling" or compared to digital janitor work. Everything from list verification to removing commas to debugging databases—that time adds up and it adds up immensely. Messy data is by far the more time-consuming aspect of the typical data scientist's work flow. And nearly 60% said they simply spent too much time doing it.



FINDINGS 6

We will use the following R packages:

```
library(dplyr)
library(readr)
```

4.2 dplyr Verbs

dplyr provides a set of verbs that help us solve the most common data manipulation challenges while working with tabular data (dataframes, tibbles):

- select
- filter
- arrange
- mutate
- summarise

4.3 Data

```
ecom <-  
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',  
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),  
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),  
      purchase = col_logical(), n_pages = col_double(), n_visit = col_double(),  
      duration = col_double(), order_value = col_double(), order_items = col_double()  
    )  
  )  
ecom
```

```
# A tibble: 1,000 x 8  
  referrer device n_visit n_pages duration purchase order_items order_value  
  <fct>      <fct>   <dbl>  <dbl>   <dbl> <lgl>         <dbl>         <dbl>  
1 google    laptop     10      1     693 FALSE           0           0  
2 yahoo     tablet      9      1     459 FALSE           0           0  
3 direct    laptop      0      1     996 FALSE           0           0  
4 bing      tablet      3     18     468 TRUE            6          434  
5 yahoo     mobile      9      1     955 FALSE           0           0  
6 yahoo     laptop      5      5     135 FALSE           0           0  
7 yahoo     mobile     10      1      75 FALSE           0           0  
8 direct    mobile     10      1     908 FALSE           0           0  
9 bing      mobile      3     19     209 FALSE           0           0  
10 google   mobile      6      1     208 FALSE           0           0  
# i 990 more rows
```

4.3.1 Data Dictionary

Below is the description of the data set:

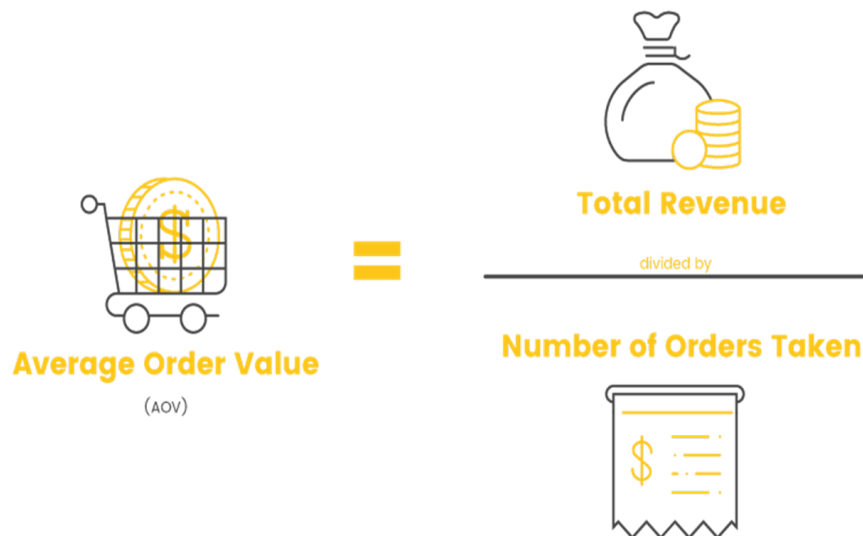
- referrer: referrer website/search engine
- device: device used to visit the website
- n_pages: number of pages visited
- duration: time spent on the website (in seconds)
- purchase: whether visitor purchased
- order_value: order value of visitor (in dollars)
- n_visit: number of visits

4.4 Case Study

We will use dplyr to answer the following:

- what is the average order value by device types?
- what is the average number of pages visited by purchasers and non-purchasers?
- what is the average time on site for purchasers vs non-purchasers?
- what is the average number of pages visited by purchasers and non-purchasers using mobile?

4.5 Average Order Value



4.6 AOV by Devices

Reading pipes

`|>` forwards the tibble on its left into the first argument of the verb on its right (see the Preface primer). Every verb below returns a tibble, so the chain reads as one sentence: keep purchasers, summarise revenue and orders by device, add AOV, keep two columns, sort.

```
ecom |>
  filter(purchase) |>
  summarise(revenue = sum(order_value), orders = n(), .by = device) |>
  mutate(
    aov = revenue / orders
  ) |>
  select(device, aov) |>
  arrange(aov)
```

```
# A tibble: 3 x 2
  device  aov
  <fct>  <dbl>
1 tablet 1426.
2 mobile 1431.
3 laptop 1824.
```

4.7 Syntax

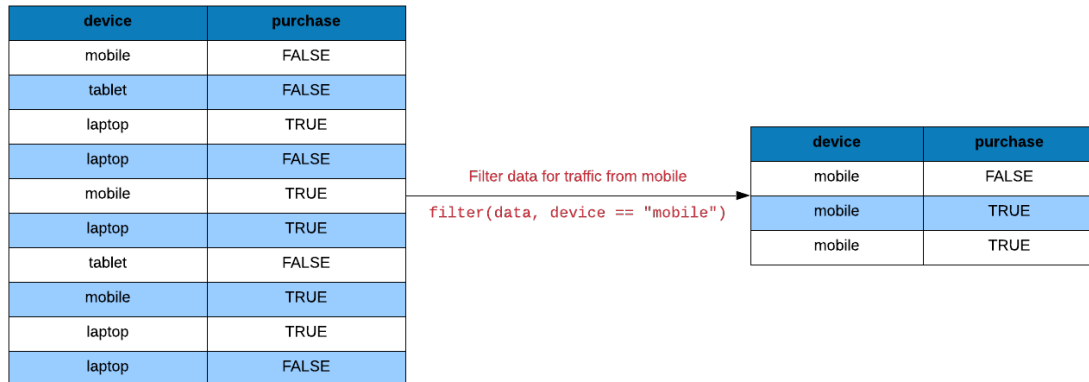
Before we start exploring the dplyr verbs, let us look at their syntax:

- the first argument is always a `data.frame` or `tibble`
- the subsequent arguments provide the information required for the verbs to take action
- the name of columns in the data need not be surrounded by quotes

4.8 Filter Rows

In order to compute the AOV, we must first separate the purchasers from non-purchasers. We will do this by filtering the data related to purchasers using the `filter()` function. It allows us to filter rows that meet a specific criteria/condition. The first argument is the name of the data frame and the rest of the arguments are expressions for filtering the data. Let us look at a few examples:

The first example we will look at filters all visits from device **mobile**. As we had learnt in the previous section, the first argument is our data set **ecom** and the next argument is the condition for filtering rows.



```
filter(ecom, device == "mobile")
```

```
# A tibble: 344 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>    <fct>   <dbl>  <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 yahoo    mobile     9      1     955 FALSE          0           0
2 yahoo    mobile    10      1      75 FALSE          0           0
3 direct   mobile    10      1     908 FALSE          0           0
4 bing     mobile     3     19     209 FALSE          0           0
5 google   mobile     6      1     208 FALSE          0           0
6 direct   mobile     9     14     406 TRUE           3          651
7 yahoo    mobile     7      1      19 FALSE          7         2423
8 google   mobile     5      1     147 FALSE          0           0
9 bing     mobile     0      7     196 FALSE          4          237
10 google   mobile    10      1     338 FALSE          0           0
# i 334 more rows
```

We can specify multiple filtering conditions as well. In the below example, we specify two filter conditions:

- visit from device **mobile**
- resulted in a purchase or conversion



```
filter(ecom, device == "mobile", purchase)
```

```
# A tibble: 36 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>      <fct>   <dbl>   <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 direct   mobile     9      14     406 TRUE          3          651
2 bing     mobile     4      20     440 TRUE          3          184
3 bing     mobile     3      18     288 TRUE          6          764
4 social   mobile    10      11     242 TRUE          4          287
5 yahoo    mobile     6      14     322 TRUE          3         1443
6 google   mobile     1      18     252 TRUE          3         2449
7 social   mobile     7      16     352 TRUE         10         2824
8 direct   mobile     4      18     324 TRUE          3         1670
9 social   mobile     1      20     520 TRUE          5         1021
10 yahoo    mobile     0      13     351 TRUE         10          288
# i 26 more rows
```

Here is another example where we specify multiple conditions:

- visit from device **tablet**
- made a purchase
- browsed less than 15 pages

```
filter(ecom, device == "tablet", purchase, n_pages < 15)
```

```
# A tibble: 12 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>      <fct>   <dbl>   <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 social   tablet     7      10     290 TRUE          9         1304
2 yahoo    tablet     2      14     364 TRUE          6         1667
3 google   tablet     7      12     324 TRUE          2         1358
4 direct   tablet     3      12     324 TRUE         10         1257
```

5	yahoo	tablet	0	13	390	TRUE	5	1748
6	social	tablet	2	12	300	TRUE	2	2754
7	direct	tablet	6	13	338	TRUE	5	683
8	yahoo	tablet	2	10	280	TRUE	4	293
9	social	tablet	10	10	290	TRUE	9	37
10	direct	tablet	3	10	260	TRUE	7	980
11	google	tablet	9	14	308	TRUE	7	2436
12	social	tablet	10	11	330	TRUE	1	2171

4.8.1 Case Study

Let us apply what we have learnt to the case study. We want to filter all visits that resulted in a purchase.

```
filter(ecom, purchase)
```

```
# A tibble: 103 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>      <fct>   <dbl>   <dbl>   <dbl> <lgl>      <dbl>      <dbl>
1 bing      tablet     3      18     468 TRUE         6        434
2 direct    mobile     9      14     406 TRUE         3        651
3 bing      tablet     5      16     368 TRUE         6       1049
4 social    tablet     7      10     290 TRUE         9       1304
5 direct    tablet     2      19     342 TRUE         5        622
6 social    tablet     9      20     420 TRUE         7       1613
7 bing      mobile     4      20     440 TRUE         3        184
8 yahoo     tablet     2      16     480 TRUE         9        286
9 bing      mobile     3      18     288 TRUE         6        764
10 yahoo    tablet     2      14     364 TRUE         6       1667
# i 93 more rows
```

4.9 Select Columns

After filtering the data, we need to select relevant variables to compute the AOV. Remember, we do not need all the columns in the data to compute a required metric (in our case, AOV). The `select()` function allows us to select a subset of columns. The first argument is the name of the data frame and the subsequent arguments specify the columns by name or position.

To select the `device` and `duration` columns, we specify the data set i.e. `ecom` followed by the name of the columns.

id	referrer	device	purchase	duration
VF001	google	mobile	FALSE	32
VF002	social	tablet	FALSE	56
VF003	direct	laptop	TRUE	306
VF004	facebook	laptop	FALSE	100
VF005	affiliate	mobile	TRUE	341
VF006	google	laptop	TRUE	432

Select device and purchase columns
select(data, device, purchase)

device	purchase
mobile	FALSE
tablet	FALSE
laptop	TRUE
laptop	FALSE
mobile	TRUE
laptop	TRUE

```
select(ecom, device, duration)
```

```
# A tibble: 1,000 x 2
  device duration
  <fct>      <dbl>
1 laptop     693
2 tablet     459
3 laptop     996
4 tablet     468
5 mobile     955
6 laptop     135
7 mobile      75
8 mobile     908
9 mobile     209
10 mobile     208
# i 990 more rows
```

We can select a set of columns using `:`. In the below example, we select all the columns starting from `referrer` up to `order_items`. Remember that we can use `:` only when the columns are adjacent to each other in the data set.

id	referrer	device	purchase	duration
VF001	google	mobile	FALSE	32
VF002	social	tablet	FALSE	56
VF003	direct	laptop	TRUE	306
VF004	facebook	laptop	FALSE	100
VF005	affiliate	mobile	TRUE	341
VF006	google	laptop	TRUE	432

Select all columns from referrer till purchase
select(data, referrer:purchase)

referrer	device	purchase
google	mobile	FALSE
social	tablet	FALSE
direct	laptop	TRUE
facebook	laptop	FALSE
affiliate	mobile	TRUE
google	laptop	TRUE

```
select(ecom, referrer:order_items)
```

```
# A tibble: 1,000 x 7
  referrer device n_visit n_pages duration purchase order_items
  <fct>      <fct>   <dbl>  <dbl>    <dbl> <lgl>      <dbl>
```

```

1 google    laptop    10      1      693 FALSE      0
2 yahoo     tablet     9      1      459 FALSE      0
3 direct    laptop     0      1      996 FALSE      0
4 bing      tablet     3     18      468 TRUE       6
5 yahoo     mobile     9      1      955 FALSE      0
6 yahoo     laptop     5      5      135 FALSE      0
7 yahoo     mobile    10      1       75 FALSE      0
8 direct    mobile    10      1      908 FALSE      0
9 bing      mobile     3     19      209 FALSE      0
10 google   mobile     6      1      208 FALSE      0
# i 990 more rows

```

What if you want to select all columns except a few? Typing the name of many columns can be cumbersome and may also result in spelling errors. We may use `:` only if the columns are adjacent to each other but that may not always be the case. `dplyr` allows us to specify columns that need not be selected using `-`. In the below example, we select all columns except `n_pages` and `duration`. Notice the `-` before both of them.

id	referrer	device	purchase	duration
VF001	google	mobile	FALSE	32
VF002	social	tablet	FALSE	56
VF003	direct	laptop	TRUE	306
VF004	facebook	laptop	FALSE	100
VF005	affiliate	mobile	TRUE	341
VF006	google	laptop	TRUE	432

Select all columns except id and duration
`select(data, -id, -duration)`

referrer	device	purchase
google	mobile	FALSE
social	tablet	FALSE
direct	laptop	TRUE
facebook	laptop	FALSE
affiliate	mobile	TRUE
google	laptop	TRUE

```
select(ecom, -n_pages, -duration)
```

```

# A tibble: 1,000 x 6
  referrer device n_visit purchase order_items order_value
  <fct>      <fct>   <dbl> <lgl>          <dbl>         <dbl>
1 google    laptop     10 FALSE          0             0
2 yahoo     tablet     9 FALSE          0             0
3 direct    laptop     0 FALSE          0             0
4 bing      tablet     3 TRUE           6            434
5 yahoo     mobile     9 FALSE          0             0
6 yahoo     laptop     5 FALSE          0             0
7 yahoo     mobile    10 FALSE          0             0
8 direct    mobile    10 FALSE          0             0
9 bing      mobile     3 FALSE          0             0
10 google   mobile     6 FALSE          0             0
# i 990 more rows

```

4.9.1 Case Study

For our case study, we need to select the column `order_value` to calculate the AOV. We also need to select the `device` column as we are computing the AOV for each device type.

```
select(ecom, device, order_value)
```

```
# A tibble: 1,000 x 2
  device order_value
  <fct>      <dbl>
1 laptop         0
2 tablet         0
3 laptop         0
4 tablet        434
5 mobile         0
6 laptop         0
7 mobile         0
8 mobile         0
9 mobile         0
10 mobile        0
# i 990 more rows
```

But we want the above data only for purchasers. Let us combine `filter()` and `select()` functions to extract `order_value` and `order_items` only for those visits that resulted in a purchase.

```
# filter all visits that resulted in a purchase
ecom1 <- filter(ecom, purchase)

# select the relevant columns
ecom2 <- select(ecom1, device, order_value)

# view data
ecom2
```

```
# A tibble: 103 x 2
  device order_value
  <fct>      <dbl>
1 tablet        434
2 mobile        651
3 tablet       1049
4 tablet       1304
5 tablet        622
6 tablet       1613
```

```

7 mobile          184
8 tablet          286
9 mobile          764
10 tablet         1667
# i 93 more rows

```

4.10 Grouping Data

We need to compute the total order value and total order items for each device in order to compute their AOV. To achieve this, we need to group the selected `order_value` and `order_items` by device type. `group_by()` allows us to group or split data based on particular (discrete) variable. The first argument is the name of the data set and the second argument is the name of the column based on which the data will be split.

To split the data by referrer type, we use `group_by` and specify the data set i.e. `ecom` and the column based on which to split the data i.e. `referrer`.

```
group_by(ecom, referrer)
```

```

# A tibble: 1,000 x 8
# Groups:   referrer [5]
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>     <fct>   <dbl>   <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 google   laptop     10      1     693 FALSE          0          0
2 yahoo    tablet      9      1     459 FALSE          0          0
3 direct   laptop      0      1     996 FALSE          0          0
4 bing     tablet      3     18     468 TRUE           6         434
5 yahoo    mobile      9      1     955 FALSE          0          0
6 yahoo    laptop      5      5     135 FALSE          0          0
7 yahoo    mobile     10      1      75 FALSE          0          0
8 direct   mobile     10      1     908 FALSE          0          0
9 bing     mobile      3     19     209 FALSE          0          0
10 google   mobile      6      1     208 FALSE          0          0
# i 990 more rows

```

4.10.1 Case Study

In the second line in the previous output, you can observe `Groups: referrer [5]`. The data is split into 5 groups as the referrer variable has 5 distinct values. For our case study, we need to group the data by device type.

```

# split ecom2 by device type
ecom3 <- group_by(ecom2, device)
ecom3

```

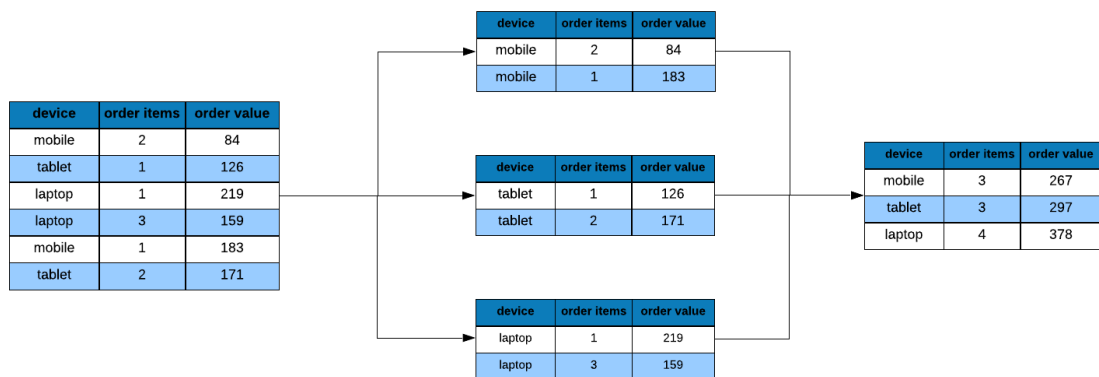
```
# A tibble: 103 x 2
# Groups:   device [3]
  device order_value
  <fct>      <dbl>
1 tablet      434
2 mobile      651
3 tablet     1049
4 tablet     1304
5 tablet      622
6 tablet     1613
7 mobile      184
8 tablet      286
9 mobile      764
10 tablet    1667
# i 93 more rows
```

4.11 Summarise Data

The next step is to compute the total order value and total order items for each device. i.e. we need to reduce the order value and order items data to a single summary. We can achieve this using `summarise()`. As usual, the first argument is the name of a data set and the subsequent arguments are functions that can summarise data. For example, we can use `min`, `max`, `sum`, `mean` etc.

Let us compute the average number of pages browsed by referrer type:

- split data by `referrer` type
- compute the average number of pages using `mean`



```
# split data by referrer type
step_1 <- group_by(ecom, referrer)

# compute average number of pages
step_2 <- summarise(step_1, mean(n_pages))
step_2
```

```
# A tibble: 5 x 2
  referrer `mean(n_pages)`
  <fct>      <dbl>
1 bing      6.13
2 direct    6.38
3 social    5.42
4 yahoo     5.99
5 google    5.73
```

Now let us compute both the mean and the median.

```
# split data by referrer type
step_1 <- group_by(ecom, referrer)

# compute average number of pages
step_2 <- summarise(step_1, mean(n_pages), median(n_pages))
step_2
```

```
# A tibble: 5 x 3
  referrer `mean(n_pages)` `median(n_pages)`
  <fct>      <dbl>      <dbl>
1 bing      6.13          1
2 direct    6.38          1
3 social    5.42          1
4 yahoo     5.99          2
5 google    5.73          1
```

The two steps above — split with `group_by()`, then `summarise()` — can also be written in one step with the `.by` argument, which groups only for that verb and leaves the data ungrouped afterwards:

```
# compute average number of pages by referrer without group_by()
step_2 <- summarise(ecom, mean(n_pages), median(n_pages), .by = referrer)
step_2
```

```
# A tibble: 5 x 3
  referrer `mean(n_pages)` `median(n_pages)`
  <fct>      <dbl>         <dbl>
1 google      5.73           1
2 yahoo       5.99           2
3 direct      6.38           1
4 bing        6.13           1
5 social      5.42           1
```

One difference to know: `group_by()` sorts the result by group, while `.by` keeps the rows in order of first appearance. Use `arrange()` afterwards when the order matters.

Another way to achieve the above result is to use `across()` inside `summarise()`. It applies the specified summary function to all the columns in the data set except for the grouping column. So we need to ensure that the data does not have any irrelevant columns.

- split data by `referrer` type
- select `order_value` and `order_items`
- compute the average number of pages by applying the `mean` function to all the columns

```
# select relevant columns
step_1 <- select(ecom, referrer, order_value)

# split data by referrer type
step_2 <- group_by(step_1, referrer)

# compute average number of pages
step_3 <- summarise(step_2, across(everything(), mean))
step_3
```

```
# A tibble: 5 x 2
  referrer order_value
  <fct>      <dbl>
1 bing      316.
2 direct    441.
3 social    380.
4 yahoo     470.
5 google    328.
```

Let us compute `mean` and `median` number of pages for each referre type using `summarise()` with `across()`.

```
# select relevant columns
step_1 <- select(ecom, referrer, order_value)

# split data by referrer type
step_2 <- group_by(step_1, referrer)

# compute mean and median number of pages
step_3 <- summarise(step_2, across(everything(), list(mean = mean, median = median)))
step_3
```

```
# A tibble: 5 x 3
  referrer order_value_mean order_value_median
  <fct>         <dbl>         <dbl>
1 bing          316.             0
2 direct        441.             0
3 social        380.             0
4 yahoo         470.             0
5 google        328.             0
```

4.11.1 Case Study

So far, we have split the data based on the device type and we have selected 2 columns, `order_value` and `order_items`. We need the sum of order value and order items. What function can we use to obtain them? The `sum()` function will generate the sum of the values and hence we will use it inside the `summarise()` function. Remember, we need to provide a name to the summary being generated.

```
ecom4 <- summarise(ecom3, revenue = sum(order_value),
  orders = n())
ecom4
```

```
# A tibble: 3 x 3
  device revenue orders
  <fct>    <dbl>  <int>
1 laptop  56531     31
2 tablet  51321     36
3 mobile  51504     36
```

There you go, we have the total order value and total order items for each device type. The same grouped summary can be written more directly with the `.by` argument, which avoids keeping the data grouped with `group_by()/ungroup()`:

```
ecom4 <- summarise(ecom2, revenue = sum(order_value), orders = n(), .by = device)
ecom4
```

```
# A tibble: 3 x 3
  device revenue orders
  <fct>    <dbl> <int>
1 tablet   51321     36
2 mobile   51504     36
3 laptop   56531     31
```

4.12 Create Columns

To create a new column, we will use `mutate()`. The first argument is the name of the data set and the subsequent arguments are expressions for creating new columns based out of existing columns.

Let us add a new column `avg_page_time` i.e. time on site divided by number of pages visited.

```
# select duration and n_pages from ecom
mutate_1 <- select(ecom, n_pages, duration)
mutate(mutate_1, avg_page_time = duration / n_pages)
```

```
# A tibble: 1,000 x 3
  n_pages duration avg_page_time
  <dbl>    <dbl>    <dbl>
1      1      693          693
2      1      459          459
3      1      996          996
4     18      468           26
5      1      955          955
6      5      135           27
7      1       75           75
8      1      908          908
9     19      209           11
10     1      208          208
# i 990 more rows
```

We can create new columns based on other columns created using `mutate`. Let us create another column `sqrt_avg_page_time` i.e. square root of the average time on page using `avg_page_time`.

```
mutate(mutate_1,
       avg_page_time = duration / n_pages,
       sqrt_avg_page_time = sqrt(avg_page_time))
```

```
# A tibble: 1,000 x 4
  n_pages duration avg_page_time sqrt_avg_page_time
  <dbl>    <dbl>         <dbl>         <dbl>
1      1      693           693           26.3
2      1      459           459           21.4
3      1      996           996           31.6
4     18      468            26            5.10
5      1      955           955           30.9
6      5      135            27            5.20
7      1       75            75            8.66
8      1      908           908           30.1
9     19      209            11            3.32
10     1      208           208           14.4
# i 990 more rows
```

4.12.1 Case Study

Back to our case study, from the last step we have the total order value and total order items for each device category and can compute the AOV. We will create a new column to store AOV.

device	order items	order value		device	order items	order value	aov		device	order items	order value	aov
mobile	3	267		mobile	3	267	267 / 3		mobile	3	267	89
tablet	3	297		tablet	3	297	297 / 3		tablet	3	297	99
laptop	4	378		laptop	4	378	378 / 4		laptop	4	378	94.5

```
ecom5 <- mutate(ecom4, aov = revenue / orders)
ecom5
```

```
# A tibble: 3 x 4
  device revenue orders  aov
  <fct>    <dbl>  <int> <dbl>
1 tablet  51321     36 1426.
2 mobile  51504     36 1431.
3 laptop  56531     31 1824.
```

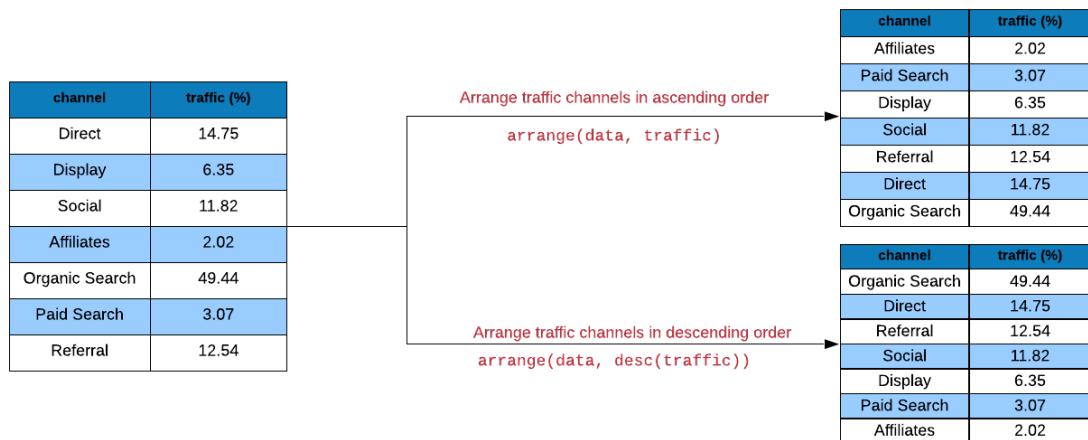
The last step is to select the relevant columns. We will select the device type and the corresponding aov while getting rid of other columns. Use `select()` to extract the relevant columns.

```
ecom6 <- select(ecom5, device, aov)
ecom6
```

```
# A tibble: 3 x 2
  device  aov
  <fct> <dbl>
1 tablet 1426.
2 mobile 1431.
3 laptop 1824.
```

4.13 Arrange Data

Arranging data in ascending or descending order is one of the most common tasks in data manipulation. We can use `arrange` to arrange data by different columns. Let us say we want to arrange data by the number of pages browsed.



```
arrange(ecom, n_pages)
```

```
# A tibble: 1,000 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>    <fct>   <dbl>  <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 google  laptop     10      1     693 FALSE           0           0
```

2	yahoo	tablet	9	1	459	FALSE	0	0
3	direct	laptop	0	1	996	FALSE	0	0
4	yahoo	mobile	9	1	955	FALSE	0	0
5	yahoo	mobile	10	1	75	FALSE	0	0
6	direct	mobile	10	1	908	FALSE	0	0
7	google	mobile	6	1	208	FALSE	0	0
8	direct	laptop	9	1	738	FALSE	0	0
9	yahoo	mobile	7	1	19	FALSE	7	2423
10	bing	laptop	1	1	995	FALSE	0	0

i 990 more rows

If we want to arrange the data in descending order, we can use `desc()`. Let us arrange the data in descending order.

```
arrange(ecom , desc(n_pages))
```

```
# A tibble: 1,000 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>      <fct>   <dbl>   <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 social    tablet     9       20     420 TRUE          7          1613
2 bing      mobile     4       20     440 TRUE          3           184
3 yahoo     tablet     0       20     200 FALSE         0           0
4 direct    tablet     6       20     580 TRUE          5          1155
5 social    mobile     1       20     520 TRUE          5          1021
6 google    mobile     8       20     300 TRUE          7          2091
7 social    laptop     4       20     200 FALSE         0           0
8 yahoo     mobile     3       20     480 FALSE         0           0
9 social    laptop    10       20     280 TRUE          1          2011
10 yahoo     mobile     2       20     240 FALSE         0           0
# i 990 more rows
```

Data can be arranged by multiple variables as well. Let us arrange data first by number of visits and then by number of pages in a descending order.

```
arrange(ecom, n_visit, desc(n_pages))
```

```
# A tibble: 1,000 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
  <fct>      <fct>   <dbl>   <dbl>   <dbl> <lgl>         <dbl>         <dbl>
1 yahoo     tablet     0       20     200 FALSE         0           0
2 google    laptop     0       19     418 TRUE          2           996
3 bing      laptop     0       18     180 FALSE         0           0
4 yahoo     laptop     0       18     522 TRUE          8          1523
5 direct    tablet     0       18     252 FALSE         0           0
```

```

6 social    laptop      0      17      204 FALSE      0      0
7 bing      laptop      0      17      272 TRUE      9     1384
8 bing      mobile      0      16      272 FALSE     0      0
9 yahoo     mobile      0      15      255 FALSE     0      0
10 direct   laptop      0      15      255 FALSE     0      0
# i 990 more rows

```

4.13.1 Case Study

If you observe `ecom6`, the `aov` column is arranged in descending order.

```
arrange(ecom6, aov)
```

```

# A tibble: 3 x 2
  device  aov
  <fct>  <dbl>
1 tablet 1426.
2 mobile 1431.
3 laptop 1824.

```

4.14 AOV by Devices

Let us combine all the code from the above steps:

```

ecom1 <- filter(ecom, purchase)
ecom2 <- select(ecom1, device, order_value)
ecom3 <- group_by(ecom2, device)
ecom4 <- summarise(ecom3, revenue = sum(order_value), orders = n())
ecom5 <- mutate(ecom4, aov = revenue / orders)
ecom6 <- select(ecom5, device, aov)
ecom7 <- arrange(ecom6, aov)
ecom7

```

```

# A tibble: 3 x 2
  device  aov
  <fct>  <dbl>
1 tablet 1426.
2 mobile 1431.
3 laptop 1824.

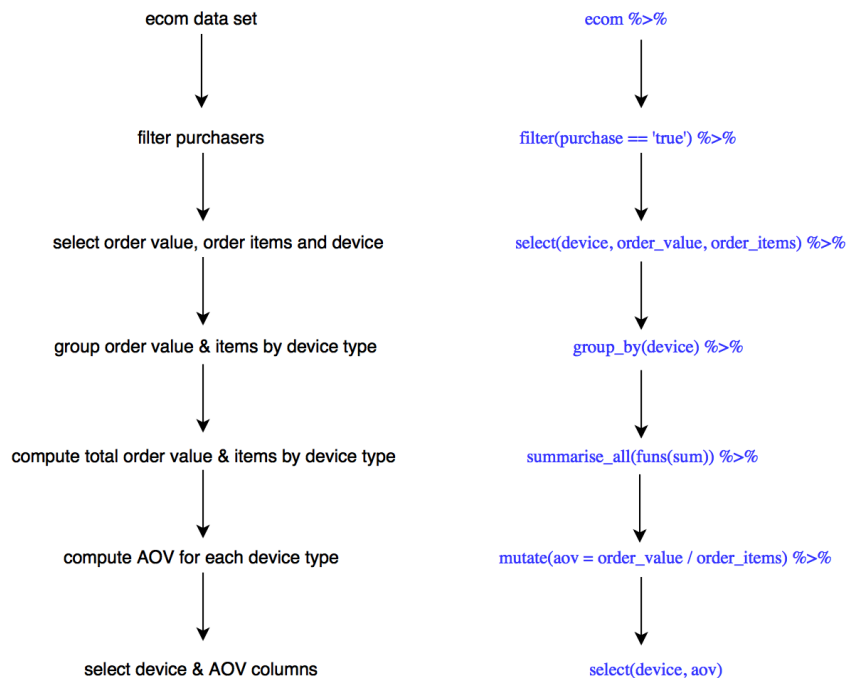
```

If you observe, at each step we create a new variable(data frame) and then use it as an input in the next step i.e. the output from one step becomes the input for the next. Can we achieve the final outcome i.e. `ecom7` without creating the intermediate data (`ecom1` - `ecom6`)? Yes, we can. We will use the `|>` operator to chain the steps and get rid of the intermediate data.

```
ecom |>
  filter(purchase) |>
  summarise(revenue = sum(order_value), orders = n(), .by = device) |>
  mutate(
    aov = revenue / orders
  ) |>
  select(device, aov) |>
  arrange(aov)
```

```
# A tibble: 3 x 2
  device  aov
  <fct>  <dbl>
1 tablet 1426.
2 mobile 1431.
3 laptop 1824.
```

Below we map the description of each step to dplyr verbs.



4.15 Your Turn

- what is the average number of pages visited by purchasers and non-purchasers?
- what is the average time on site for purchasers vs non-purchasers?
- what is the average number of pages visited by purchasers and non-purchasers using mobile?

Try it live

💡 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
    "ecom_sample.csv"
  )
}
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)
ecom |>
  filter(purchase) |>
  summarise(revenue = sum(order_value), orders = n(), .by = device) |>
  mutate(aov = revenue / orders) |>
  arrange(desc(aov))
```

5 Joining Tables using dplyr

5.1 Introduction

In this chapter, we will learn to combine tables using different `*_join` functions provided in `dplyr`.

We will use the following R packages:

```
library(dplyr)
library(readr)
options(tibble.width = Inf)
```

5.2 Case Study

For our case study, we will use two data sets. The first one, `order`, contains details of orders placed by different customers. The second data set, `customer` contains details of each customer. The below table displays the details stored in each data set.

Order
order_id
order_date
order_number
customer_id
amount

Customer
customer_id
first_name
last_name
city
phone number

Let us import both the data sets using `read_csv`.

5.2.1 Data: Orders

```
order <- read_delim('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/orders.csv')
order
```

```
# A tibble: 300 x 3
      id order_date amount
  <dbl> <chr>      <dbl>
1   368 7/2/2016      365.
2   286 11/2/2016    2064.
3    28 2/22/2017     432.
4   309 3/5/2017     480.
5     2 12/28/2016    235.
6    31 12/30/2016   2745.
7   179 12/21/2016   2358.
8   484 11/24/2016   1031.
9   115 9/9/2016     1218.
10  340 5/6/2017     1184.
# i 290 more rows
```

5.2.2 Data: Customers

```
customer <- read_delim('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/customers.csv')
customer
```

```
# A tibble: 91 x 3
      id first_name city
  <dbl> <chr>      <chr>
1     1 Elbertine California
2     2 Marcella Colorado
3     3 Daria Florida
4     4 Sherilyn Distric...
5     5 Ketty Texas
6     6 Jethro California
7     7 Jeremiah California
8     8 Constancia Texas
9     9 Muire Idaho
10    10 Abigail Texas
# i 81 more rows
```

We will explore the following in the case study:

- details of customers who have placed orders and their order details
- details of customers and their orders irrespective of whether a customer has placed orders or not
- customer details for each order
- details of customers who have placed orders
- details of customers who have not placed orders
- details of all customers and all orders

5.3 Example Data

We will use another data set to illustrate how the different joins work. You can view the example data sets below.

Age

Name	Age
John	26
Jenny	24
Jacob	28

Height

Name	Height
John	170
Jenny	174
Janet	166

5.4 Specifying keys with `join_by()`

Every join below matches rows on the `id` column present in both tables, written as `by = join_by(id)`. Use `join_by()` (dplyr >= 1.1) instead of a plain string whenever keys need attention:

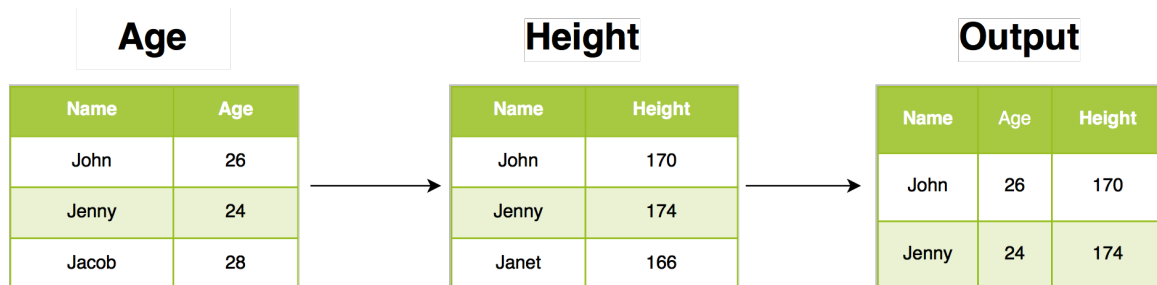
- same name in both tables: `by = join_by(id)`
- different names: `by = join_by(customer_id == id)`
- multiple keys: `by = join_by(id, region)`

Audit habit

Before trusting any join, run `anti_join()` in both directions (see Anti Join). Rows on either side with no match are data-quality findings, not join errors — investigate them first.

5.5 Inner Join

Inner join return all rows from **Age** where there are matching values in **Height**, and all columns from **Age** and **Height**. If there are multiple matches between **Age** and **Height**, all combination of the matches are returned.



5.5.1 Case Study: Details of customers who have placed orders and their order details

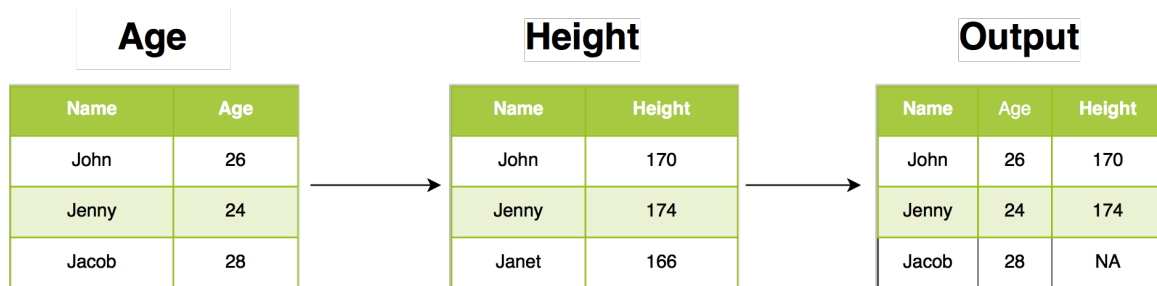
To get data for all those customers who have placed orders in the past let us join the **order** data with the **customer** data using `inner_join`.

```
inner_join(customer, order, by = join_by(id))
```

```
# A tibble: 55 x 5
   id first_name city      order_date amount
  <dbl> <chr>      <chr>      <chr>      <dbl>
1     2 Marcella Colorado 12/28/2016  235.
2     2 Marcella Colorado  8/31/2016 1150.
3     5 Ketty    Texas    1/17/2017  346.
4     6 Jethro    California 1/27/2017 2317.
5     7 Jeremiah California 6/21/2016  136.
6     7 Jeremiah California 2/13/2017 1407.
7     7 Jeremiah California 7/8/2016  1914.
8     8 Constancia Texas    11/5/2016 2461.
9     8 Constancia Texas    5/19/2017 2714.
10    9 Muire    Idaho    12/28/2016 187.
# i 45 more rows
```

5.6 Left Join

Left join return all rows from **Age**, and all columns from **Age** and **Height**. Rows in **Age** with no match in **Height** will have NA values in the new columns. If there are multiple matches between **Age** and **Height**, all combinations of the matches are returned.



5.7 Case Study: Details of customers and their orders irrespective of whether a customer has

placed orders or not.

To get data for all those customers and their orders irrespective of whether a customer has placed orders or not let us join the `order` data with the `customer` data using `left_join`.

```
left_join(customer, order, by = join_by(id))
```

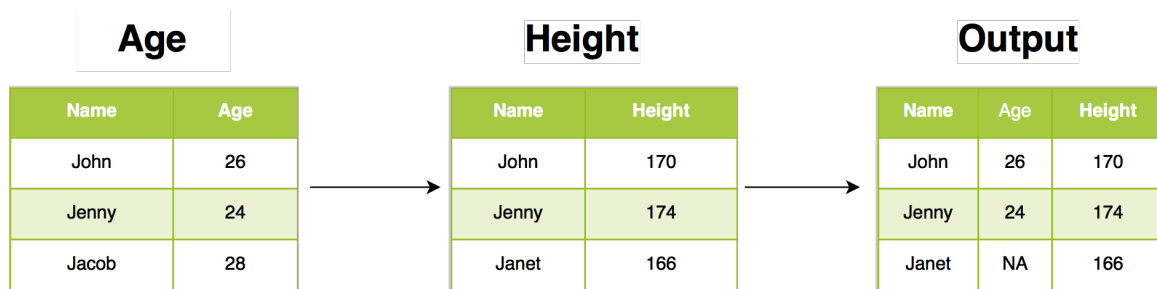
```
# A tibble: 104 x 5
```

```

      id first_name city      order_date amount
  <dbl> <chr>      <chr>      <chr>      <dbl>
1     1 Elbertine California <NA>         NA
2     2 Marcella Colorado  12/28/2016  235.
3     2 Marcella Colorado  8/31/2016  1150.
4     3 Daria     Florida   <NA>         NA
5     4 Sherilyn Distric... <NA>         NA
6     5 Ketty     Texas     1/17/2017   346.
7     6 Jethro    California 1/27/2017  2317.
8     7 Jeremiah California 6/21/2016   136.
9     7 Jeremiah California 2/13/2017  1407.
10    7 Jeremiah California 7/8/2016   1914.
# i 94 more rows
```

5.8 Right Join

Right join return all rows from `Height`, and all columns from `Age` and `Height`. Rows in `Height` with no match in `Age` will have NA values in the new columns. If there are multiple matches between `Age` and `Height`, all combinations of the matches are returned.



5.8.1 Case Study: Customer details for each order

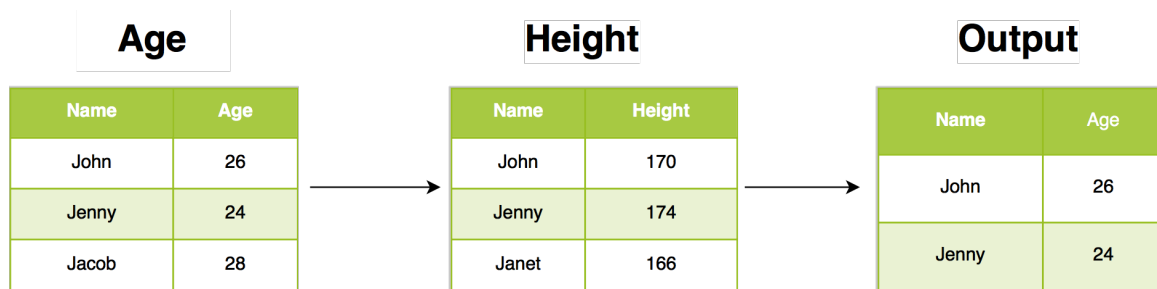
To get customer data for all orders, let us join the `order` data with the `customer` data using `right_join`.

```
right_join(customer, order, by = join_by(id))
```

```
# A tibble: 300 x 5
      id first_name city      order_date amount
  <dbl> <chr>      <chr>      <chr>      <dbl>
1     2 Marcella Colorado 12/28/2016   235.
2     2 Marcella Colorado 8/31/2016  1150.
3     5 Ketty    Texas    1/17/2017   346.
4     6 Jethro    California 1/27/2017  2317.
5     7 Jeremiah California 6/21/2016   136.
6     7 Jeremiah California 2/13/2017  1407.
7     7 Jeremiah California 7/8/2016   1914.
8     8 Constancia Texas    11/5/2016  2461.
9     8 Constancia Texas    5/19/2017  2714.
10    9 Muire    Idaho    12/28/2016   187.
# i 290 more rows
```

5.9 Semi Join

Semi join return all rows from `Age` where there are matching values in `Height`, keeping just columns from `Age`. A semi join differs from an inner join because an inner join will return one row of `Age` for each matching row of `Height`, where a semi join will never duplicate rows of `Age`.



5.9.1 Case Study: Details of customers who have placed orders

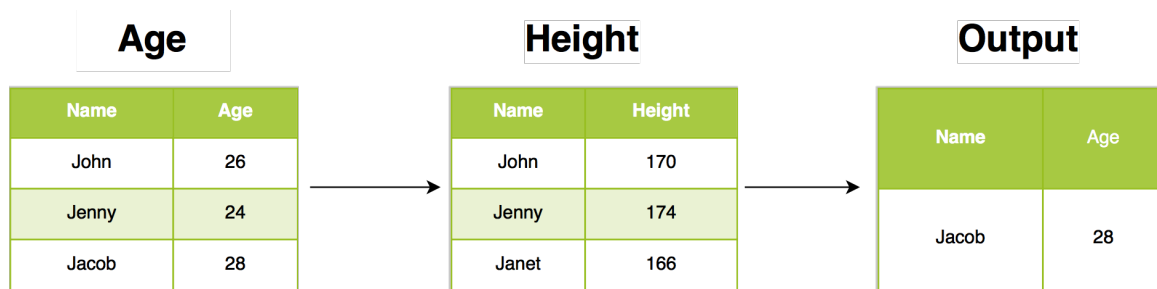
To get customer data for all orders where customer data exists, let us join the `order` data with the `customer` data using `semi_join`. You can observe that data is returned only for those cases where customer data is present.

```
semi_join(customer, order, by = join_by(id))
```

```
# A tibble: 42 x 3
   id first_name city
<dbl> <chr>    <chr>
1     2 Marcella Colorado
2     5 Ketty    Texas
3     6 Jethro    California
4     7 Jeremiah California
5     8 Constancia Texas
6     9 Muire    Idaho
7    15 Valentijn California
8    16 Monique Missouri
9    20 Colette Texas
10   28 Avrit    Texas
# i 32 more rows
```

5.10 Anti Join

Anti join return all rows from `Age` where there are not matching values in `Height`, keeping just columns from `Age`.



5.10.1 Case Study: Details of customers who have not placed orders

To get details of customers who have not placed orders, let us join the `order` data with the `customer` data using `anti_join`.

```
anti_join(customer, order, by = join_by(id))
```

```
# A tibble: 49 x 3
   id first_name city
<dbl> <chr>    <chr>
1     1 Elbertine California
2     3 Daria    Florida
3     4 Sherilyn Distric...
4    10 Abigail   Texas
5    11 Wynne    Georgia
6    12 Pietra   Minnesota
7    13 Bram     Iowa
8    14 Rees     New York
9    17 Orazio   Louisiana
10   18 Mason     Texas
# i 39 more rows
```

5.10.2 Audit pattern: check mismatches in both directions

A single `anti_join()` shows only one side. Before trusting a join, check both:

```
# customers with no orders
anti_join(customer, order, by = join_by(id))
```

```
# A tibble: 49 x 3
   id first_name city
<dbl> <chr>    <chr>
1     1 Elbertine California
2     3 Daria    Florida
```

```

3      4 Sherilyn  Distric...
4     10 Abigail   Texas
5     11 Wynne    Georgia
6     12 Pietra   Minnesota
7     13 Bram     Iowa
8     14 Rees     New York
9     17 Orazio   Louisiana
10    18 Mason    Texas
# i 39 more rows

```

```

# orders with no matching customer record
anti_join(order, customer, by = join_by(id))

```

```

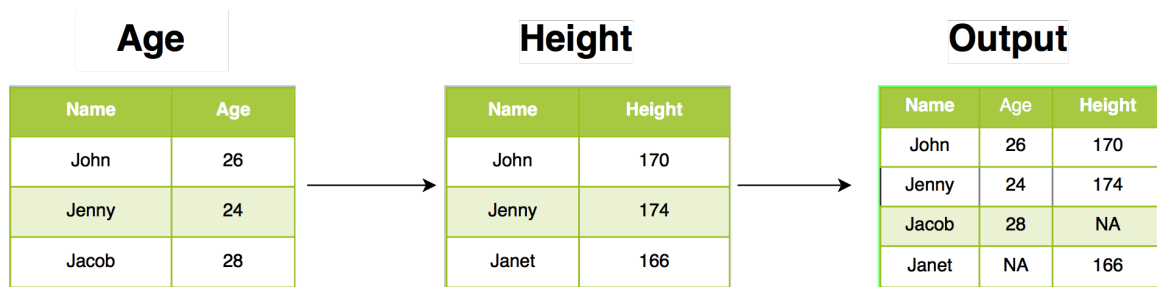
# A tibble: 245 x 3
      id order_date amount
  <dbl> <chr>      <dbl>
1   368 7/2/2016     365.
2   286 11/2/2016    2064.
3   309 3/5/2017     480.
4   179 12/21/2016   2358.
5   484 11/24/2016   1031.
6   115 9/9/2016     1218.
7   340 5/6/2017     1184.
8   493 7/11/2016    1558.
9   340 2/15/2017    2330.
10  235 2/27/2017     149.
# i 235 more rows

```

In this data, 49 customers placed no orders — and 245 orders have no matching customer record. Those 245 rows are a data-quality finding (missing or mismatched customer data upstream), not a join mistake: an `inner_join()` will silently drop them, and a `left_join()/right_join()` will carry them with NAs. Decide deliberately which behavior the analysis needs.

5.11 Full Join

Full join return all rows and all columns from both `Age` and `Height`. Where there are not matching values, returns NA for the one missing.



5.11.1 Case Study: Details of all customers and all orders

To get details of all customers and all orders, let us join the `order` data with the `customer` data using `full_join`.

```
full_join(customer, order, by = join_by(id))
```

```
# A tibble: 349 x 5
      id first_name city      order_date amount
  <dbl> <chr>      <chr>      <chr>      <dbl>
1     1 Elbertine California <NA>         NA
2     2 Marcella Colorado  12/28/2016  235.
3     2 Marcella Colorado  8/31/2016  1150.
4     3 Daria     Florida   <NA>         NA
5     4 Sherilyn Distric... <NA>         NA
6     5 Ketty     Texas     1/17/2017   346.
7     6 Jethro    California 1/27/2017  2317.
8     7 Jeremiah California 6/21/2016   136.
9     7 Jeremiah California 2/13/2017  1407.
10    7 Jeremiah California 7/8/2016   1914.
# i 339 more rows
```

5.12 Cross Join

`cross_join()` returns every combination of rows from both tables (a Cartesian product), so it takes no key. Use it for grids — e.g. all size × color combinations — never for large tables, since the result has `nrow(x) * nrow(y)` rows.

```
cross_join(tibble(size = c("S", "M")), tibble(color = c("red", "blue")))
```

```
# A tibble: 4 x 2
  size color
  <chr> <chr>
1 S     red
2 S     blue
3 M     red
4 M     blue
```

```
1 S    red
2 S    blue
3 M    red
4 M    blue
```

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(readr)
order <- read_delim("https://raw.githubusercontent.com/rsquaredacademy/datasets/master/order")
customer <- read_delim("https://raw.githubusercontent.com/rsquaredacademy/datasets/master/customer")
left_join(customer, order, by = join_by(id))
```

6 dplyr Helpers

6.1 Introduction

In this chapter, we will explore a set of helper functions in order to:

- extract unique rows
- rename columns
- sample data
- extract columns
- slice rows
- arrange rows
- compare tables
- extract/mutate data using predicate functions
- count observations for different levels of a variable

We will use the following R packages:

```
library(dplyr)
library(readr)
```

6.2 Case Study

Let us look at a case study (e-commerce data) and see how we can use dplyr helper functions to answer questions we have about and to modify/transform the underlying data set.

6.2.1 Data

```
ecom <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),
      purchase = col_logical(), bouncers = col_logical(), duration = col_double(),
      n_visit = col_double(), n_pages = col_double()
    )
  )
```

```
ecom
```

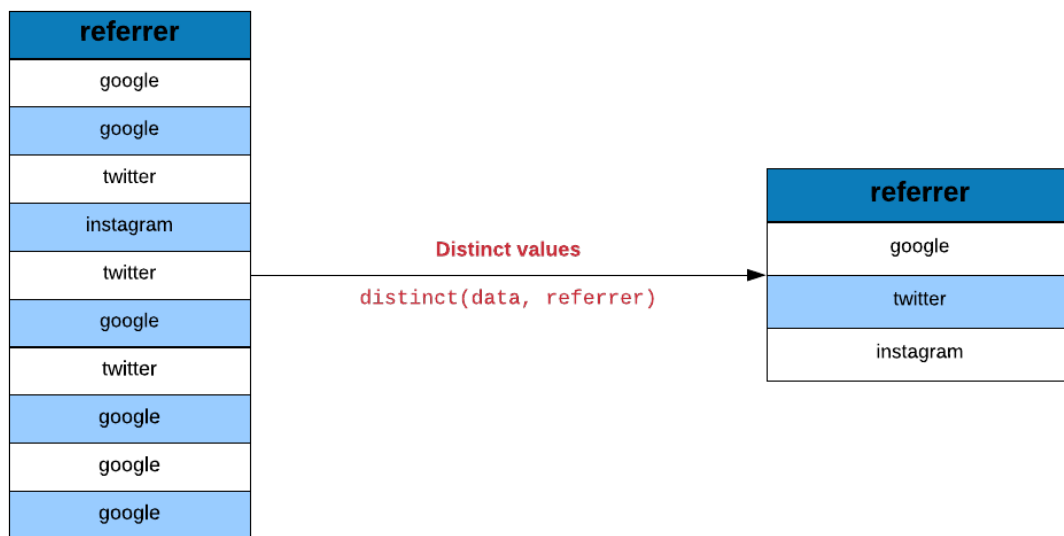
```
# A tibble: 1,000 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>    <fct> <lgl>      <dbl>   <dbl>   <dbl> <lgl>
1 google  laptop TRUE         10      1     693 FALSE
2 yahoo   tablet TRUE          9      1     459 FALSE
3 direct  laptop TRUE          0      1     996 FALSE
4 bing    tablet FALSE         3     18     468 TRUE
5 yahoo   mobile TRUE          9      1     955 FALSE
6 yahoo   laptop FALSE         5      5     135 FALSE
7 yahoo   mobile TRUE         10      1      75 FALSE
8 direct  mobile TRUE         10      1     908 FALSE
9 bing    mobile FALSE         3     19     209 FALSE
10 google mobile TRUE          6      1     208 FALSE
# i 990 more rows
```

6.2.2 Data Dictionary

- referrer: referrer website/search engine
- device: device used to visit the website
- bouncers: whether a visit bounced (exited from landing page)
- duration: time spent on the website (in seconds)
- purchase: whether visitor purchased
- n_visit: number of visits
- n_pages: number of pages visited/browsed

6.3 Data Sanitization

Let us ensure that the data is sanitized by checking the sources of traffic and devices used to visit the site. We will use `distinct` to examine the values in the `referrer` column



```
distinct(ecom, referrer)
```

```
# A tibble: 5 x 1
  referrer
  <fct>
1 google
2 yahoo
3 direct
4 bing
5 social
```

and the device column as well.

```
distinct(ecom, device)
```

```
# A tibble: 3 x 1
  device
  <fct>
1 laptop
2 tablet
3 mobile
```

6.4 Rename Columns

Columns can be renamed using `rename()`.

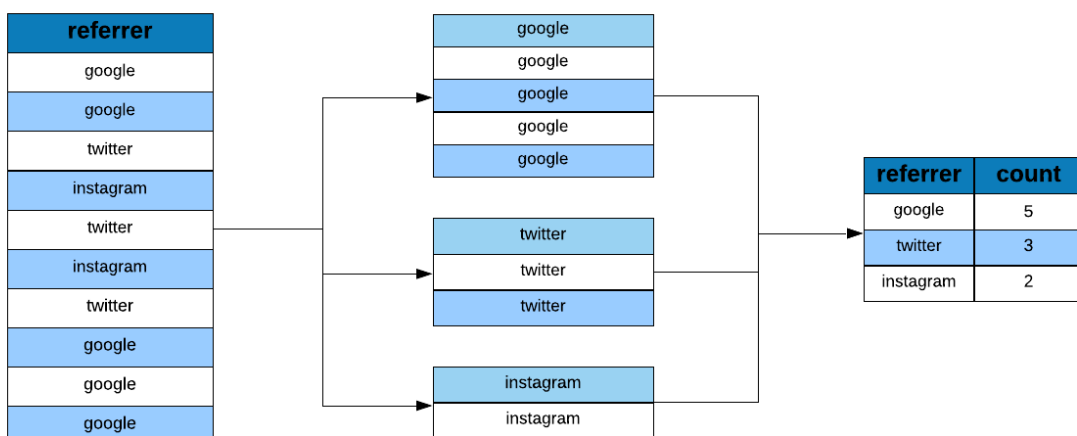
device	order items	order value		device	items	order value
mobile	3	267	Rename order items as items <code>rename(data, items = `order items`)</code>	mobile	3	267
tablet	3	297		tablet	3	297
laptop	4	378		laptop	4	378

```
rename(ecom, time_on_site = duration)
```

```
# A tibble: 1,000 x 7
  referrer device bouncers n_visit n_pages time_on_site purchase
  <fct>      <fct> <lgl>      <dbl>   <dbl>      <dbl> <lgl>
1 google  laptop TRUE         10     1         693 FALSE
2 yahoo   tablet TRUE          9     1         459 FALSE
3 direct  laptop TRUE          0     1         996 FALSE
4 bing    tablet FALSE         3    18         468 TRUE
5 yahoo   mobile TRUE          9     1         955 FALSE
6 yahoo   laptop FALSE         5     5         135 FALSE
7 yahoo   mobile TRUE         10     1          75 FALSE
8 direct  mobile TRUE         10     1         908 FALSE
9 bing    mobile FALSE         3    19         209 FALSE
10 google  mobile TRUE          6     1         208 FALSE
# i 990 more rows
```

6.5 Data Tabulation

Let us now look at the proportion or share of visits driven by different sources of traffic.



```
ecom |>
  group_by(referrer) |>
  tally()
```

```
# A tibble: 5 x 2
  referrer      n
  <fct>    <int>
1  bing      194
2  direct    191
3  social    200
4  yahoo     207
5  google    208
```

We would also like to know the number of bouncers driven by the different sources of traffic.

```
ecom |>
  group_by(referrer, bouncers) |>
  tally()
```

```
# A tibble: 10 x 3
# Groups:   referrer [5]
  referrer bouncers      n
  <fct>    <lg1>    <int>
1  bing    FALSE     104
2  bing    TRUE       90
3  direct  FALSE     98
4  direct  TRUE       93
5  social  FALSE     93
6  social  TRUE     107
7  yahoo   FALSE    110
8  yahoo   TRUE      97
9  google  FALSE    101
10 google  TRUE     107
```

Let us look at how many conversions happen across different devices.

```
ecom |>
  group_by(device, purchase) |>
  tally() |>
  filter(purchase)
```

```
# A tibble: 3 x 3
# Groups:   device [3]
```

	device	purchase	n
	<fct>	<lgl>	<int>
1	laptop	TRUE	31
2	tablet	TRUE	36
3	mobile	TRUE	36

Another way to extract the above information is by using `count`

```
ecom |>
  count(referrer, purchase) |>
  filter(purchase)
```

```
# A tibble: 5 x 3
  referrer purchase     n
  <fct>     <lgl>   <int>
1  bing      TRUE     17
2  direct    TRUE     25
3  social    TRUE     20
4  yahoo     TRUE     22
5  google    TRUE     19
```

The `group_by()` + `tally()` pairs above can also be written with the `.by` argument, which groups only for that verb and leaves the data ungrouped afterwards:

```
ecom |>
  summarise(n = n(), .by = referrer)
```

```
# A tibble: 5 x 2
  referrer     n
  <fct>   <int>
1  google  208
2  yahoo   207
3  direct  191
4  bing    194
5  social  200
```

Two more helpers round out grouped work. `summarise()` must return a single row per group; when a summary needs several rows, use `reframe()` instead:

```
ecom |>
  reframe(quantile = quantile(n_pages, c(0.25, 0.75)), .by = purchase)
```

```
# A tibble: 4 x 2
  purchase quartile
  <lgl>         <dbl>
1 FALSE         1
2 FALSE         8
3 TRUE          13
4 TRUE          18
```

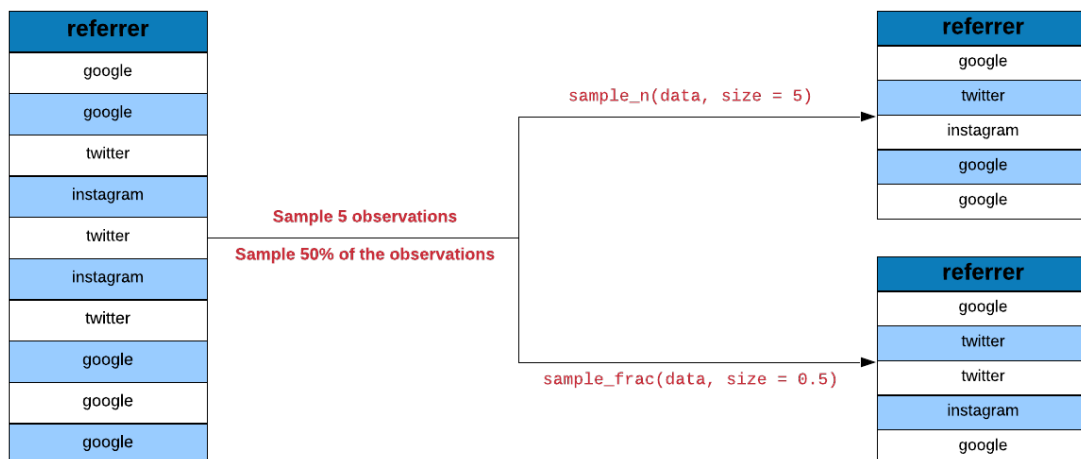
And `pick()` bundles columns into a data frame inside a verb, so a computation needing several columns at once does not repeat the data name:

```
ecom |>
  summarise(cor = with(pick(n_pages, duration), cor(n_pages, duration)), .by = purchase)
```

```
# A tibble: 2 x 2
  purchase    cor
  <lgl>      <dbl>
1 FALSE   -0.251
2 TRUE     0.429
```

6.6 Sampling Data

`dplyr` offers a sampling function which allows us to specify either the number or percentage of observations. `slice_sample()` allows sampling a specific number of observations.



```
slice_sample(ecom, n = 700)
```

```
# A tibble: 700 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>      <fct> <lgl>      <dbl>   <dbl>   <dbl> <lgl>
1 direct    tablet TRUE         8       1     404 FALSE
2 bing      tablet FALSE        1     14     168 FALSE
3 google    tablet TRUE         8       1     617 FALSE
4 bing      laptop TRUE         7       1     811 FALSE
5 google    tablet TRUE         6       1     551 FALSE
6 direct    mobile FALSE        2       4       76 FALSE
7 bing      laptop FALSE        8      10     260 TRUE
8 google    laptop FALSE        2      17     391 FALSE
9 social    laptop FALSE        8      12     276 FALSE
10 social    mobile FALSE        7      16     352 TRUE
# i 690 more rows
```

We can combine the sampling function with grouping as shown below where we sample observations after grouping them according to the source of traffic.

```
slice_sample(ecom, n = 100, by = referrer)
```

```
# A tibble: 500 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>      <fct> <lgl>      <dbl>   <dbl>   <dbl> <lgl>
1 google    mobile TRUE         7       1     241 FALSE
2 google    laptop TRUE         6       1     369 FALSE
3 google    laptop TRUE        10       1     693 FALSE
4 google    laptop FALSE        6      13     325 TRUE
5 google    tablet TRUE         9       1     382 FALSE
6 google    mobile TRUE         2       1     337 FALSE
7 google    mobile FALSE        5       7     133 FALSE
8 google    tablet FALSE        1       9       90 FALSE
9 google    mobile FALSE       10       5       50 FALSE
10 google    mobile TRUE         6       1     208 FALSE
# i 490 more rows
```

`slice_sample()` with the `prop` argument allows sampling a specific percentage of observations.

```
slice_sample(ecom, prop = 0.7)
```

```
# A tibble: 700 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>      <fct> <lgl>      <dbl>   <dbl>   <dbl> <lgl>
1 social    mobile TRUE         10      1     198 FALSE
2 yahoo     tablet FALSE          9     13     182 FALSE
3 yahoo     tablet TRUE          1      1     949 FALSE
4 google    tablet FALSE          1      7     112 FALSE
5 google    laptop FALSE          2     17     391 TRUE
6 google    tablet FALSE          1      9      90 FALSE
7 direct    mobile TRUE         10      1     908 FALSE
8 bing      tablet FALSE          9      2      50 FALSE
9 direct    laptop FALSE          1     10     120 FALSE
10 bing      mobile FALSE          0      7     196 FALSE
# i 690 more rows
```

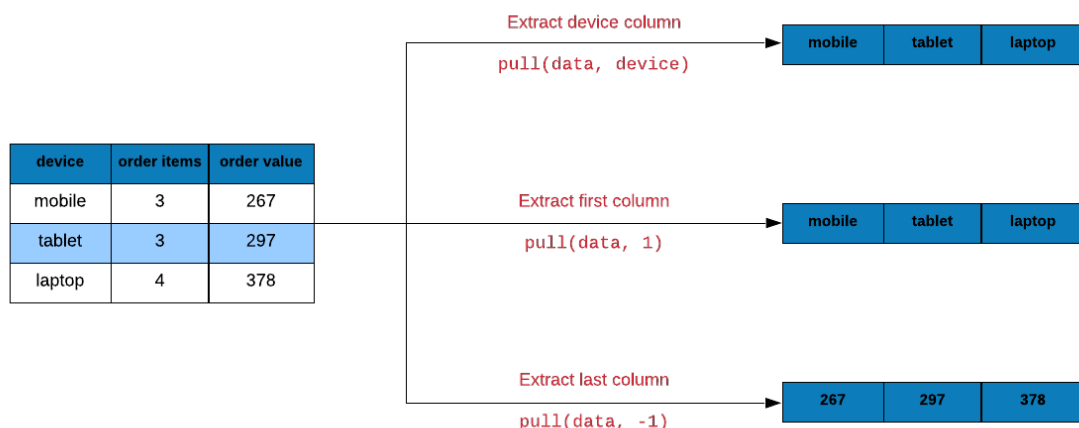
6.7 Data Extraction

In the first [chapter](#), we had observed that dplyr verbs always returned a tibble. What if you want to extract a specific column or a bunch of rows but not as a tibble?

Use `pull` to extract columns either by name or position. It will return a vector. In the below example, we extract the `device` column as a vector. I am using `head` in addition to limit the output printed.

6.7.1 Sample Data

```
ecom_mini <- slice_sample(ecom, n = 10)
```



```
pull(ecom_mini, device)
```

```
[1] mobile laptop tablet laptop laptop mobile mobile laptop mobile mobile  
Levels: laptop tablet mobile
```

Let us extract the first column from `ecom` using column position instead of name.

```
pull(ecom_mini, 1)
```

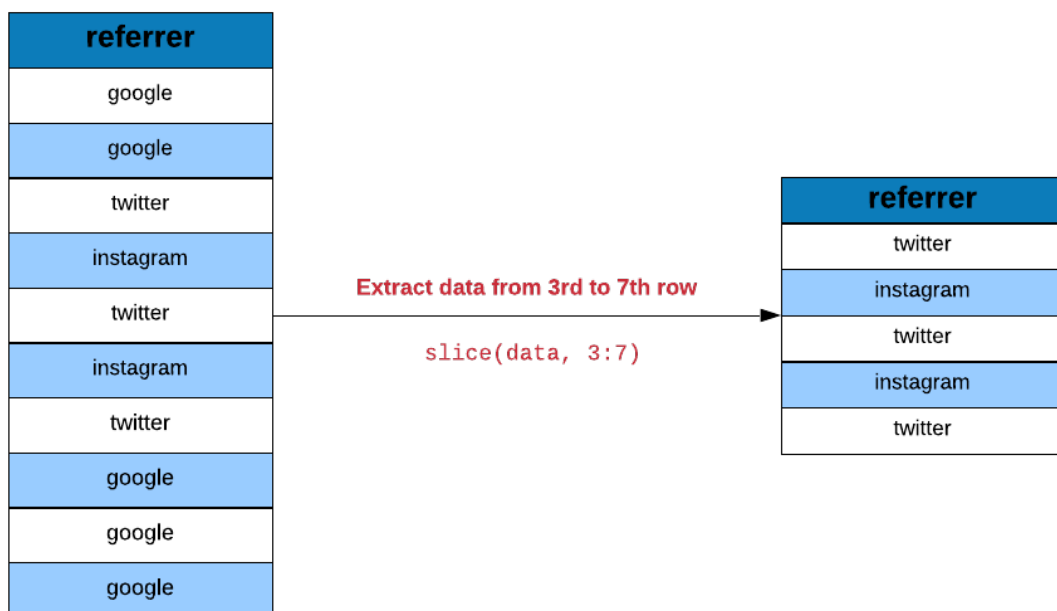
```
[1] direct social yahoo direct yahoo yahoo yahoo yahoo bing yahoo  
Levels: bing direct social yahoo google
```

You can use `-` before the column position to indicate the position in reverse. The below example extracts data from the last column.

```
pull(ecom_mini, -1)
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Let us now look at extracting rows using `slice()`. In the below example, we extract data starting from the 5th row and upto the 15th row.



```
slice(ecom, 5:15)
```

```
# A tibble: 11 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>    <fct> <lgl>      <dbl>  <dbl>    <dbl> <lgl>
1 yahoo    mobile TRUE         9      1      955 FALSE
2 yahoo    laptop FALSE        5      5      135 FALSE
3 yahoo    mobile TRUE        10      1       75 FALSE
4 direct   mobile TRUE        10      1      908 FALSE
5 bing     mobile FALSE         3     19      209 FALSE
6 google   mobile TRUE         6      1      208 FALSE
7 direct   laptop TRUE         9      1      738 FALSE
8 direct   tablet FALSE        6     12      132 FALSE
9 direct   mobile FALSE        9     14      406 TRUE
10 yahoo    tablet FALSE        5      8       80 FALSE
11 yahoo    mobile FALSE        7      1       19 FALSE
```

Use `n()` inside `slice()` to extract the last row.

```
slice(ecom, n())
```

```
# A tibble: 1 x 7
  referrer device bouncers n_visit n_pages duration purchase
  <fct>    <fct> <lgl>      <dbl>  <dbl>    <dbl> <lgl>
1 google   mobile TRUE         9      1      269 FALSE
```

6.8 Between

`between()` allows us to test if the values in a column lie between two specific values. In the below example, we check how many visits browsed pages between 5 and 15.

```
ecom_sample <- slice_sample(ecom, n = 30)
```

```
ecom_sample |>
  pull(n_pages) |>
  between(5, 15)
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
[13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
[25] FALSE FALSE FALSE FALSE FALSE TRUE
```

6.9 Case When

`case_when()` is an alternative to `if else`. It allows us to lay down the conditions clearly and makes the code more readable. In the below example, we create a new column `repeat_visit` from `n_visit` (the number of previous visits).

```
ecom |>
  mutate(
    repeat_visit = case_when(
      n_visit > 0 ~ TRUE,
      TRUE ~ FALSE
    )
  ) |>
  select(n_visit, repeat_visit)
```

```
# A tibble: 1,000 x 2
  n_visit repeat_visit
  <dbl>   <lgl>
1      10      TRUE
2       9      TRUE
3       0     FALSE
4       3      TRUE
5       9      TRUE
6       5      TRUE
7      10      TRUE
8      10      TRUE
9       3      TRUE
10      6      TRUE
# i 990 more rows
```

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
```

```
    "ecom_sample.csv"  
  )  
}  
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)  
slice_sample(ecom, n = 10)
```

7 tibbles

7.1 Introduction

A **tibble**, or `tbl_df`, is a modern reimagining of the `data.frame`, keeping what time has proven to be effective, and throwing out what is not. Tibbles are `data.frames` that are lazy and surly: they do less (i.e. they don't change variable names or types, and don't do partial matching) and complain more (e.g. when a variable does not exist). This forces you to confront problems earlier, typically leading to cleaner, more expressive code. Tibbles also have an enhanced `print method()` which makes them easier to use with large datasets containing complex objects.

Source: <https://tibble.tidyverse.org/>

In this chapter, we will explore tibbles. To be more precise, we will learn:

- how tibbles are different from data frames?
- how to create tibbles?
- how to manipulate tibbles?

We will use the following R packages:

```
library(tibble)
library(dplyr)
```

7.2 Creating tibbles

tibble can be created using any of the following:

- `tibble()`
- `as_tibble()`
- `tribble()`

Let us start with `tibble()`.

```
tibble(x = letters,
       y = 1:26,
       z = sample(100, 26))
```

```
# A tibble: 26 x 3
  x         y     z
  <chr> <int> <int>
1 a         1    21
2 b         2    64
3 c         3    96
4 d         4    25
5 e         5    56
6 f         6    51
7 g         7    55
8 h         8    71
9 i         9    62
10 j        10    35
# i 16 more rows
```

We mentioned the column names followed by the data. If you do not specify the column names, `tibble()` will supply them. Ensure that the length of each column is same.

7.3 tibble features

7.3.1 never changes input's types

`tibble()` will never alter the input's type. For example, if you supply a character vector it will not be converted to factor unlike `data.frame` where you need to set `stringsAsFactors` to `FALSE`.

```
tibble(x = letters,
       y = 1:26,
       z = sample(100, 26))
```

```
# A tibble: 26 x 3
  x         y     z
  <chr> <int> <int>
1 a         1    41
2 b         2    51
3 c         3    94
4 d         4    87
5 e         5    44
6 f         6    69
7 g         7    29
8 h         8    30
9 i         9    59
10 j        10     9
# i 16 more rows
```

7.3.2 never adjusts variable names

`tibble()` will never modify the column names. In the below example, you can observe that while `data.frame` adds a `.`, `tibble()` retains the column names as is.

```
names(data.frame(`order value` = 10))
```

```
[1] "order.value"
```

```
names(tibble(`order value` = 10))
```

```
[1] "order value"
```

7.3.3 never prints all rows

`tibble()` will never print all the rows and clutter your console. It will only print the first 10 rows and only as many columns that fit the width of the console.

```
x <- 1:100
y <- letters[1]
z <- sample(c(TRUE, FALSE), 100, replace = TRUE)
tibble(x, y, z)
```

```
# A tibble: 100 x 3
      x y      z
  <int> <chr> <lgl>
1     1 a    TRUE
2     2 a    TRUE
3     3 a    TRUE
4     4 a   FALSE
5     5 a   FALSE
6     6 a    TRUE
7     7 a    TRUE
8     8 a   FALSE
9     9 a    TRUE
10    10 a    TRUE
# i 90 more rows
```

7.3.4 never recycles vector of length greater than 1

Recycling vectors of length greater than 1 often leads to errors and as such `tibble()` will only recycle vectors of length 1.

```
x <- 1:100
y <- letters
z <- sample(c(TRUE, FALSE), 100, replace = TRUE)
tibble(x, y, z)
Error in overscope_eval_next(overscope, expr) : object 'y' not found
```

7.4 Membership Testing

We can test if an object is a tibble using `is_tibble()`.

```
is_tibble(mtcars)
```

```
[1] FALSE
```

```
is_tibble(as_tibble(mtcars))
```

```
[1] TRUE
```

7.5 Tribble

Another way to create tibbles is using `tribble()`:

- it is short for transposed tibbles
- it is customized for data entry in code
- column names start with ~
- and values are separated by commas

```
tribble(
  ~x, ~y, ~z,
  #--|--|----
  1, TRUE, 'a',
  2, FALSE, 'b'
)
```

```
# A tibble: 2 x 3
      x y      z
<dbl> <lgl> <chr>
1     1 TRUE  a
2     2 FALSE b
```

7.6 Column Names

Names of the columns in tibbles need not be valid R variable names. They can contain unusual characters like a space or a smiley but must be enclosed in ticks.

```
tibble(  
  ` ` = 'space',  
  `2` = 'integer',  
  `:)` = 'smiley'  
)
```

```
# A tibble: 1 x 3  
  ` ` `2` `:)`  
  <chr> <chr> <chr>  
1 space integer smiley
```

7.7 Add Rows

Let us add data related to **Safari** browser to the web traffic data using `add_row()`.

```
browsers <- enframe(c(chrome = 40, firefox = 20, edge = 30))  
browsers
```

```
# A tibble: 3 x 2  
  name    value  
  <chr>   <dbl>  
1 chrome    40  
2 firefox   20  
3 edge      30
```

```
add_row(browsers, name = 'safari', value = 10)
```

```
# A tibble: 4 x 2  
  name    value  
  <chr>   <dbl>  
1 chrome    40  
2 firefox   20  
3 edge      30  
4 safari    10
```

If we want to add the data at a particular row, we can specify the row number using the `.before` argument. Let us add the data related to **Safari** browser in the second row instead of the last row.

```
add_row(browsers, name = 'safari', value = 10, .before = 2)
```

```
# A tibble: 4 x 2
  name      value
  <chr>    <dbl>
1 chrome      40
2 safari      10
3 firefox     20
4 edge        30
```

7.8 Add Columns

`add_column()` adds a new column to tibbles.

```
browsers <- enframe(c(chrome = 40, firefox = 20, edge = 30, safari = 10))
add_column(browsers, visits = c(4000, 2000, 3000, 1000))
```

```
# A tibble: 4 x 3
  name      value visits
  <chr>    <dbl> <dbl>
1 chrome      40   4000
2 firefox     20   2000
3 edge        30   3000
4 safari      10   1000
```

7.9 Rownames

The `tibble` package provides a set of functions to deal with rownames. Remember, `tibble` does not have `rownames` unlike `data.frame`. To check whether a data set has rownames, use `has_rownames()`.

```
has_rownames(mtcars)
```

```
[1] TRUE
```

7.9.1 Remove Rownames

```
remove_rownames(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
2	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
3	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
4	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
5	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
6	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
7	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
8	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
9	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
10	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
11	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
12	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
13	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
14	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
15	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
16	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
17	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
18	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
19	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
20	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
21	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
22	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
23	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
24	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
25	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
26	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
27	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
28	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
29	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
30	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
31	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
32	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

7.9.2 Rownames to Column

```
head(rownames_to_column(mtcars))
```

	rowname	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
2	Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
3	Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
4	Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
5	Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
6	Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

7.9.3 Column to Rownames

To convert the first column in the data set to rownames, use `column_to_rownames()`:

```
mtcars_tbl <- rownames_to_column(mtcars)
column_to_rownames(mtcars_tbl)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2

Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

7.10 Glimpse

Use `glimpse()` to get an overview of the data.

```
glimpse(mtcars)
```

```
Rows: 32
```

```
Columns: 11
```

```
$ mpg <dbl> 21.0, 21.0, 22.8, 21.4, 18.7, 18.1, 14.3, 24.4, 22.8, 19.2, 17.8, ~
$ cyl <dbl> 6, 6, 4, 6, 8, 6, 8, 4, 4, 6, 6, 8, 8, 8, 8, 8, 4, 4, 4, 4, 8, ~
$ disp <dbl> 160.0, 160.0, 108.0, 258.0, 360.0, 225.0, 360.0, 146.7, 140.8, 16~
$ hp <dbl> 110, 110, 93, 110, 175, 105, 245, 62, 95, 123, 123, 180, 180, 180~
$ drat <dbl> 3.90, 3.90, 3.85, 3.08, 3.15, 2.76, 3.21, 3.69, 3.92, 3.92, 3.92, ~
$ wt <dbl> 2.620, 2.875, 2.320, 3.215, 3.440, 3.460, 3.570, 3.190, 3.150, 3.~
$ qsec <dbl> 16.46, 17.02, 18.61, 19.44, 17.02, 20.22, 15.84, 20.00, 22.90, 18~
$ vs <dbl> 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, ~
$ am <dbl> 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, ~
$ gear <dbl> 4, 4, 4, 3, 3, 3, 3, 4, 4, 4, 4, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 3, ~
$ carb <dbl> 4, 4, 1, 1, 2, 1, 4, 2, 2, 4, 4, 3, 3, 3, 4, 4, 4, 1, 2, 1, 1, 2, ~
```

7.11 Check Column

`has_name()` can be used to check if a tibble has a specific column.

```
has_name(mtcars, 'cyl')
```

```
[1] TRUE
```

```
has_name(mtcars, 'gears')
```

```
[1] FALSE
```

7.12 Summary

7.12.1 Creating tibbles

- use `tibble()` to create tibbles
- use `as_tibble()` to coerce other objects to tibble
- use `enframe()` to coerce vector to tibble
- use `tribble()` to create tibble using data entry

7.12.2 Modifying tibbles

- use `add_row()` to add a new row
- use `add_column()` to add a new column
- use `remove_rownames()` to remove rownames from data
- use `rownames_to_column()` to coerce rowname to first column
- use `column_to_rownames()` to coerce first column to rownames

7.12.3 Testing tibbles

- use `is_tibble()` to test if an object is a tibble
- use `has_rownames()` to check whether a data set has rownames
- use `has_name()` to check if tibble has a specific column
- use `glimpse()` to get an overview of data

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(tibble)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
    "ecom_sample.csv"
  )
}
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)
as_tibble(ecom) |> glimpse()
```

8 Tidying Data with tidyr

8.1 Introduction

Tidy data has one row per observation and one column per variable. Real data rarely arrives that way: metrics sprawl across columns, several values share one column, and combinations go missing. In this chapter we reshape data with **tidyr**:

- `pivot_longer()` / `pivot_wider()` — move between long and wide layouts
- `separate_wider_delim()` / `unite()` — split one column into several and back
- `drop_na()` / `complete()` / `fill()` — handle missing values and combinations

We will use the following R packages:

```
library(dplyr)
library(tidyr)
library(readr)
library(lubridate)
```

8.2 Data

We reuse the ecommerce data from @dplyr-basics, so no new data to learn. The URL and date examples borrow the `mock_strings` and `transact` data sets already used in @strings-in-r and @date-and-time-in-r.

```
ecom <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),
      purchase = col_logical(), n_pages = col_double(), n_visit = col_double(),
      duration = col_double(), order_value = col_double(), order_items = col_double()
    )
  )
ecom
```

```
# A tibble: 1,000 x 8
  referrer device n_visit n_pages duration purchase order_items order_value
<fct>      <fct>    <dbl>  <dbl>    <dbl> <lgl>          <dbl>      <dbl>
1 google   laptop      10      1      693 FALSE          0          0
2 yahoo    tablet       9      1      459 FALSE          0          0
3 direct   laptop       0      1      996 FALSE          0          0
4 bing     tablet       3     18      468 TRUE           6         434
5 yahoo    mobile       9      1      955 FALSE          0          0
6 yahoo    laptop       5      5      135 FALSE          0          0
7 yahoo    mobile     10      1       75 FALSE          0          0
8 direct   mobile     10      1      908 FALSE          0          0
9 bing     mobile       3     19      209 FALSE          0          0
10 google   mobile       6      1      208 FALSE          0          0
# i 990 more rows
```

8.3 Long format with pivot_longer()

Suppose we average three metrics by device. The result is *wide*: one column per metric, which is awkward to plot or rank.

```
wide <- ecom |>
  summarise(across(c(n_pages, duration, order_value), \(x) mean(x, na.rm = TRUE)), .by = d
wide
```

```
# A tibble: 3 x 4
  device n_pages duration order_value
<fct>    <dbl>    <dbl>    <dbl>
1 laptop  5.36      376.      441.
2 tablet  6.23      354.      352.
3 mobile  6.17      337.      370.
```

`pivot_longer()` stacks the metric columns into `metric/mean_value` pairs — one row per device-metric observation:

```
long <- wide |>
  pivot_longer(-device, names_to = "metric", values_to = "mean_value")
long
```

```
# A tibble: 9 x 3
  device metric      mean_value
<fct>  <chr>        <dbl>
1 laptop n_pages      5.36
2 laptop duration    376.
```

```

3 laptop order_value      441.
4 tablet n_pages          6.23
5 tablet duration         354.
6 tablet order_value      352.
7 mobile n_pages          6.17
8 mobile duration         337.
9 mobile order_value      370.

```

8.4 Wide format with pivot_wider()

The reverse trip is just as common. Counts by device and purchase come out long (one row per combination):

```

counts <- ecom |>
  count(device, purchase)
counts

```

```

# A tibble: 6 x 3
  device purchase     n
  <fct>   <lgl>   <int>
1 laptop FALSE    294
2 laptop TRUE     31
3 tablet FALSE    295
4 tablet TRUE     36
5 mobile FALSE    308
6 mobile TRUE     36

```

`pivot_wider()` spreads `purchase` into columns so each device occupies a single row — the layout managers usually want in a report:

```

counts |>
  pivot_wider(names_from = purchase, values_from = n, names_prefix = "purchase_")

```

```

# A tibble: 3 x 3
  device purchase_FALSE purchase_TRUE
  <fct>           <int>         <int>
1 laptop           294             31
2 tablet           295             36
3 mobile           308             36

```

8.5 Split columns with `separate_wider_delim()`

The `url` column of `mock_strings` packs the protocol and the address into one string, separated by `://`. `separate_wider_delim()` splits it into two columns:

```
mockstring <-  
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/mock_strings.csv')  
  
tibble(url = head(mockstring$url, 3)) |>  
  separate_wider_delim(url, delim = "://", names = c("protocol", "rest")) |>  
  select(protocol)
```

```
# A tibble: 3 x 1  
  protocol  
  <chr>  
1 https  
2 http  
3 https
```

The same verb splits calendar dates. The `Due` column of `transact` is `YYYY-MM-DD`; splitting on `"-"` recovers the components:

```
transact <-  
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/transact.csv')  
  
tibble(due_iso = as.character(head(transact$Due, 5))) |>  
  separate_wider_delim(due_iso, delim = "-", names = c("due_year", "due_month", "due_day"))
```

```
# A tibble: 5 x 3  
  due_year due_month due_day  
  <chr>    <chr>    <chr>  
1 2013     02      01  
2 2013     02      25  
3 2013     08      02  
4 2013     03      12  
5 2012     11      24
```

8.6 Combine columns with `unite()`

`unite()` is the inverse: it pastes several columns back into one. Splitting the ISO date and reuniting it returns the input exactly — a useful check that a `separate/unite` round-trip loses nothing:

```
tibble(due_iso = as.character(head(transact$Due, 5))) |>
  separate_wider_delim(due_iso, delim = "-", names = c("due_year", "due_month", "due_day"))
  unite("due_rebuilt", due_year, due_month, due_day, sep = "-")
```

```
# A tibble: 5 x 1
  due_rebuilt
  <chr>
1 2013-02-01
2 2013-02-25
3 2013-08-02
4 2013-03-12
5 2012-11-24
```

8.7 Drop missing values with `drop_na()`

`ecom` itself has no missing values, but joins create them: the full join of `customer` and `order` from `@joining-tables-in-r-dplyr` carries NAs for every unmatched row. `drop_na()` keeps only complete rows:

```
order <- read_delim('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/order.csv')
customer <- read_delim('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/customer.csv')

joined <- full_join(customer, order, by = join_by(id))
nrow(joined)
```

```
[1] 349
```

```
nrow(drop_na(joined))
```

```
[1] 55
```

Only the 55 fully matched rows survive — the same rows `inner_join()` returns. Use `drop_na()` when a downstream step needs complete cases, and prefer it over filtering each column by hand.

8.8 Complete combinations with `complete()`

Counts over a filtered subset can silently omit combinations. Purchasers only span `purchase == TRUE`, so the `FALSE` rows are missing rather than zero:

```
purchasers <- ecom |>
  filter(purchase) |>
  count(device, purchase)
purchasers
```

```
# A tibble: 3 x 3
  device purchase     n
  <fct>   <lgl>   <int>
1 laptop  TRUE     31
2 tablet  TRUE     36
3 mobile  TRUE     36
```

`complete()` adds the missing combinations and fills the count with 0, so plots and tables show every device × purchase cell:

```
purchasers |>
  complete(device, purchase = c(TRUE, FALSE), fill = list(n = 0L))
```

```
# A tibble: 6 x 3
  device purchase     n
  <fct>   <lgl>   <int>
1 laptop FALSE     0
2 laptop TRUE     31
3 tablet FALSE     0
4 tablet TRUE     36
5 mobile FALSE     0
6 mobile TRUE     36
```

8.9 Fill down labels with `fill()`

Spreadsheet exports often write a group label once and leave the rest blank. `fill()` carries the last non-missing value down (or up with `.direction = "up"`):

```
report <- tibble(device = c("laptop", NA, "tablet", NA),
                 metric = c("aov", "orders", "aov", "orders"))
report |>
  fill(device)
```

```
# A tibble: 4 x 2
  device metric
  <chr>   <chr>
1 laptop aov
```

```
2 laptop orders
3 tablet aov
4 tablet orders
```

8.10 Your Turn

- reshape the mean of `n_visit` and `order_items` by `referrer` from wide to long
- split the `email` column of `mockstring` at "@" and reunite it — does the round-trip hold?
- run the `complete()` example without `fill`: what does the missing count become, and why?

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(tidyr)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
    "ecom_sample.csv"
  )
}
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)
ecom |>
  summarise(across(c(n_pages, duration, order_value), \(x) mean(x, na.rm = TRUE)), .by = d
  pivot_longer(-device, names_to = "metric", values_to = "mean_value")
```

9 Hacking Strings

9.1 Introduction

In this chapter, we will learn to work with string data in R using [stringr](#). As we did in the other chapters, we will use a case study to explore the various features of the stringr package. We will use the following R packages:

```
library(stringr)
library(tibble)
library(magrittr)
library(purrr)
library(dplyr)
library(readr)
```

9.2 Case Study

- extract domain name from random email ids
- extract image type from url
- extract image dimension from url
- extract extension from domain name
- extract http protocol from url
- extract file type from url

9.2.1 Data

```
mockstring <- read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/mockstring')
```

```
# A tibble: 1,000 x 12
```

	id	image_url	domain	imageurl	email	filename	phone	address	url	full_name
	<dbl>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	1	https://r~	addto~	http://~	mnew~	PedeMal~	66-(~	8 Anha~	http~	Mufi Ruit
2	2	https://r~	gmpg.~	http://~	mdan~	Loborti~	351-~	697 Ea~	http~	Leese Fu~
3	3	https://r~	samsu~	http://~	hgir~	CongueD~	33-(~	89 Dot~	http~	Blakelee~

```

4      4 https://r~ spoti~ http://~ pmcm~ Eleifen~ 86-(~ 98135 ~ http~ Terencio~
5      5 https://r~ wunde~ http://~ dris~ PurusPh~ 223-~ 7814 P~ http~ Debee Mc~
6      6 https://r~ alexa~ http://~ cphl~ Element~ 420-~ 4897 L~ http~ Fran Pai~
7      7 https://r~ googl~ http://~ kdod~ Mattis.~ 1-(7~ 53541 ~ http~ Frasco B~
8      8 https://r~ ed.gov http://~ vhou~ PurusEu~ 62-(~ 4819 H~ http~ Car Pont~
9      9 https://r~ jigsy~ http://~ rdik~ JustoEt~ 1-(6~ 68096 ~ http~ Tades Ch~
10     10 https://r~ jugem~ http://~ tdud~ Ante.ti~ 30-(~ 9595 S~ http~ Wilton K~
# i 990 more rows
# i 2 more variables: currency <chr>, passwords <chr>

```

9.2.2 Data Dictionary

- domain: dummy website domain
- imageurl: url of an image
- email: dummy email id
- filename: dummy file name with different extensions
- phone: dummy phone number
- address: dummy address with door and street names
- url: randomly generated urls
- full_name: dummy first and last names
- currency: different currencies
- passwords: dummy passwords

9.3 Overview

Before we start with the case study, let us take a quick tour of **stringr** and introduce ourselves to some of the functions we will be using later in the case study. One of the columns in the case study data is **email**. It contains random email ids. We want to ensure that the email ids adhere to a particular format .i.e

- they contain @
- they contain only one @

Let us first detect if the email ids contain @. Since the data set has 1000 rows, we will use a smaller sample in the examples.

```

mockdata <- slice(mockstring, 1:10)
mockdata

```

```

# A tibble: 10 x 12
      id image_url domain imageurl email filename phone address url  full_name
<dbl> <chr>      <chr> <chr>    <chr> <chr>      <chr> <chr> <chr> <chr>
1     1 https://r~ addto~ http://~ mnew~ Pedemaal~ 66-(~ 8 Anha~ http~ Mufi Ruit
2     2 https://r~ gmpg.~ http://~ mdan~ Loborti~ 351-~ 697 Ea~ http~ Leese Fu~

```

```

3      3 https://r~ samsu~ http://~ hgir~ CongueD~ 33-(~ 89 Dot~ http~ Blakelee~
4      4 https://r~ spoti~ http://~ pmcm~ Eleifen~ 86-(~ 98135 ~ http~ Terencio~
5      5 https://r~ wunde~ http://~ dris~ PurusPh~ 223--~ 7814 P~ http~ Debee Mc~
6      6 https://r~ alexa~ http://~ cphl~ Element~ 420--~ 4897 L~ http~ Fran Pai~
7      7 https://r~ googl~ http://~ kdod~ Mattis.~ 1-(7~ 53541 ~ http~ Frasco B~
8      8 https://r~ ed.gov http://~ vhou~ PurusEu~ 62-(~ 4819 H~ http~ Car Pont~
9      9 https://r~ jigsy~ http://~ rdik~ JustoEt~ 1-(6~ 68096 ~ http~ Tades Ch~
10     10 https://r~ jugem~ http://~ tdud~ Ante.ti~ 30-(~ 9595 S~ http~ Wilton K~
# i 2 more variables: currency <chr>, passwords <chr>

```

email id		Count @
mnewburn0@fastcompany.com	Count the occurrence of @ in the email id <code>str_count(pattern = "@")</code>	1
sahgirhard2@altervista.orgs		1
drisbrough4#bandcamp.com		0
rdike8@times@online.co.uk		2
tdudbridge9\$clickbank.net		0
mdankersley1@digg.com		1

Use `str_detect()` to detect @ and `str_count()` to count the number of times @ appears in the email ids.

email id		Detect @
mnewburn0@fastcompany.com	Detect the pattern @ in the email id <code>str_detect(pattern = "@")</code>	TRUE
sahgirhard2@altervista.orgs		TRUE
drisbrough4#bandcamp.com		FALSE
rdike8@timesonline.co.uk		TRUE
tdudbridge9\$clickbank.net		FALSE
mdankersley1@digg.com		TRUE

```

# detect @
str_detect(mockdata$email, pattern = "@")

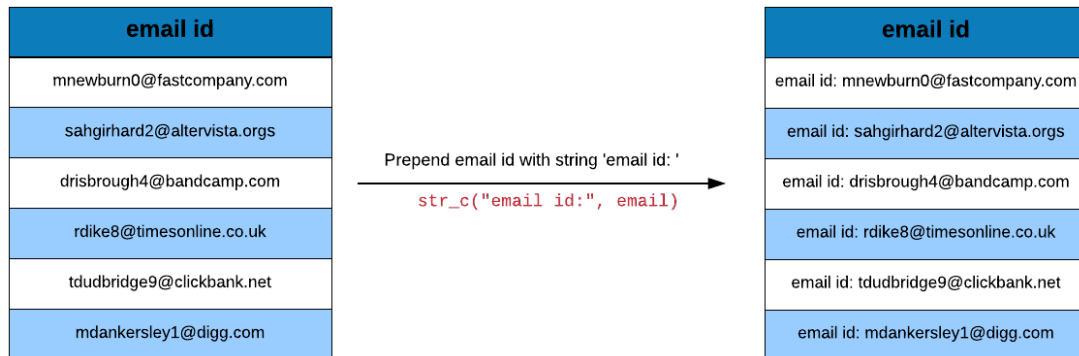
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
# count @
str_count(mockdata$email, pattern = "@")
```

```
[1] 1 1 1 1 1 1 1 1 1 1
```

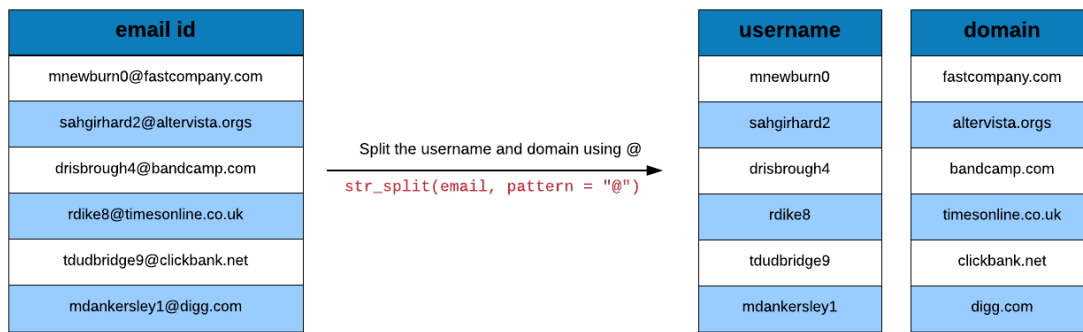
We can use `str_c()` to concatenate strings. Let us add the string `email id:` before each email id in the data set.



```
str_c("email id:", mockdata$email)
```

```
[1] "email id:mnewburn0@fastcompany.com"
[2] "email id:mdankersley1@digg.com"
[3] "email id:hgirhard2@altervista.org"
[4] "email id:pmcmenamy3@sciencedirect.com"
[5] "email id:drisbrough4@bandcamp.com"
[6] "email id:cphlippi5@surveymonkey.com"
[7] "email id:kdodswell6@un.org"
[8] "email id:vhourihane7@ovh.net"
[9] "email id:rdike8@timesonline.co.uk"
[10] "email id:tdudbridge9@clickbank.net"
```

If we want to split a string into two parts using a particular pattern, we use `str_split()`. Let us split the domain name and extension from the domain column in the data. The domain name and extension are separated by `.` and we will use it to split the domain column. Since `.` is a special character, we will use two slashes to escape the special character.



```
str_split(mockdata$domain, pattern = "\\\\.")
```

```
[[1]]
[1] "addtoany" "com"

[[2]]
[1] "gmpg" "org"

[[3]]
[1] "samsung" "com"

[[4]]
[1] "spotify" "com"

[[5]]
[1] "wunderground" "com"

[[6]]
[1] "alexa" "com"

[[7]]
[1] "google" "it"

[[8]]
[1] "ed" "gov"

[[9]]
[1] "jigsy" "com"

[[10]]
[1] "jugem" "jp"
```

We can truncate a string using `str_trunc()`. The default truncation happens at the

beginning of the string but we can truncate the central part or the end of the string as well.

```
str_trunc(mockdata$email, width = 10)
```

```
[1] "mnewbur..." "mdanker..." "hgirhar..." "pmcmena..." "drisbro..."
[6] "cphlipp..." "kdodswe..." "vhourih..." "rdike8@..." "tdudbri..."
```

```
str_trunc(mockdata$email, width = 10, side = "left")
```

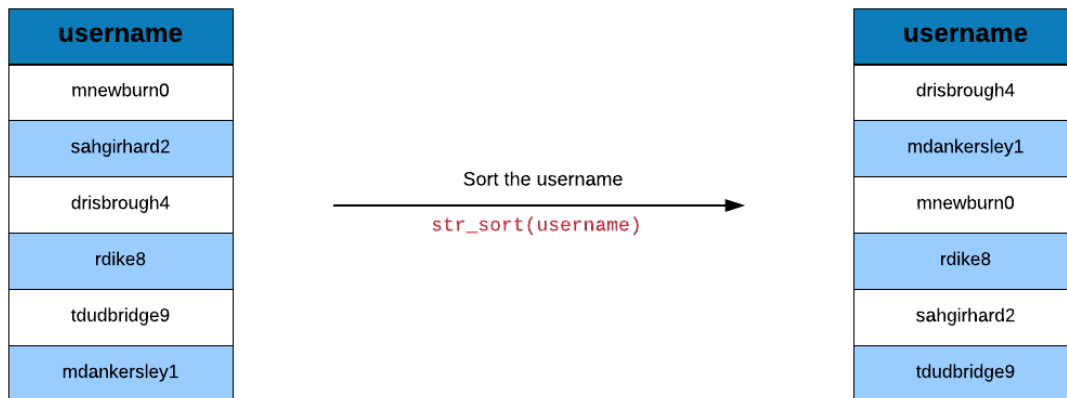
```
[1] "...any.com" "...igg.com" "...sta.org" "...ect.com" "...amp.com"
[6] "...key.com" "...@un.org" "...ovh.net" "...e.co.uk" "...ank.net"
```

```
str_trunc(mockdata$email, width = 10, side = "center")
```

```
[1] "mnew...com" "mdan...com" "hgir...org" "pmcm...com" "dris...com"
[6] "cphl...com" "kdod...org" "vhou...net" "rdik...uk" "tdud...net"
```



Strings can be sorted using `str_sort()`. Let us quickly sort the emails in both ascending and descending orders.



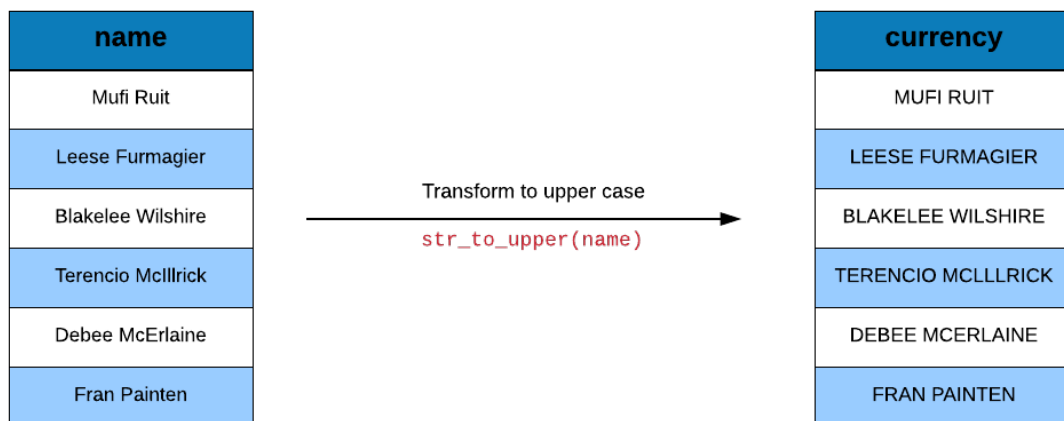
```
str_sort(mockdata$email)
```

```
[1] "cphlippi5@surveymonkey.com"  "drisbrough4@bandcamp.com"
[3] "hgirhard2@altervista.org"    "kdodswell16@un.org"
[5] "mdankersley1@digg.com"       "mnewburn0@fastcompany.com"
[7] "pmcmenamy3@sciencedirect.com" "rdike8@timesonline.co.uk"
[9] "tdudbridge9@clickbank.net"   "vhourihane7@ovh.net"
```

```
str_sort(mockdata$email, decreasing = TRUE)
```

```
[1] "vhourihane7@ovh.net"          "tdudbridge9@clickbank.net"
[3] "rdike8@timesonline.co.uk"     "pmcmenamy3@sciencedirect.com"
[5] "mnewburn0@fastcompany.com"    "mdankersley1@digg.com"
[7] "kdodswell16@un.org"          "hgirhard2@altervista.org"
[9] "drisbrough4@bandcamp.com"     "cphlippi5@surveymonkey.com"
```

The case of a string can be changed to upper, lower or title case as shown below.



```
str_to_upper(mockdata$full_name)
```

```
[1] "MUFI RUIT"          "LEESE FURMAGIER"    "BLAKELEE WILSHIRE"
[4] "TERENCIO MCILLRICK" "DEBEE MCERLAINE"    "FRAN PAINTEN"
[7] "FRASCO BOWICH"      "CAR PONTEN"         "TADES CHECCUCCI"
[10] "WILTON KEMMEY"
```

```
str_to_lower(mockdata$full_name)
```

```
[1] "mufi ruit"          "leese furmagier"    "blakelee wilshire"
[4] "terencio mcillrick" "debee mcerlaine"    "fran painten"
[7] "frasco bowich"      "car ponten"         "tades checcucci"
[10] "wilton kemmey"
```

Parts of a string can be replaced using `str_replace()`. In the `address` column of the data set, let us replace:

- Street with ST
- Road with RD

address
1 Elgar Road
7814 Pennsylvania Street
8 Manley Drive
659 7th Avenue
69 Gina Street
9 Hoepker Street

Replace Street with ST
`str_replace(address, "Street", "ST")`

address
1 Elgar Road
7814 Pennsylvania ST
8 Manley Drive
659 7th Avenue
69 Gina ST
9 Hoepker ST

```
str_replace(mockdata$address, "Street", "ST")
```

```
[1] "8 Anhalt Crossing"      "697 East Avenue"
[3] "89 Dottie Circle"      "98135 Blue Bill Park Drive"
[5] "7814 Pennsylvania ST"  "4897 Little Fleur Drive"
[7] "53541 Morrow Center"  "4819 Hermina Parkway"
[9] "68096 Monument Park"  "9595 Spaight Avenue"
```

```
str_replace(mockdata$address, "Road", "RD")
```

```
[1] "8 Anhalt Crossing"      "697 East Avenue"
[3] "89 Dottie Circle"      "98135 Blue Bill Park Drive"
[5] "7814 Pennsylvania Street" "4897 Little Fleur Drive"
[7] "53541 Morrow Center"  "4819 Hermina Parkway"
[9] "68096 Monument Park"  "9595 Spaight Avenue"
```

We can extract parts of the string that match a particular pattern using `str_extract()`.

email id
mnewburn0@fastcompany.com
sahgirhard2@altervista.orgs
drisbrough4@bandcamp.com
rdike8@timesonline.co.uk
tdudbridge9@clickbank.net
mdankersley1@digg.com

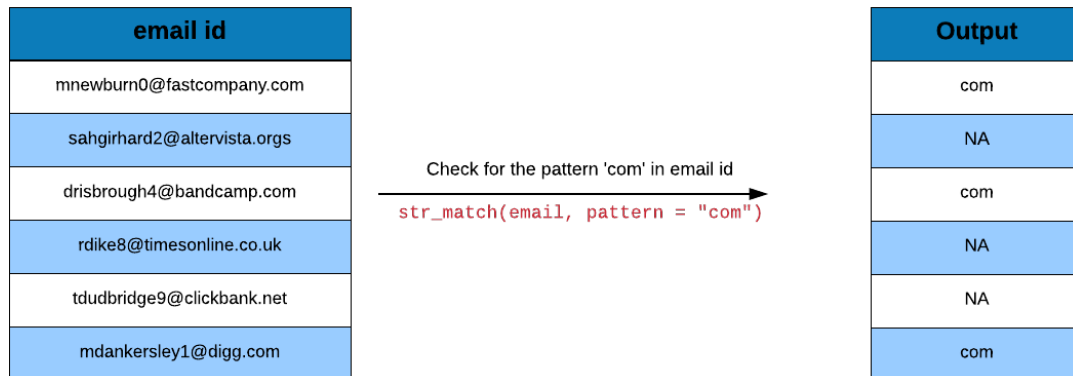
Extract the pattern 'com' from email id
`str_extract(email, pattern = "com")`

Output
com
NA
com
NA
NA
com

```
str_extract(mockdata$email, pattern = "org")
```

```
[1] NA      NA      "org" NA      NA      NA      "org" NA      NA      NA
```

Before we extract, we need to know whether the string contains text that match our pattern. Use `str_match()` to see if the pattern is present in the string.



```
str_match(mockdata$email, pattern = "org")
```

```
      [,1]
[1,] NA
[2,] NA
[3,] "org"
[4,] NA
[5,] NA
[6,] NA
[7,] "org"
[8,] NA
[9,] NA
[10,] NA
```

If we are dealing with a character vector and know that the pattern we are looking at is present in the vector, we might want to know the index of the strings in which it is present. Use `str_which()` to identify the index of the strings that match our pattern.

email id	
mnewburn0@fastcompany.com	
sahgirhard2@altervista.orgs	
drisbrough4@bandcamp.com	Which observations match the pattern 'com'?
rdike8@timesonline.co.uk	<code>str_which(email, pattern = "com")</code>
tdudbridge9@clickbank.net	
mdankersley1@digg.com	

1	3	6
---	---	---

```
str_which(mockdata$email, pattern = "org")
```

[1] 3 7

Another objective might be to locate the position of the pattern we are looking for in the string. For example, if we want to know the position of @ in the email ids, we can use `str_locate()`.

email id		start	end
mnewburn0@fastcompany.com		15	17
sahgirhard2@altervista.orgs		NA	NA
drisbrough4@bandcamp.com	Index of 'com' in email id	22	24
rdike8@timesonline.co.uk	<code>str_locate(email, pattern = "com")</code>	NA	NA
tdudbridge9@clickbank.net		NA	NA
mdankersley1@digg.com		19	21

```
str_locate(mockdata$email, pattern = "@")
```

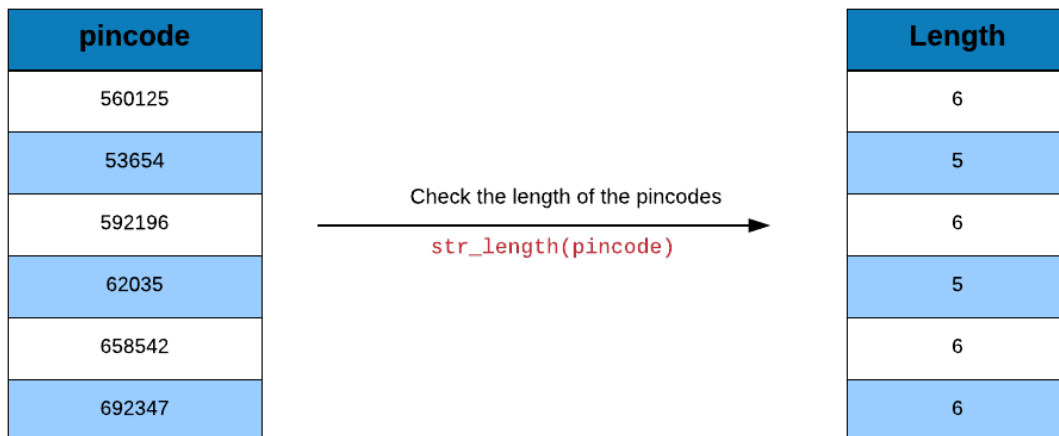
```

      start end
[1,]    10  10
[2,]    13  13
[3,]    10  10
[4,]    11  11
[5,]    12  12
[6,]    10  10
[7,]    11  11
[8,]    12  12

```

```
[9,]      7      7
[10,]     12     12
```

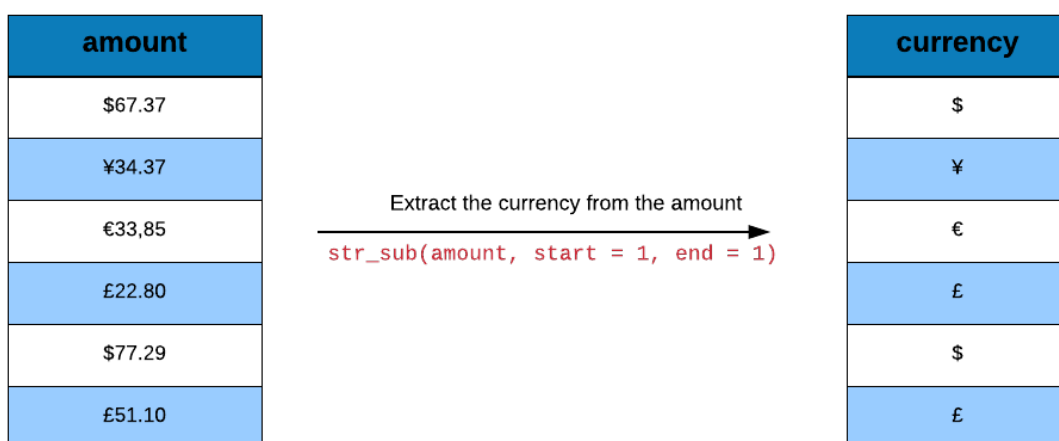
The length of the string can be computed using `str_length()`. Let us ensure that the length of the strings in the `password` column is 16.



```
str_length(mockdata$passwords)
```

```
[1] 16 16 16 16 16 16 16 16 16 16
```

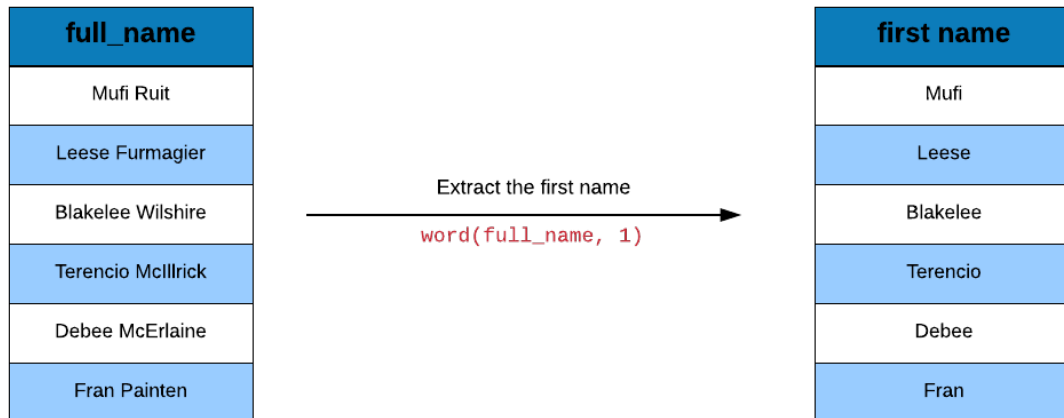
We can extract parts of a string by specifying the starting and ending position using `str_sub()`. Let us extract the currency type from the `currency` column.



```
str_sub(mockdata$currency, start = 1, end = 1)
```

```
[1] "¥" "$" "€" "€" "€" "¥" "$" "¥" "€" "€"
```

One final function that we will look at before the case study is `word()`. It extracts word(s) from sentences. We do not have any sentences in the data set, but let us use it to extract the first and last name from the `full_name` column.



```
word(mockdata$full_name, 1)
```

```
[1] "Mufi"      "Leese"     "Blakelee"  "Terencio"  "Debee"     "Fran"
[7] "Frasco"    "Car"       "Tades"     "Wilton"
```

```
word(mockdata$full_name, 2)
```

```
[1] "Ruit"      "Furmagier" "Wilshire"  "McIllick"  "McErlaine" "Painten"
[7] "Bowich"    "Ponten"    "Checcucci" "Kemney"
```

Alright, now let us apply what we have learned so far to our case study.

9.4 Extract domain name from email ids

9.4.1 Steps

- split email using pattern @

- extract the second element from the resulting list
- split the above using pattern `\\.`
- extract the first element from the resulting list

Let us take a look at the emails before we extract the domain names.

```
emails <-
  mockstring |>
  pull(email) |>
  head()

emails
```

```
[1] "mnewburn0@fastcompany.com"      "mdankersley1@digg.com"
[3] "hgirhard2@altervista.org"      "pmcmenary3@sciencedirect.com"
[5] "drisbrough4@bandcamp.com"      "cphlippi5@surveymonkey.com"
```

9.4.1.1 Step 1: Split email using pattern `@`.

We will split the email using `str_split`. It will split a string using the pattern supplied. In our case the pattern is `@`.

```
str_split(emails, pattern = '@')

[[1]]
[1] "mnewburn0"      "fastcompany.com"

[[2]]
[1] "mdankersley1" "digg.com"

[[3]]
[1] "hgirhard2"      "altervista.org"

[[4]]
[1] "pmcmenary3"      "sciencedirect.com"

[[5]]
[1] "drisbrough4"    "bandcamp.com"

[[6]]
[1] "cphlippi5"      "surveymonkey.com"
```

9.4.1.2 Step 2: Extract the second element from the resulting list.

Step 1 returned a list. Each element of the list has two values. The first one is the username and the second is the domain name. Since we are extracting the domain name, we want the second value from each element of the list.

We will use `map_chr()` from `purrr` to extract the domain names. It will return the second value from each element in the list. Since the domain name is a string, `map_chr()` will return a character vector.

```
emails |>
  str_split(pattern = '@') |>
  map_chr(2)
```

```
[1] "fastcompany.com" "digg.com" "altermvista.org"
[4] "sciencedirect.com" "bandcamp.com" "surveymonkey.com"
```

9.4.1.3 Step 3: Split the above using pattern '\\..'

We want the domain name and not the extension. Step 2 returned a character vector and we need to split the domain name and the domain extension. They are separated by `..`. Since `.` is a special character, we will use `\\` before `.` to escape it. Let us split the domain name and domain extension using `str_split` and `\\..` as the pattern.

```
emails |>
  str_split(pattern = '@') |>
  map_chr(2) |>
  str_split(pattern = '\\..')
```

```
[[1]]
[1] "fastcompany" "com"

[[2]]
[1] "digg" "com"

[[3]]
[1] "altermvista" "org"

[[4]]
[1] "sciencedirect" "com"

[[5]]
[1] "bandcamp" "com"
```

```
[[6]]  
[1] "surveymonkey" "com"
```

9.4.1.4 Step 4: Extract the first element from the resulting list.

Now that we have separated the domain name from its extension, let us extract the first value from each element in the list returned in step 3. We will again use `map_chr` to achieve this.

```
emails |>  
  str_split(pattern = '@') |>  
  map_chr(2) |>  
  str_split(pattern = '\\\\.') |>  
  map_chr(1)
```

```
[1] "fastcompany" "digg" "altervista" "sciencedirect"  
[5] "bandcamp" "surveymonkey"
```

9.5 Extract Domain Extension

The below code extracts the domain extension instead of the domain name.

```
emails |>  
  str_split(pattern = '@') |>  
  map_chr(2) |>  
  str_split(pattern = '\\\\. ', simplify = TRUE) |>  
  (\\(m) m[, 2])()
```

```
[1] "com" "com" "org" "com" "com" "com"
```

9.6 Extract image type from URL

9.6.1 Steps

- split imageurl using pattern `\\.`
- extract the third value from each element of the resulting list
- subset the string using the index position

Let us take a look at the URL of the image.

```
img <-
  mockstring |>
  pull(imageurl) |>
  head()

img
```

```
[1] "http://dummyimage.com/130x183.jpg/dddddd/000000"
[2] "http://dummyimage.com/106x217.bmp/dddddd/000000"
[3] "http://dummyimage.com/146x127.bmp/cc0000/ffffff"
[4] "http://dummyimage.com/181x194.png/5fa2dd/ffffff"
[5] "http://dummyimage.com/220x123.jpg/ff4444/ffffff"
[6] "http://dummyimage.com/118x176.bmp/dddddd/000000"
```

9.6.1.1 Step 1: Split imageurl using pattern \\.

Let us split imageurl using `str_split` and the pattern `\\.`

```
str_split(img, pattern = '\\.')
```

```
[[1]]
[1] "http://dummyimage" "com/130x183"      "jpg/dddddd/000000"

[[2]]
[1] "http://dummyimage" "com/106x217"      "bmp/dddddd/000000"

[[3]]
[1] "http://dummyimage" "com/146x127"      "bmp/cc0000/ffffff"

[[4]]
[1] "http://dummyimage" "com/181x194"      "png/5fa2dd/ffffff"

[[5]]
[1] "http://dummyimage" "com/220x123"      "jpg/ff4444/ffffff"

[[6]]
[1] "http://dummyimage" "com/118x176"      "bmp/dddddd/000000"
```

9.6.1.2 Step 2: Extract the third value from each element of the resulting list

Step 1 returned a list the elements of which have 3 values each. If you observe the list, the image type is in the 3rd value. We will now extract the third value from each element of the list using `map_chr`.

```
img |>
  str_split(pattern = '\\\\.') |>
  map_chr(3)
```

```
[1] "jpg/dddddd/000000" "bmp/dddddd/000000" "bmp/cc0000/ffffff"
[4] "png/5fa2dd/ffffff" "jpg/ff4444/ffffff" "bmp/dddddd/000000"
```

9.6.1.3 Step 3: Subset the string using the index position

We can now extract the image type in two ways:

- subset the first 3 characters of the string
- split the string using pattern / and extract the first value from the elements of the resulting list

Below is the first method. We know that the image type is 3 characters. So we use `str_sub` to subset the first 3 characters. The index positions are mentioned using `start` and `stop`.

```
img |>
  str_split(pattern = '\\\\.') |>
  map_chr(3) |>
  str_sub(start = 1, end = 3)
```

```
[1] "jpg" "bmp" "bmp" "png" "jpg" "bmp"
```

In case you are not sure about the length of the image type. In such cases, we will split the string using pattern / and then use `map_chr` to extract the first value of each element of the resulting list.

```
img |>
  str_split(pattern = '\\\\.') |>
  map_chr(3) |>
  str_split(pattern = '/') |>
  map_chr(1)
```

```
[1] "jpg" "bmp" "bmp" "png" "jpg" "bmp"
```

9.7 Extract Image Dimension from URL

9.7.1 Steps

- locate numbers between 0 and 9
- extract part of url starting with image dimension
- split the string using the pattern `\\.`
- extract the first element

9.7.1.1 Step 1: Locate numbers between 0 and 9.

Let us inspect the image url. The dimension of the image appears after the domain extension and there are no numbers in the url before. We will locate the position or index of the first number in the url using `str_locate()` and using the pattern `[0-9]` which instructs to look for any number between and including 0 and 9.

```
str_locate(img, pattern = "[0-9]")
```

```
      start end
[1,]     23  23
[2,]     23  23
[3,]     23  23
[4,]     23  23
[5,]     23  23
[6,]     23  23
```

9.7.1.2 Step 2: Extract url

We know where the dimension is located in the url. Let us extract the part of the url that contains the image dimension using `str_sub()`.

```
str_sub(img, start = 23)
```

```
[1] "130x183.jpg/dddddd/000000" "106x217.bmp/dddddd/000000"
[3] "146x127.bmp/cc0000/ffffff" "181x194.png/5fa2dd/ffffff"
[5] "220x123.jpg/ff4444/ffffff" "118x176.bmp/dddddd/000000"
```

9.7.1.3 Step 3: Split the string using the pattern '\\..'

From the previous step, we have the part of the url that contains the image dimension. To extract the dimension, we will split it from the rest of the url using `str_split()` and using the pattern '\\..' as it separates the dimension and the image extension.

```
img |>
  str_sub(start = 23) |>
  str_split(pattern = '\\..')
```



```
[[1]]
[1] "130x183"      "jpg/dddddd/000000"

[[2]]
[1] "106x217"      "bmp/dddddd/000000"

[[3]]
[1] "146x127"      "bmp/cc0000/ffffff"

[[4]]
[1] "181x194"      "png/5fa2dd/ffffff"

[[5]]
[1] "220x123"      "jpg/ff4444/ffffff"

[[6]]
[1] "118x176"      "bmp/dddddd/000000"
```

9.7.1.4 Step 4: Extract the first element.

The above step resulted in a list which contains the image dimension and the rest of the url. Each element of the list is a character vector. We want to extract the first value in the character vector. Let us use `map_chr()` to extract the first value from each element of the list.

```
img |>
  str_sub(start = 23) |>
  str_split(pattern = '\\..') |>
  map_chr(1)
```



```
[1] "130x183" "106x217" "146x127" "181x194" "220x123" "118x176"
```

9.8 Extract HTTP Protocol from URL

```
url1 <-  
  mockstring |>  
  pull(url1) |>  
  first()  
  
url1
```

```
[1] "https://engadget.com/nascetur/ridiculus/mus/vivamus/vestibulum.jsp?eu=est&tincidunt=r
```

9.8.1 Steps

- split the url using the pattern ://
- extract the first element

9.8.1.1 Step 1: Split the url using the pattern ://.

The HTTP protocol is the first part of the url and is separated from the rest of the url by `://`. Let us split the url using `str_split()` and using the pattern `://`. Since `:` is a special character, we will escape it using `\\`.

```
str_split(url1, pattern = '://')
```

```
[[1]]  
[1] "https"  
[2] "engadget.com/nascetur/ridiculus/mus/vivamus/vestibulum.jsp?eu=est&tincidunt=risus&in=
```

9.8.1.2 Step 2: Extract the first element.

The HTTP protocol is the first value in each element of the list. As we did in the previous example, we will extract it using `map_chr()` with the element position.

```
url1 |>  
  str_split(pattern = '://') |>  
  map_chr(1)
```

```
[1] "https"
```

9.9 Extract file type

```
urls <-  
  mockstring |>  
  pull(url) |>  
  head(3)
```

9.9.1 Steps

- check if there are only 2 dots in the URL
- check if there is only 1 question mark in the URL
- detect the starting position of file type
- detect the ending position of file type
- use the locations to specify the index position for extracting file type

9.9.1.1 Step 1: Check if there are only 2 dots in the URL

Let us locate all the dots in the url using `str_locate_all()` and see if any of them contain more than 2 dots.

```
urls |>  
  str_locate_all(pattern = '\\\\.') |>  
  map_int(nrow) |>  
  is_greater_than(2) |>  
  sum()
```

```
[1] 0
```

9.9.1.2 Step 2: Check if there is only 1 question mark in the URL

The next step is to check if there is only one ? (question mark) in the url.

```
urls |>  
  str_locate_all(pattern = "[?]") |>  
  map_int(nrow) |>  
  is_greater_than(1) |>  
  sum()
```

```
[1] 0
```

9.9.1.3 Step 3: Detect the starting position of file type

Since the file type is located between the second dot and the first question mark in the url, let us extract the location of the second dot and add 1 as the file type starts after the dot.

```
d <-  
  urls |>  
  str_locate_all(pattern = '\\\\.') |>  
  map_int(2) |>  
  add(1)  
  
d
```

```
[1] 64 47 48
```

9.9.1.4 Step 4: Detect the ending position of file type

In step 2, we confirmed that the url has only one question mark. Let us locate the question mark in the url and subtract 1 (as the file type ends before the question mark) so that we get the ending position of the file type. .

```
q <-  
  urls |>  
  str_locate_all(pattern = "[?]") |>  
  map_int(1) |>  
  subtract(1)  
  
q
```

```
[1] 66 50 51
```

9.9.1.5 Step 5: Specify the index position for extracting file type

From steps 3 and 4, we have the location of the second dot and the first question mark in the url. Let us use them with `str_sub()` to extract the file type.

```
str_sub(urls, start = d, end = q)
```

```
[1] "jsp" "json" "json"
```

Try it live

💡 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(stringr)
library(readr)
if (!file.exists("mockstrings_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/mockstrings_sample.csv",
    "mockstrings_sample.csv"
  )
}
mock <- read_csv("mockstrings_sample.csv", show_col_types = FALSE)
str_detect(mock$email, pattern = "@")
```

10 Date & Time: Parsing and Components

i Continued in part B

This chapter was split for readability. Arithmetic, intervals, time zones and rounding continue in @date-arithmetic-time-zones.

This chapter is the first of two on dates and times. Here we parse character data into dates and extract components (year, month, weekday, quarter). @date-arithmetic-time-zones continues with arithmetic, intervals, time zones, rounding and rollback.

10.1 Introduction

Let us begin by looking at the current date and time.

10.1.1 Date

`Sys.Date()` and `today()` will return the current date.

```
Sys.Date()
```

```
[1] "2026-10-02"
```

```
lubridate::today()
```

```
[1] "2026-10-02"
```

10.1.2 Time

`Sys.time()` and `now()` return the date, time and the timezone. In `now()`, we can specify the timezone using the `tzzone` argument.

```
Sys.time()
```

```
[1] "2026-10-02 08:23:42 UTC"
```

```
lubridate::now()
```

```
[1] "2026-10-02 08:23:42 UTC"
```

```
lubridate::now(tzone = "UTC")
```

```
[1] "2026-10-02 08:23:42 UTC"
```

10.1.3 AM or PM?

`am()` and `pm()` allow us to check whether date/time occur in the am or pm? They return a logical value i.e. TRUE or FALSE

```
lubridate::am(now())
```

```
[1] TRUE
```

```
lubridate::pm(now())
```

```
[1] FALSE
```

10.1.4 Leap Year

We can also check if the current year is a leap year using `leap_year()`.

```
lubridate::leap_year(Sys.Date())
```

```
[1] FALSE
```

10.1.5 Summary

Function	Description
<code>'Sys.Date()'</code>	Current Date
<code>'lubridate::today()'</code>	Current Date
<code>'Sys.time()'</code>	Current Time
<code>'lubridate::now()'</code>	Current Time
<code>'lubridate::am()'</code>	Whether time occurs in am?
<code>'lubridate::pm()'</code>	Whether time occurs in pm?

`'lubridate::leap_year()'` Check if the year is a leap year?

10.1.6 Your Turn

- get current date
- get current time
- check whether the time occurs in am or pm?
- check whether the following years were leap years
 - 2018
 - 2016

10.2 Case Study

Throughout the tutorial, we will work on a case study related to transactions of a imaginary company. The data set includes information about invoice and payment dates.

10.2.1 Data

```
transact <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/main/transact.csv')
```

```
# A tibble: 2,466 x 3
  Invoice    Due    Payment
  <date>    <date>    <date>
1 2013-01-02 2013-02-01 2013-01-15
2 2013-01-26 2013-02-25 2013-03-03
3 2013-07-03 2013-08-02 2013-07-08
4 2013-02-10 2013-03-12 2013-03-17
5 2012-10-25 2012-11-24 2012-11-28
6 2012-01-27 2012-02-26 2012-02-22
7 2013-08-13 2013-09-12 2013-09-09
8 2012-12-16 2013-01-15 2013-01-12
9 2012-05-14 2012-06-13 2012-07-01
10 2013-07-01 2013-07-31 2013-07-26
# i 2,456 more rows
```

10.2.2 Data Dictionary

The data set has 3 columns. All the dates are in the format (yyyy-mm-dd).

Column	Description
Invoice	Invoice Date
Due	Due Date
Payment	Payment Date

In the case study, we will try to answer a few questions we have about the `transact` data.

- extract date, month and year from Due
- compute the number of days to settle invoice
- compute days over due
- check if due year is a leap year
- check when due day in february is 29, whether it is a leap year
- how many invoices were settled within due date
- how many invoices are due in each quarter

10.3 Date & Time Classes

10.3.1 Introduction

In this section, we will look at two things. First, how to create date/time data in R, and second, how to convert other data types to date/time. Let us begin by creating the latest R release date manually.

```
release_date <- 2019-12-12
release_date
```

```
[1] 1995
```

Okay! Why do we see 1995 when we call the date? What is happening here? Let us quickly check the data type of `release_date`.

```
class(release_date)
```

```
[1] "numeric"
```

The data type is `numeric` i.e. R has subtracted 12 twice from 2019 to return 1995. Clearly, the above method is not the right way to store date/time. Let us see if we can get some hints from the builtin R functions we used in the previous section. If you observe the output, all of them returned date/time wrapped in quotes. Hmmm... let us wrap our date in quotes and see what happens.

```
release_date <- "2019-12-12"  
release_date
```

```
[1] "2019-12-12"
```

Alright, now R does not do any arithmetic and returns the date as we specified. Great! Is this the right format to store date/time? No. Why? What is the problem if date/time is saved as character/string? The problem is the nature or type of operations done on date or time is different when compared to string/character, number or logical values.

- how do we add/subtract dates?
- how do we extract components such as year, month, day etc.

To answer the above questions, we will first check the data type of `Sys.Date()` and `now()`.

```
class(Sys.Date())
```

```
[1] "Date"
```

```
class(lubridate::now())
```

```
[1] "POSIXct" "POSIXt"
```

```
class(release_date)
```

```
[1] "character"
```

As you can see from the above output, there are 3 different classes for storing date/time in R

- Date
- POSIXct
- POSIXlt

Let us explore each of the above classes one by one.

10.3.2 Date

10.3.2.1 Introduction

The `Date` class represents calendar dates. Let us go back to `Sys.Date()`. If you check the class of `Sys.Date()`, it is `Date`. Internally, this date is a number i.e. an integer. The `unclass()` function will show dates are stored internally.

```
unclass(Sys.Date())
```

```
[1] 20728
```

What does this integer represent? Why has R stored the date as an integer? Before we answer this question, we need to know something else. In R, dates are represented as the number of days since 1970-01-01. All the dates in R are internally stored in this way. Before we explore this concept further, let us learn to create `Date` objects in R. We will continue to use the latest R release date, 2019-12-12.

Until now, we have stored the above date as character/string but now we will use `as.Date()` to save it as a `Date` object. `as.Date()` is the easiest and simplest way to create dates in R.

```
release_date <- as.Date("2019-12-12")  
release_date
```

```
[1] "2019-12-12"
```

The `as_date()` function from the `lubridate` package is similar to `as.Date()`.

```
release_date <- lubridate::as_date("2019-12-12")  
release_date
```

```
[1] "2019-12-12"
```

If you look at the difference between `release_date` and 1970-01-01, it will be the same as `unclass(release_date)`.

```
release_date - as.Date("1970-01-01")
```

Time difference of 18242 days

```
unclass(release_date)
```

```
[1] 18242
```

Let us come back to 1970-01-01 i.e. the origin for dates in R.

```
lubridate::origin
```

```
[1] "1970-01-01 UTC"
```

From the previous examples, we know that dates are internally stored as number of days since 1970-01-01. How about dates older than the origin? How are they stored? Let us look at that briefly.

```
unclass(as.Date("1963-08-28"))
```

```
[1] -2318
```

Dates older than the origin are stored as negative integers. For those who are not aware, Martin Luther King, Jr. delivered his famous **I Have a Dream** speech on 1963-08-28. Let us move on and learn how to convert numbers into dates.

10.3.2.2 Convert Numeric

The `as.Date()` function can be used to convert any of the following to a `Date` object

- character/string
- number
- factor (categorical/qualitative)

We have explored how to convert strings to date. How about converting numbers to date? Sure, we can create date from numbers by specifying the origin and number of days since it.

```
as.Date(18242, origin = "1970-01-01")
```

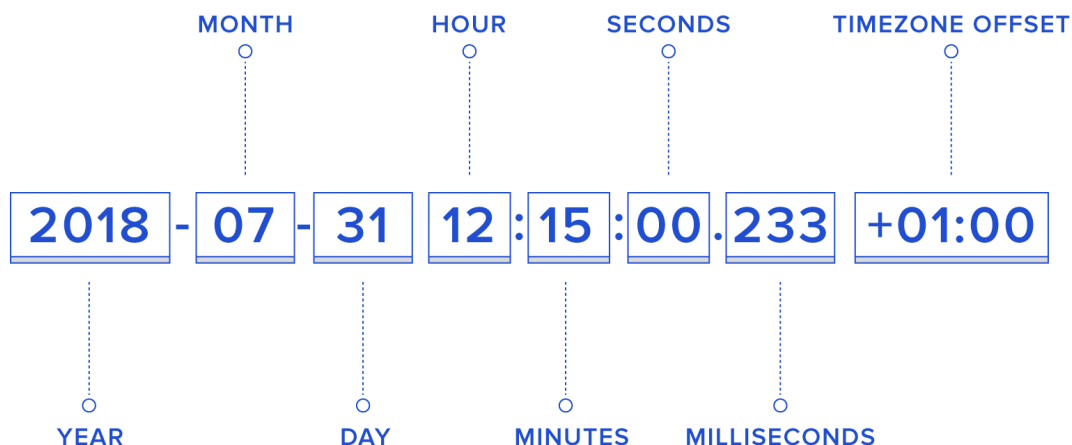
```
[1] "2019-12-12"
```

The origin can be changed to another date (while changing the number as well.)

```
as.Date(7285, origin = "2000-01-01")
```

```
[1] "2019-12-12"
```

10.3.3 ISO 8601



If you have carefully observed, the format in which we have been specifying the dates as well as of those returned by functions such as `Sys.Date()` or `Sys.time()` is the same i.e. YYYY-MM-DD. It includes

- the year including the century
- the month
- the date

The month and date separated by `-`. This default format used in R is the ISO 8601 standard for date/time. ISO 8601 is the internationally accepted way to represent dates and times and uses the 24 hour clock system. Let us create the release date using another function `ISOdate()`.

```
ISOdate(year = 2019,  
        month = 12,  
        day   = 12,  
        hour  = 8,  
        min   = 5,  
        sec   = 3,  
        tz    = "UTC")
```

```
[1] "2019-12-12 08:05:03 UTC"
```

We will look at all the different weird ways in which date/time are specified in the real world in the [Date & Time Formats](#) section. For the time being, let us continue exploring date/time classes in R. The next class we are going to look at is `POSIXct/POSIXlt`.

10.3.4 POSIX

You might be wondering what is this POSIX thing? POSIX stands for **P**ortable **O**perating **S**ystem **I**nterface. It is a family of standards specified for maintaining compatibility between different operating systems. Before we learn to create POSIX objects, let us look at `now()` from `lubridate`.

```
class(lubridate::now())
```

```
[1] "POSIXct" "POSIXt"
```

`now()` returns current date/time as a `POSIXct` object. Let us look at its internal representation using `unclass()`

```
unclass(lubridate::now())
```

```
[1] 1790929422  
attr("tzone")  
[1] ""
```

The output you see is the number of seconds since January 1, 1970.

10.3.4.1 POSIXct

`POSIXct` represents the number of seconds since the beginning of 1970 (UTC) and `ct` stands for calendar time. To store date/time as `POSIXct` objects, use `as.POSIXct()`. Let us now store the latest R release date as `POSIXct` as shown below

```
release_date <- as.POSIXct("2019-12-12 08:05:03")  
class(release_date)
```

```
[1] "POSIXct" "POSIXt"
```

```
unclass(release_date)
```

```
[1] 1576137903  
attr("tzone")  
[1] ""
```

10.3.4.2 POSIXlt

POSIXlt represents the following information in a list

- seconds
- minutes
- hour
- day of the month
- month
- year
- day of week
- day of year
- daylight saving time flag
- time zone
- offset in seconds from GMT

The `lt` in `POSIXlt` stands for local time. Use `as.POSIXlt()` to store date/time as `POSIXlt` objects. Let us store the release date as a `POSIXlt` object as shown below

```
release_date <- as.POSIXlt("2019-12-12 08:05:03")
release_date
```

```
[1] "2019-12-12 08:05:03 UTC"
```

As we said earlier, `POSIXlt` stores date/time components in a list and these can be extracted. Let us look at the date/time components returned by `POSIXlt` using `unclass()`.

```
release_date <- as.POSIXlt("2019-12-12 08:05:03")
unclass(release_date)
```

```
$sec
[1] 3
```

```
$min
[1] 5
```

```
$hour
[1] 8
```

```
$mday
[1] 12
```

```
$mon
[1] 11
```

```

$year
[1] 119

$yday
[1] 4

$yday
[1] 345

$isdst
[1] 0

$zone
[1] "UTC"

$gmtoff
[1] 0

attr(,"tzone")
[1] "UTC"
attr(,"balanced")
[1] TRUE

```

Use `unlist()` if you want the components returned as a **vector**.

```

release_date <- as.POSIXlt("2019-12-12 08:05:03")
unlist(release_date)

```

sec	min	hour	mday	mon	year	wday	yday	isdst	zone	gmtoff
"3"	"5"	"8"	"12"	"11"	"119"	"4"	"345"	"0"	"UTC"	"0"

To extract specific components, use `$`.

```

release_date <- as.POSIXlt("2019-12-12 08:05:03")
release_date$hour

```

```

[1] 8

```

```

release_date$mon

```

```

[1] 11

```

```
release_date$zone
```

```
[1] "UTC"
```

Now, let us look at the components returned by `POSIXlt`. Some of them are intuitive

Component	Description
'sec'	Second
'min'	Minute
'hour'	Hour of the day
'mon'	Month of the year (0-11)
'zone'	Timezone
'wday'	Day of week
'mday'	Day of month
'year'	Years since 1900
'yday'	Day of year
'isdst'	Daylight saving flag
'gmtoff'	Offset in seconds from GMT

Great! We will end this section with a few tips/suggestions on when to use `Date` or `POSIXct`/`POSIXlt`.

- use `Date` when there is no time component
- use `POSIX` when dealing with time and timezones
- use `POSIXlt` when you want to access/extract the different components

10.3.5 Your Turn

R 1.0.0 was released on 2000-02-29 08:55:23 UTC. Save it as

- `Date` using character
- `Date` using origin and number
- `POSIXct`
- `POSIXlt` and extract
 - month
 - day
 - month
 - zone
- `ISODate`

10.4 Date & Time Formats

10.4.1 Introduction

After the timezones and daylight savings detour, let us get back on path and explore another important aspect, date & time formats. Although it is a good practice to adhere to ISO 8601 format, not all date/time data will comply with it. In real world, date/time data may come in all types of weird formats. Below is a sample

Format
December 12, 2019
12th Dec, 2019
Dec 12th, 19
12-Dec-19
2019 December
12.12.19

When the data is not in the default ISO 8601 format, we need to explicitly specify the format in R. We do this using conversion specifications. A conversion specification is introduced by %, usually followed by a single letter or O or E and then a single letter.

10.4.2 Conversion Specifications

Specification	Description	Example
'%d'	Day of the month (decimal number)	12
'%m'	Month (decimal number)	12
'%b'	Month (abbreviated)	Dec
'%B'	Month (full name)	December
'%y'	Year (2 digit)	19
'%Y'	Year (4 digit)	2019
%H	Hour	8
%M	Minute	5
%S	Second	3

Time to work through a few examples. Let us say you are dealing with dates in the format 19/12/12. In this format, the year comes first followed by month and the date; each separated by a backslash (/). The year consists of only 2 digits i.e. it does not include the century. Let us now map each component of the date to the format table shown at the beginning.

Date	Specification
19	'%y'
12	'%m'
12	'%d'

Using the format argument, we will specify the date format as a character vector i.e. enclosed in quotes.

```
as.Date("19/12/12", format = "%y/%m/%d")
```

```
[1] "2019-12-12"
```

Another way in which the release data can be written is 2019-Dec-12. We still have the year followed by the month and the date but there are a few changes here:

- the components are separated by a - instead of /
- year has 4 digits i.e. includes the century
- the month is specified using abbreviation instead of digits

Let us map the components to the format table:

Date	Specification
2019	'%Y'
Dec	'%b'
12	'%d'

Let us specify the format for the date using the above mapping.

```
as.Date("2019-Dec-12", format = "%Y-%b-%d")
```

```
[1] "2019-12-12"
```

In both the above examples, we have not dealt with time components. Let us include the time of the latest R release in the next one i.e. 19/12/12 08:05:03.

Date	Specification
19	'%y'
12	'%m'
12	'%d'
08	'%H'
05	'%M'

Since we are dealing with time, we will use `as.POSIXct()` instead of `as.Date()`.

```
as.POSIXct("19/12/12 08:05:03", tz = "UTC", format = "%y/%m/%d %H:%M:%S")
```

```
[1] "2019-12-12 08:05:03 UTC"
```

In the below table, we look at some of the most widely used conversion specifications. You can learn more about these specifications by running `?strptime` or `help(strptime)`.

Specification	Description
‘%a’	Abbreviated weekday
‘%A’	Full weekday
‘%C’	Century (00-99)
‘%D’	Same as ‘%m/%d/%y’
‘%e’	Day of month [1 - 31]
‘%F’	Same as ‘%Y-%m-%d’
‘%h’	Same as ‘%b’
‘%I’	Hours as decimal [01 - 12]
‘%j’	Day of year [001 - 366]
‘%R’	Same as ‘%H:%M’
‘%t’	Tab
‘%T’	Same as ‘%H:%M:%S’
‘%u’	Weekday [1 - 7](Monday is 1)
‘%U’	Week of year [00 - 53]
‘%V’	Week of year [01 - 53]
‘%w’	Weekday [0 - 6](sunday is 0)
‘%W’	Week of year [00 - 53]

We have included a lot of practice questions for you to explore the different date/time formats. The solutions are available in the Learning Management system as well as in our GitHub repo. Try them and let us know if you have any doubts.

10.4.3 Guess Format

`guess_formats()` from `lubridate` is a very useful function. It will guess the date/time format if you specify the order in which year, month, date, hour, minute and second appear.

```
release_date_formats <- c("December 12th 2019",
                          "Dec 12th 19",
                          "dec 12 2019")

guess_formats(release_date_formats,
              orders = "mdy",
              print_matches = TRUE)
```

```

               Omdy          mdy
[1,] "December 12th 2019" "%Om %dth %Y" "%B %dth %Y"
[2,] "Dec 12th 19"       "%Om %dth %y" "%b %dth %y"
[3,] "dec 12 2019"       "%Om %d %Y"  "%b %d %Y"
```

```

               Omdy          Omdy          Omdy          mdy          mdy
"%Om %dth %Y" "%Om %dth %y" "%Om %d %Y" "%B %dth %Y" "%b %dth %y"
               mdy
               "%b %d %Y"
```

10.4.4 Your Turn

Below, we have specified July 5th, 2019 in different ways. Create the date using `as.Date()` while specifying the correct format for each of them.

- July-05-19
- JUL-05-19
- 05.07.19
- 5-July 2019
- July 5th, 2019
- July 05, 2019
- 2019-July- 05
- 05/07/2019
- 07/05/2019
- 7/5/2019
- 07/5/19
- 2019-07-05

10.5 Parse Date & Time

While creating date-time objects, we specified different formats using the conversion specification but most often you will not create date/time and instead deal with data that comes your way from a system or colleague/collaborator. In such cases, we need to be able to parse date/time from the data provided to us. In this section, we will focus on parsing date/time

from character data. Both base R and the lubridate package offer functions to parse date and time and we will explore a few of them in this section. We will initially use functions from base R and later on explore those from lubridate which will give us an opportunity to compare and contrast. It will also allow us to choose the functions based on the data we are dealing with.

`strptime()` will convert character data to `POSIXlt`. You will use this when converting from character data to date/time. On the other hand, if you want to convert date/time to character data, use any of the following:

- `strftime()`
- `format()`
- `as.character()`

The above functions will convert `POSIXct`/`POSIXlt` to character. Let us start with a simple example. The data we have been supplied has date/time as character data and in the format `YYYYMMDD` i.e. nothing separates the year, month and date from each other. We will use `strptime()` to convert this to an object of class `POSIXlt`.

```
rel_date <- strptime("20191212", format = "%Y%m%d")
class(rel_date)
```

```
[1] "POSIXlt" "POSIXt"
```

If you have a basic knowledge of conversion specifications, you can use `strptime()` to convert character data to `POSIXlt`. Let us quickly explore the functions to convert date/time to character data before moving on to the functions from lubridate.

```
rel_date_strf <- strftime(rel_date)
class(rel_date_strf)
```

```
[1] "character"
```

```
rel_date_format <- format(rel_date)
class(rel_date_format)
```

```
[1] "character"
```

```
rel_date_char <- as.character(rel_date)
class(rel_date_char)
```

```
[1] "character"
```

As you can see, all the 3 functions converted date/time to character. Time to move on and explore the lubridate package. We will start with an example in which the release date is formatted in 3 different ways but they have one thing in common i.e. the order in which the components appear. In all the 3 formats, the year is followed by the month and then the date.

To parse the release date, we will use `parse_date_time()` from lubridate which parses the input into POSIXct objects.

```
release_date <- c("19-12-12", "20191212", "19-12 12")
parse_date_time(release_date, "ymd")
```

```
[1] "2019-12-12 UTC" "2019-12-12 UTC" "2019-12-12 UTC"
```

```
parse_date_time(release_date, "y m d")
```

```
[1] "2019-12-12 UTC" "2019-12-12 UTC" "2019-12-12 UTC"
```

```
parse_date_time(release_date, "%y%m%d")
```

```
[1] "2019-12-12 UTC" "2019-12-12 UTC" "2019-12-12 UTC"
```

Try to use `strptime()` in the above example and see what happens. Now, let us look at another data set.

```
release_date <- c("19-07-05", "2019-07-05", "05-07-2019", "07-05-2019")
```

What happens in the below case? The same date appears in multiple formats. How do we parse them? `parse_date_time()` allows us to specify mutiple date-time formats. Let us first map the dates to their formats.

Date	Specification
19-07-05	'ymd'
2019-07-05	'ymd'
05-07-2019	'dmy'
07-05-2019	'mdy'

The above specifications can be supplied as a character vector.

```
parse_date_time(release_date, c("ymd", "ymd", "dmy", "mdy"))
```

```
[1] "2019-07-05 UTC" "2019-07-05 UTC" "2019-07-05 UTC" "2019-05-07 UTC"
```

Great! We have used both `strptime()` and `parse_date_time()` now. Can you tell what differentiates `parse_date_time()` when compared to `strptime()`? We summarize it in the points below:

- no need to include % prefix or separator
- specify several date/time formats

There are other helper functions that can be used to

- parse dates with year, month, day components
- parse dates with year, month, day, hour, minute, seconds components
- parse period with hour, minute, second components

and are explored in the below examples.

```
# year/month/date
ymd("2019-12-12")
```

```
[1] "2019-12-12"
```

```
# year/month/date
ymd("19/12/12")
```

```
[1] "2019-12-12"
```

```
# date/month/year
dmy(121219)
```

```
[1] "2019-12-12"
```

```
# year/month/date/hour/minute/second
ymd_hms(191212080503)
```

```
[1] "2019-12-12 08:05:03 UTC"
```

```
# hour/minute/second
hms("8, 5, 3")
```

```
[1] "8H 5M 3S"
```

```
# hour/minute/second  
hms("08:05:03")
```

```
[1] "8H 5M 3S"
```

```
# minute/second  
ms("5,3")
```

```
[1] "5M 3S"
```

```
# hour/minute  
hm("8, 5")
```

```
[1] "8H 5M 0S"
```

Note, in a couple of cases where the components are not separated by /, - or space, we have not enclosed the values in quotes.

10.5.1 Your Turn

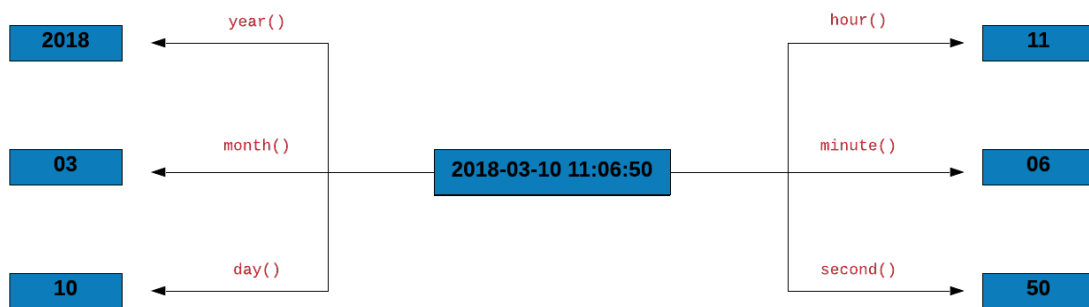
Below, we have specified July 5th, 2019 in different ways. Parse the dates using `strptime()` or `parse_date_time()` or any other helper function.

- July-05-19
- JUL-05-19
- 05.07.19
- 5-July 2019
- July 5th, 2019
- July 05, 2019
- 2019-July- 05
- 05/07/2019
- 07/05/2019
- 7/5/2019
- 07/5/19
- 2019-07-05

10.6 Date & Time Components

In the second section, we discussed the downside of saving date/time as character/string in R. One of the points we discussed was that we can't extract components such as year, month, day etc. In this section, we will learn to extract date/time components such as

- year
- month
- date
- week
- day
- quarter
- semester
- hour
- minute
- second
- timezone



The below table outlines the functions we will explore in the first part of this section.

Function	Description
'year()'	Get year
'month()'	Get month (number)
'month(label = TRUE)'	Get month (abbreviated name)
'month(abbr = FALSE)'	Get month (full name)
'months()'	Get month
'week()'	Get week

10.6.1 Year

```
release_date <- ymd_hms("2019-12-12 08:05:03")
year(release_date)
```

```
[1] 2019
```

10.6.2 Month

`month()` will return the month as a number i.e. 07 for July.

```
month(release_date)
```

```
[1] 12
```

If you want the name of the month instead, use the `label` argument and set it to `TRUE`. Now it returns Jul instead of 07.

```
month(release_date, label = TRUE)
```

```
[1] Dec
12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

But this is the abbreviated name and not the full name. How do we get the full name of the month? Set the `abbr` argument to `FALSE`.

```
month(release_date, label = TRUE, abbr = FALSE)
```

```
[1] December
12 Levels: January < February < March < April < May < June < ... < December
```

Ah! now we can see the full name of the month. `months()` from base R will return the full name of the month by default. If you want the abbreviated name, use the `abbreviate` argument and set it to `TRUE`.

```
months(release_date)
```

```
[1] "December"
```

10.6.3 Week

`week()` returns the number of complete 7 day periods between the date and 1st January plus one.

```
week(release_date)
```

```
[1] 50
```

10.6.4 Day

Use `day()` to extract the date component. There are other variations such as

Function	Description
<code>'day'</code>	Get day
<code>'mday()'</code>	Day of the month
<code>'wday()'</code>	Day of the week
<code>'qday()'</code>	Day of quarter
<code>'yday()'</code>	Day of year
<code>'weekdays()'</code>	Day of week
<code>'days_in__month()'</code>	Days in the month

```
day(release_date)
```

```
[1] 12
```

```
mday(release_date)
```

```
[1] 12
```

```
qday(release_date)
```

```
[1] 73
```

```
yday(release_date)
```

```
[1] 346
```

`wday` can return

- a number
- abbreviation of the weekday
- full name of the weekday

```
wday(release_date)
```

```
[1] 5
```

```
wday(release_date, label = TRUE)
```

```
[1] Thu  
Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

```
wday(release_date, label = TRUE, abbr = FALSE)
```

```
[1] Thursday  
7 Levels: Sunday < Monday < Tuesday < Wednesday < Thursday < ... < Saturday
```

`weekdays()` from base R also returns the day of the week (the name and not the number). If you want the abbreviated name, use the `abbreviate` argument.

```
weekdays(release_date)
```

```
[1] "Thursday"
```

```
weekdays(release_date, abbreviate = TRUE)
```

```
[1] "Thu"
```

10.6.5 Days in Month

If you want to know the number of days in the month, use `days_in_month()`. In our example, the month is December and it has 31 days.

```
days_in_month(release_date)
```

```
Dec  
31
```

10.6.6 Hour, Minute & Seconds

Function	Description
<code>'hour()'</code>	Get hour
<code>'minute()'</code>	Get minute
<code>'second()'</code>	Get second
<code>'seconds()'</code>	Number of seconds since '1970-01-01'

So far we have been looking at date components. Now, let us look at time components.

```
hour(release_date)
```

```
[1] 8
```

```
minute(release_date)
```

```
[1] 5
```

```
second(release_date)
```

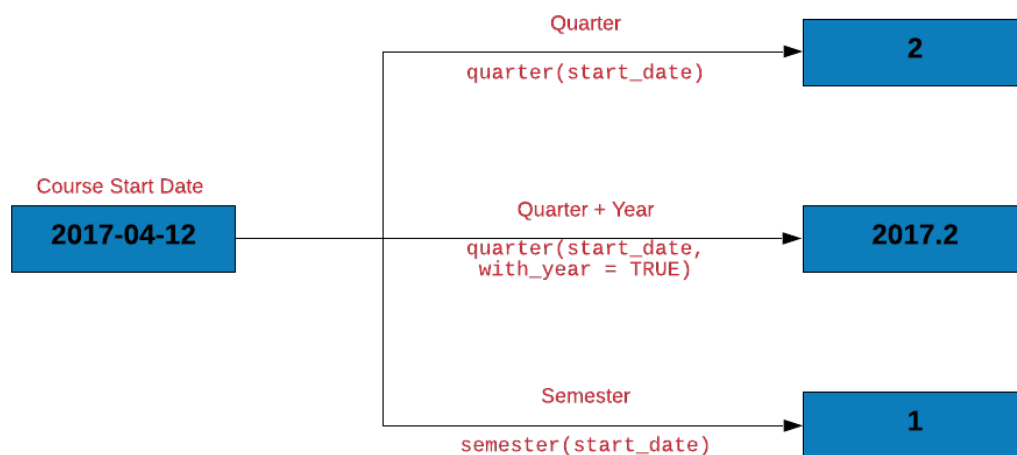
```
[1] 3
```

`seconds()` returns the number of seconds since 1970-01-01.

```
seconds(release_date)
```

```
[1] "1576137903S"
```

10.6.7 Quarter & Semester



`quarter()` will return the quarter from the date. December is in the 4th quarter and hence it returns 4.

```
quarter(release_date)
```

```
[1] 4
```

If you want the year along with the quarter, set the `with_year` argument to `TRUE`.

```
quarter(release_date, with_year = TRUE)
```

```
[1] 2019.4
```

In India, the fiscal starts in April and December falls in the 3rd quarter. How can we accommodate this change? The `fiscal_start` argument allows us to set the month in which the fiscal begins. We will set it to 4 for April. Now it returns 3 instead of 4.

```
quarter(release_date, fiscal_start = 4)
```

```
[1] 3
```

`quarters()` from base R also returns the quarter.

```
quarters(release_date)
```

```
[1] "Q4"
```

Function	Description
<code>'quarter()'</code>	Get quarter
<code>'quarter(with_year = TRUE)'</code>	Quarter with year
<code>'quarter(fiscal_start = 4)'</code>	Fiscal starts in April
<code>'quarters()'</code>	Get quarter
<code>'semester()'</code>	Get semester

10.6.8 Case Study

10.6.8.1 Extract Date, Month & Year from Due Date

Let us now extract the date, month and year from the Due column.

```
transact |>
  mutate(
    due_day   = day(Due),
    due_month = month(Due),
    due_year  = year(Due)
  )
```

```
# A tibble: 2,466 x 6
```

	Invoice <date>	Due <date>	Payment <date>	due_day <int>	due_month <dbl>	due_year <dbl>
1	2013-01-02	2013-02-01	2013-01-15	1	2	2013
2	2013-01-26	2013-02-25	2013-03-03	25	2	2013
3	2013-07-03	2013-08-02	2013-07-08	2	8	2013
4	2013-02-10	2013-03-12	2013-03-17	12	3	2013
5	2012-10-25	2012-11-24	2012-11-28	24	11	2012
6	2012-01-27	2012-02-26	2012-02-22	26	2	2012
7	2013-08-13	2013-09-12	2013-09-09	12	9	2013
8	2012-12-16	2013-01-15	2013-01-12	15	1	2013
9	2012-05-14	2012-06-13	2012-07-01	13	6	2012
10	2013-07-01	2013-07-31	2013-07-26	31	7	2013

```
# i 2,456 more rows
```

10.6.8.2 Data Sanitization

Let us do some data sanitization. If the due day happens to be February 29, let us ensure that the due year is a leap year. Below are the steps to check if the due year is a leap year:

- we will extract the following from the due date:
 - day
 - month
 - year
- we will then create a new column `is_leap` which will have be set to `TRUE` if the year is a leap year else it will be set to `FALSE`
- filter all the payments due on 29th Feb
- select the following columns:
 - Due
 - is_leap

```
transact |>
  mutate(
    due_day   = day(Due),
    due_month = month(Due),
    due_year  = year(Due),
    is_leap   = leap_year(due_year)
  ) |>
  filter(due_month == 2 & due_day == 29) |>
  select(Due, is_leap)
```

```
# A tibble: 4 x 2
  Due          is_leap
<date>      <lgl>
1 2012-02-29 TRUE
2 2012-02-29 TRUE
3 2012-02-29 TRUE
4 2012-02-29 TRUE
```

10.6.8.3 Invoices Distribution by Quarter

Let us count the invoices due for each quarter.

```
transact |>
  mutate(
    quarter_due = quarter(Due)
  ) |>
  count(quarter_due)
```

```
# A tibble: 4 x 2
  quarter_due     n
    <int> <int>
1         1   521
2         2   661
3         3   618
4         4   666
```

10.6.9 Your Turn

Using the `transact` data, tabulate the following from the `Due` column:

- year
- month with label
- weekday with label
- hour
- and quarter

10.7 Create, Update & Verify

In the second section, we learnt to create date-time objects using `as.Date()`, `as.POSIXct()` etc. In this section, we will explore a few other functions that will allow us to do the same

- `make_date()`
- `make_datetime()`

10.7.1 Create

To create date without time components, use `make_date()` and specify the following:

- year
- month
- date

We need to specify all the components in numbers i.e. we cannot use `Jul` or `July` for the month. It has to be `7`.

```
make_date(year = 2019,
          month = 12,
          day   = 12)
```

```
[1] "2019-12-12"
```

When you need to include time components, use `make_datetime()`.

```
make_datetime(year = 2019,
              month = 12,
              day   = 12,
              hour  = 08,
              min   = 05,
              sec   = 03,
              tz    = "UTC")
```

```
[1] "2019-12-12 08:05:03 UTC"
```

10.7.2 Update

Let us look at another scenario. You have a date-time object and want to change one of its components i.e. any of the following

- year
- month
- date

Instead of creating another date-time object, you can change any of the components using `update()`. In the below example, we will start with the date of release of R version 3.6.1 and using `update()`, we will change it to 2019-12-12.

```
prev_release <- ymd("2019-07-05")
prev_release |>
  update(year = 2019,
         month = 12,
         mday  = 12)
```

```
[1] "2019-12-12"
```

10.7.3 Date Sequence

So far we have created a single date-time instance. How about creating a sequence of dates? We can do that using `seq.Date()`. We need to specify the from date as the bare minimum input. If the end date is not specified, it will create the sequence until the current date.

The interval of the sequence can be specified in any of the following units:

- day
- week
- month

- quarter
- year

We can add the following to the interval units

- integer
- + / - (increment or decrement)

Using the integer, we can specify multiples of the units mentioned and using the sign, we can specify whether to increment or decrement.

The below table displays the main arguments used in `seq.Date()`:

Function	Description
'from'	Starting date of the sequence
'by'	End date of the sequence
'to'	Date increment of the sequence
'length.out'	Length of the sequence
'along.with'	Use length of this value as length of sequence

In the first example, we will create a sequence of dates from 2010-01-01 to 2019-12-31. The unit of increment should be a year i.e. the difference between the dates in the sequence should be 1 year, specified using the `by` argument.

```
seq.Date(from = as.Date("2010-01-01"), to = as.Date("2019-12-31"), by = "year")
```

```
[1] "2010-01-01" "2011-01-01" "2012-01-01" "2013-01-01" "2014-01-01"
[6] "2015-01-01" "2016-01-01" "2017-01-01" "2018-01-01" "2019-01-01"
```

In the next example, we change the unit of increment to a quarter i.e. the difference between the dates in the sequence should be a quarter or 3 months.

```
seq.Date(from = as.Date("2009-12-12"), to = as.Date("2019-12-12"), by = "quarter")
```

```
[1] "2009-12-12" "2010-03-12" "2010-06-12" "2010-09-12" "2010-12-12"
[6] "2011-03-12" "2011-06-12" "2011-09-12" "2011-12-12" "2012-03-12"
[11] "2012-06-12" "2012-09-12" "2012-12-12" "2013-03-12" "2013-06-12"
[16] "2013-09-12" "2013-12-12" "2014-03-12" "2014-06-12" "2014-09-12"
[21] "2014-12-12" "2015-03-12" "2015-06-12" "2015-09-12" "2015-12-12"
[26] "2016-03-12" "2016-06-12" "2016-09-12" "2016-12-12" "2017-03-12"
[31] "2017-06-12" "2017-09-12" "2017-12-12" "2018-03-12" "2018-06-12"
[36] "2018-09-12" "2018-12-12" "2019-03-12" "2019-06-12" "2019-09-12"
[41] "2019-12-12"
```

We will now create a sequence of dates but instead of specifying the unit of increment, we specify the number of dates in the sequence i.e. the length of the sequence. We do this using the `length.out` argument which specifies the desired length of the sequence. We want the sequence to have 10 dates including the start and end date, and hence we supply the value 10 for the `length.out` argument.

```
seq.Date(from = as.Date("2010-01-01"), to = as.Date("2019-12-31"), length.out = 10)
```

```
[1] "2010-01-01" "2011-02-10" "2012-03-22" "2013-05-02" "2014-06-11"  
[6] "2015-07-22" "2016-08-31" "2017-10-10" "2018-11-20" "2019-12-31"
```

In all of the previous examples, we have specified both the start and the end date. Let us look at a few examples where we create a sequence of dates where we only specify the start date. In the below example, we want to create a sequence of dates starting from 2010-01-01. The unit of increment should be 1 year i.e. the difference between the dates in the sequence should be 1 year and the length of the sequence should be 10 i.e. the number of dates including the start date should be 10.

```
seq.Date(from = as.Date("2010-01-01"), by = "year", length.out = 10)
```

```
[1] "2010-01-01" "2011-01-01" "2012-01-01" "2013-01-01" "2014-01-01"  
[6] "2015-01-01" "2016-01-01" "2017-01-01" "2018-01-01" "2019-01-01"
```

The unit of increment can include multiples and +/- sign i.e. it can be an unit of increment or decrement. In the next example, we can increment the dates in the sequence by 2 i.e. the difference between the dates should be 2 instead of 1. This is achieved by specifying the unit of increment (multiple) first followed by a space and then the unit. In our example, it is 2 **year**. As you can see, the sequence now goes all the way till 2028 and the gap between the dates is 2 years.

```
seq.Date(from = as.Date("2010-01-01"), by = "2 year", length.out = 10)
```

```
[1] "2010-01-01" "2012-01-01" "2014-01-01" "2016-01-01" "2018-01-01"  
[6] "2020-01-01" "2022-01-01" "2024-01-01" "2026-01-01" "2028-01-01"
```

Let us say instead of increment we want to decrement the dates i.e. the sequence of dates will go backwards as shown in the next example. We achieve this by using the - sign along with the unit of decrement. The sequence of dates in next example starts from 2010 and goes back upto 1992 and the difference between the dates is 2 years.

```
seq.Date(from = as.Date("2010-01-01"), by = "-2 year", length.out = 10)
```

```
[1] "2010-01-01" "2008-01-01" "2006-01-01" "2004-01-01" "2002-01-01"
[6] "2000-01-01" "1998-01-01" "1996-01-01" "1994-01-01" "1992-01-01"
```

In the last example, we will explore the `along.with` argument. Here we have supplied a vector which is a sequence of numbers from 1 to 10. The length of this vector is 10 and the same length is used as the length of the sequence i.e. the length of value supplied to `along.with` is also the length of the sequence.

```
seq.Date(from = as.Date("2010-01-01"), by = "-2 year", along.with = 1:10)
```

```
[1] "2010-01-01" "2008-01-01" "2006-01-01" "2004-01-01" "2002-01-01"
[6] "2000-01-01" "1998-01-01" "1996-01-01" "1994-01-01" "1992-01-01"
```

10.7.4 Verify Type

How do you check if the data is a date-time object? You can do that using any of the following from the lubridate package.

- `is.Date()`
- `is.POSIXct()`
- `is.POSIXlt()`

```
is.Date(release_date)
```

```
[1] FALSE
```

```
is.POSIXct(release_date)
```

```
[1] TRUE
```

```
is.POSIXlt(release_date)
```

```
[1] FALSE
```

10.7.5 Your Turn

- R 2.0.0 was released on 2004-10-04 14:24:38. Create this date using both `make_date()` and `make_datetime()`
- R 3.0.0 was released on 2013-04-03 07:12:36. Update the date created in the previous step to the above using `update()`

Try it live

💡 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(lubridate)
library(readr)
if (!file.exists("transact_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/transact_sample.csv",
    "transact_sample.csv"
  )
}
transact <- read_csv("transact_sample.csv", show_col_types = FALSE)
transact |>
  mutate(due_year = year(Due), due_month = month(Due, label = TRUE)) |>
  head()
```

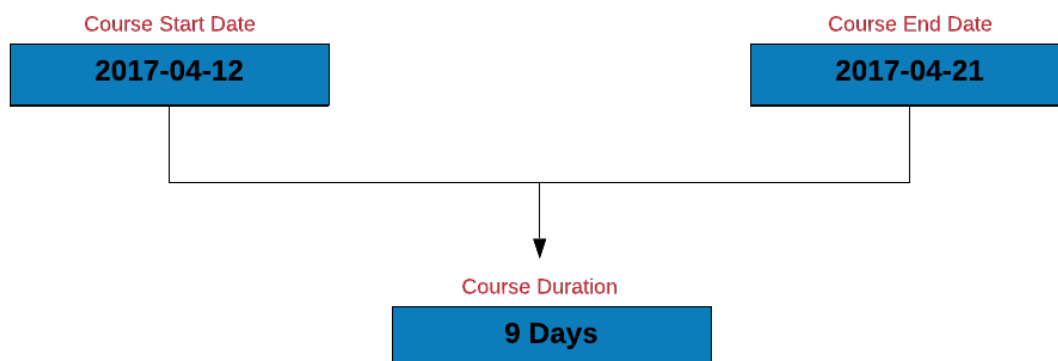
11 Date & Time: Arithmetic and Time Zones

This chapter continues `@date-and-time-in-r`. We do arithmetic with dates, compare durations, periods and intervals, handle time zones and DST, and round and roll back dates — including the receivables overdue case study.

11.1 Date Arithmetic

11.1.1 Introduction

Time to do some arithmetic with the dates. Let us calculate the length of a course you have enrolled for (Become a Rock Star Data Scientist in 10 Days) by subtracting the course start date from the course end date.

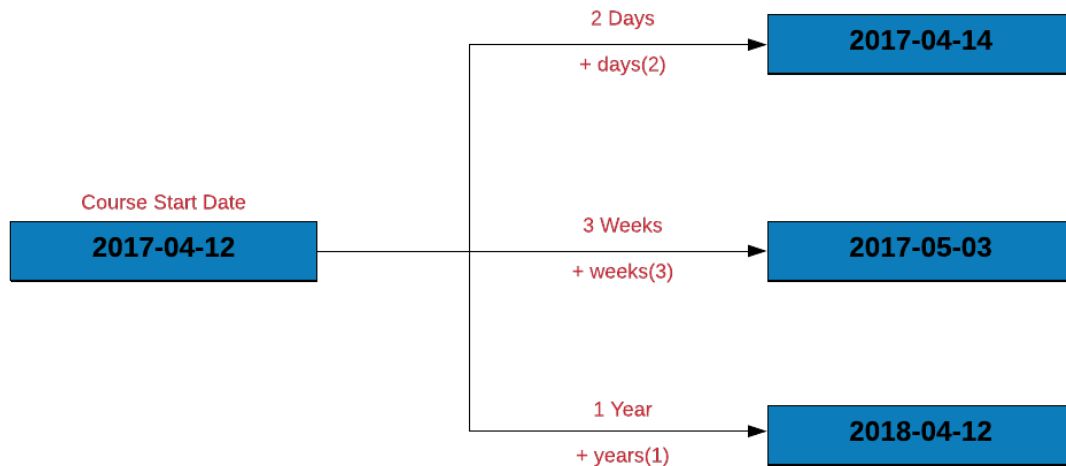


```
course_start <- as_date('2017-04-12')
course_end   <- as_date('2017-04-21')
course_duration <- course_end - course_start
course_duration
## Time difference of 9 days
```

11.1.2 Shift Date

Time to shift the course dates. We can shift a date by days, weeks or months. Let us shift the course start date by:

- 2 days
- 3 weeks
- 1 year



```
course_start + days(2)
## [1] "2017-04-14"
course_start + weeks(3)
## [1] "2017-05-03"
course_start + years(1)
## [1] "2018-04-12"
```

11.1.3 Case Study

11.1.3.1 Compute days to settle invoice

Let us estimate the number of days to settle the invoice by subtracting the date of invoice from the date of payment.

```
transact |>
  mutate(
    days_to_pay = Payment - Invoice
  )
```

```
# A tibble: 2,466 x 4
  Invoice      Due      Payment  days_to_pay
  <date>      <date>      <date>      <drtn>
1 2013-01-02 2013-02-01 2013-01-15 13 days
2 2013-01-26 2013-02-25 2013-03-03 36 days
3 2013-07-03 2013-08-02 2013-07-08  5 days
4 2013-02-10 2013-03-12 2013-03-17 35 days
5 2012-10-25 2012-11-24 2012-11-28 34 days
6 2012-01-27 2012-02-26 2012-02-22 26 days
7 2013-08-13 2013-09-12 2013-09-09 27 days
8 2012-12-16 2013-01-15 2013-01-12 27 days
9 2012-05-14 2012-06-13 2012-07-01 48 days
10 2013-07-01 2013-07-31 2013-07-26 25 days
# i 2,456 more rows
```

11.1.3.2 Compute days over due

How many of the invoices were settled post the due date? We can find this by:

- subtracting the due date from the payment date
- counting the number of rows where delay < 0

```
transact |>
  mutate(
    delay = Due - Payment
  ) |>
  filter(delay < 0) |>
  mutate(
    delay = delay * -1
  ) |>
  count(delay)
```

```
# A tibble: 36 x 2
  delay      n
  <drtn> <int>
1 1 days    61
2 2 days    65
3 3 days    51
4 4 days    62
5 5 days    69
6 6 days    56
7 7 days    55
8 8 days    49
9 9 days    38
10 10 days   33
```

```
# i 26 more rows
```

11.1.4 Your Turn

- compute the length of a vacation which begins on 2020-04-19 and ends on 2020-04-25
- recompute the length of the vacation after shifting the vacation start and end date by 10 days and 2 weeks
- compute the days to settle invoice and days overdue from the `receivables.csv` data set
- compute the length of employment (only for those employees who have been terminated) from the `hr-data.csv` data set (use date of hire and termination)

11.2 Time Zones

11.2.1 Introduction



In the previous section, `POSIXlt` stored date/time components as a list. Among the different components it returned were

- `gmtoff`
- `zone`

`gmtoff` is offset in seconds from GMT i.e. difference in hours and minutes from UTC. Wait.. What do UTC and GMT stand for?

- Coordinated Universal Time (UTC)
- Greenwich Meridian Time (GMT)

Since we are talking about UTC, GMT etc., let us spend a little time on understanding the basics of time zones and daylight savings.

11.2.2 Time Zones

Timezones exist because different parts of the Earth receive sun light at different times. If there was a single timezone, noon or morning would mean different things in different parts of the world. The timezones are based on Earth's rotation. The Earth moves ~15 degrees every 60 minutes i.e. 360 degrees in 24 hours. The planet is divided into 24 timezones each 15 degrees of longitude width.

Now, you have heard of Greenwich Meridian Time (GMT) right? We just saw GMT off set in `POSIXlt` and you would have come across it in other time formats as well. For example, India timezone is given as GMT +5:30. Let us explore GMT in a little more detail. Greenwich is a suburb of London and the time at Greenwich is **Greenwich Mean Time**. As you move West from Greenwich, every 15 degree section is one hour earlier than GMT and every 15 degree section to the East is an hour later.

Alright! What is **UTC** then? **Coordinated Universal Time (UTC)** , on the other hand, is the time standard commonly used across the world. Even though they share the same current time, **GMT** is a **timezone** while **UTC** is a **time standard**.

So how do we check the timezone in R? When you run `Sys.timezone()`, you should be able to see the timezone you are in.

```
Sys.timezone()
```

```
[1] "UTC"
```

If you do not see the timezone, use `Sys.getenv()` to get the value of the **TZ** environment variable.

```
Sys.getenv("TZ")
```

```
[1] "UTC"
```

If nothing is returned, it means we have to set the timezone. Use `Sys.setenv()` to set the timezone as shown below. The author resides in India and hence the timezone is set to **Asia/Calcutta**. You need to set the timezone in which you reside or work.

```
Sys.setenv(TZ = "Asia/Calcutta")
```

Another way to get the timezone is through `tz()` from the `lubridate` package.

```
lubridate::tz(Sys.time())
```

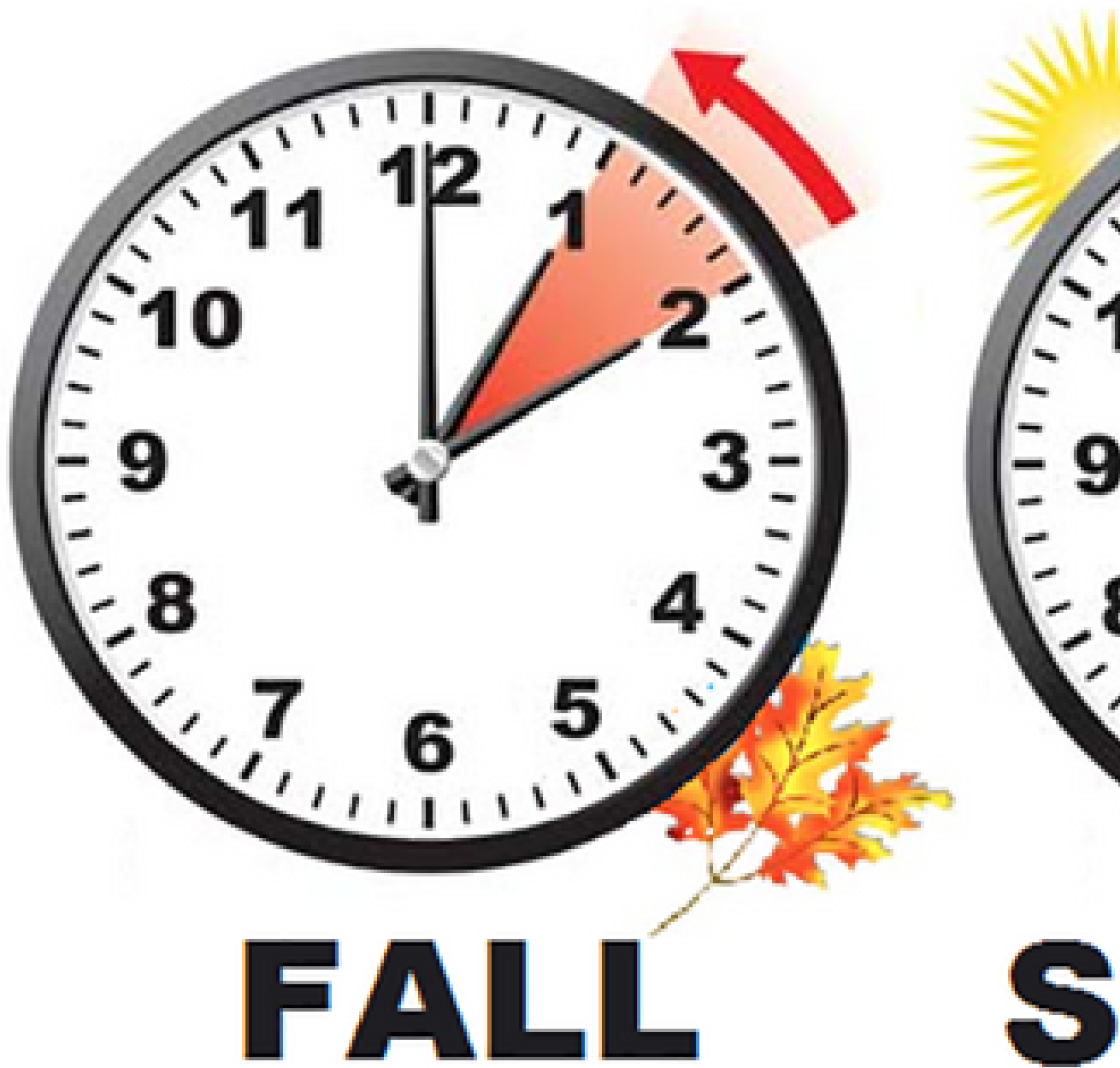
```
[1] ""
```

If you want to view the time in a different timezone, use `with_tz()`. Let us look at the current time in **UTC** instead of **Indian Standard Time**.

```
lubridate::with_tz(Sys.time(), "UTC")
```

```
[1] "2026-10-02 08:23:44 UTC"
```

11.2.3 Daylight Savings



Daylight savings also known as

- daylight saving time

- daylight savings time
- daylight time
- summer time

is the practice of advancing clocks during summer months so that darkness falls later each day according to the clock. In other words

- advance clock by one hour in spring (spring forward)
- retard clocks by one hour in autumn (fall back)

In R, the `dst()` function is an indicator for daylight savings. It returns `TRUE` if daylight saving is in force, `FALSE` if not and `NA` if unknown.

```
dst(Sys.Date())
```

```
[1] FALSE
```

11.2.4 Your Turn

- check the timezone you live in
- check if daylight savings in on
- check the current time in **UTC** or a different time zone

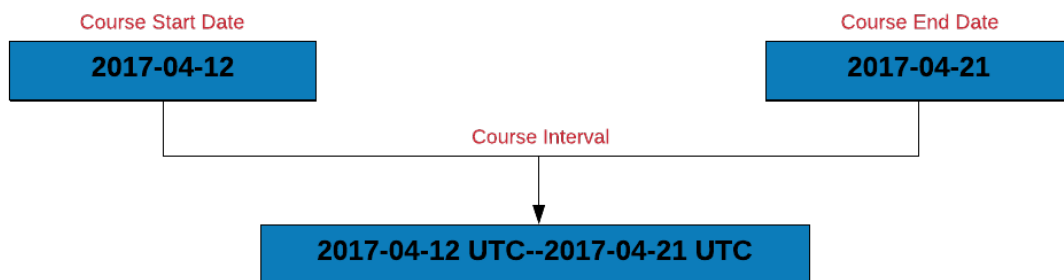
11.3 Intervals, Duration & Period

In this chapter, we will learn about

- intervals
- duration
- and period

11.3.1 Interval

An interval is a timespan defined by two date-times. Let us represent the length of the course using `interval`.



```
course_start <- as_date('2017-04-12')
course_end   <- as_date('2017-04-21')
interval(course_start, course_end)
```

```
[1] 2017-04-12 UTC--2017-04-21 UTC
```

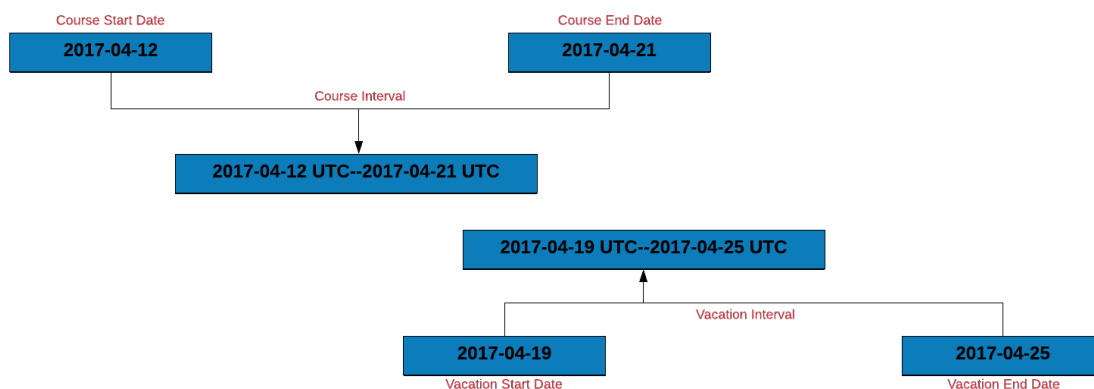
If you observe carefully, the interval is represented by the course start and end dates. We will learn how to use intervals in the case study.

11.3.1.1 Overlapping Intervals

Let us say you are planning a vacation and want to check if the vacation dates overlap with the course dates. You can do this by:

- creating vacation and course intervals
- use `int_overlaps()` to check if two intervals overlap. It returns `TRUE` if the intervals overlap else `FALSE`.

Let us use the vacation start and end dates to create `vacation_interval` and then check if it overlaps with `course_interval`.



```

vacation_start    <- as_date('2017-04-19')
vacation_end      <- as_date('2017-04-25')
course_interval   <- interval(course_start, course_end)
vacation_interval <- interval(vacation_start, vacation_end)
int_overlaps(course_interval, vacation_interval)
## [1] TRUE

```

11.3.1.2 How many invoices were settled within due date?

Let us use intervals to count the number of invoices that were settled within the due date. To do this, we will:

- create an interval for the invoice and due date
- create a new column `due_next` by incrementing the due date by 1 day
- another interval for `due_next` and the payment date
- if the intervals overlap, the payment was made within the due date

```

transact |>
  mutate(
    inv_due_interval = interval(Invoice, Due),
    due_next         = Due + days(1),
    due_pay_interval = interval(due_next, Payment),
    overlaps         = int_overlaps(inv_due_interval, due_pay_interval)
  ) |>
  select(Invoice, Due, Payment, overlaps)

```

```

# A tibble: 2,466 x 4
  Invoice    Due      Payment overlaps
  <date>    <date>    <date>    <lgl>
1 2013-01-02 2013-02-01 2013-01-15 TRUE
2 2013-01-26 2013-02-25 2013-03-03 FALSE
3 2013-07-03 2013-08-02 2013-07-08 TRUE
4 2013-02-10 2013-03-12 2013-03-17 FALSE
5 2012-10-25 2012-11-24 2012-11-28 FALSE
6 2012-01-27 2012-02-26 2012-02-22 TRUE
7 2013-08-13 2013-09-12 2013-09-09 TRUE
8 2012-12-16 2013-01-15 2013-01-12 TRUE
9 2012-05-14 2012-06-13 2012-07-01 FALSE
10 2013-07-01 2013-07-31 2013-07-26 TRUE
# i 2,456 more rows

```

Below we show another method to count the number of invoices paid within the due date. Instead of using `days` to change the due date, we use `int_shift` to shift it by 1 day.

```

transact |>
  mutate(
    inv_due_interval = interval(Invoice, Due),
    due_pay_interval = interval(Due, Payment),
    due_pay_next      = int_shift(due_pay_interval, by = days(1)),
    overlaps          = int_overlaps(inv_due_interval, due_pay_next)
  ) |>
  select(Invoice, Due, Payment, overlaps)

```

```

# A tibble: 2,466 x 4
  Invoice    Due      Payment overlaps
  <date>    <date>    <date>    <lgl>
1 2013-01-02 2013-02-01 2013-01-15 TRUE
2 2013-01-26 2013-02-25 2013-03-03 FALSE
3 2013-07-03 2013-08-02 2013-07-08 TRUE
4 2013-02-10 2013-03-12 2013-03-17 FALSE
5 2012-10-25 2012-11-24 2012-11-28 FALSE
6 2012-01-27 2012-02-26 2012-02-22 TRUE
7 2013-08-13 2013-09-12 2013-09-09 TRUE
8 2012-12-16 2013-01-15 2013-01-12 TRUE
9 2012-05-14 2012-06-13 2012-07-01 FALSE
10 2013-07-01 2013-07-31 2013-07-26 TRUE
# i 2,456 more rows

```

You might be thinking why we incremented the due date by a day before creating the interval between the due day and the payment day. If we do not increment, both the intervals will share a common date i.e. the due date and they will always overlap as shown below:

```

transact |>
  mutate(
    inv_due_interval = interval(Invoice, Due),
    due_pay_interval = interval(Due, Payment),
    overlaps         = int_overlaps(inv_due_interval, due_pay_interval)
  ) |>
  select(Invoice, Due, Payment, overlaps)

```

```

# A tibble: 2,466 x 4
  Invoice    Due      Payment overlaps
  <date>    <date>    <date>    <lgl>
1 2013-01-02 2013-02-01 2013-01-15 TRUE
2 2013-01-26 2013-02-25 2013-03-03 TRUE
3 2013-07-03 2013-08-02 2013-07-08 TRUE
4 2013-02-10 2013-03-12 2013-03-17 TRUE
5 2012-10-25 2012-11-24 2012-11-28 TRUE

```

```

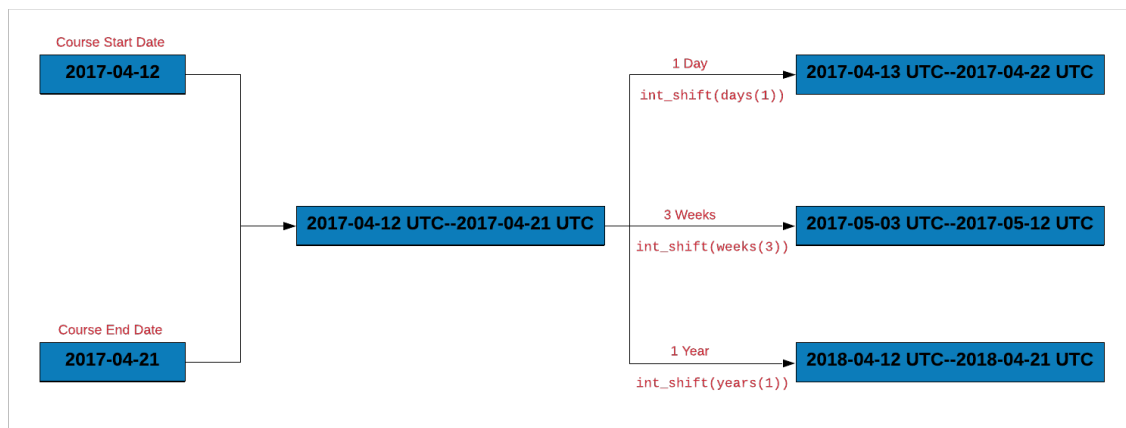
6 2012-01-27 2012-02-26 2012-02-22 TRUE
7 2013-08-13 2013-09-12 2013-09-09 TRUE
8 2012-12-16 2013-01-15 2013-01-12 TRUE
9 2012-05-14 2012-06-13 2012-07-01 TRUE
10 2013-07-01 2013-07-31 2013-07-26 TRUE
# i 2,456 more rows

```

11.3.1.3 Shift Interval

Intervals can be shifted too. In the below example, we shift the course interval by:

- 1 day
- 3 weeks
- 1 year



```

course_interval <- interval(course_start, course_end)

# shift course_interval by 1 day
int_shift(course_interval, by = days(1))
## [1] 2017-04-13 UTC--2017-04-22 UTC

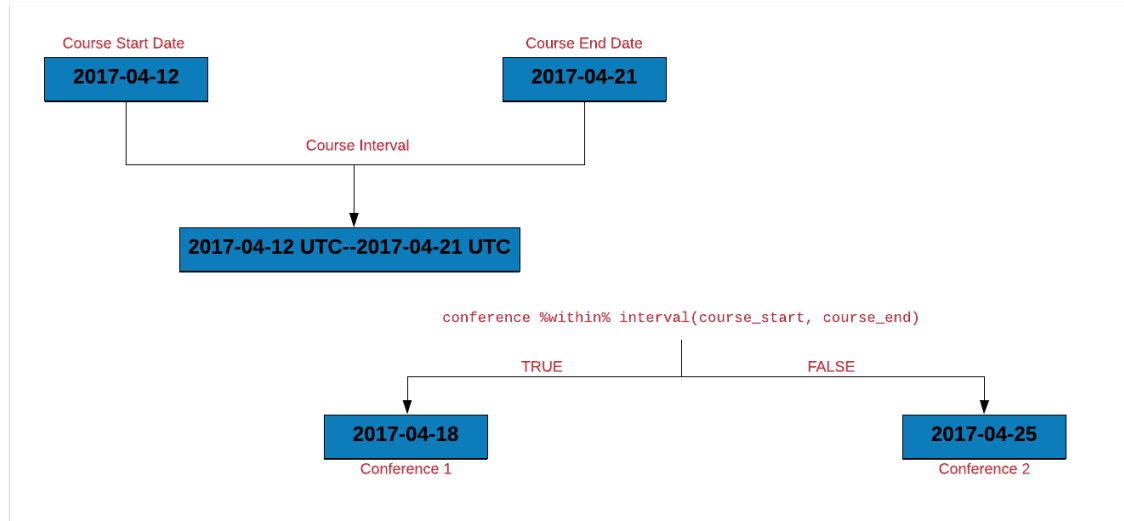
# shift course_interval by 3 weeks
int_shift(course_interval, by = weeks(3))
## [1] 2017-05-03 UTC--2017-05-12 UTC

# shift course_interval by 1 year
int_shift(course_interval, by = years(1))
## [1] 2018-04-12 UTC--2018-04-21 UTC

```

11.3.2 Within

Let us assume that we have to attend a conference in April 2017. Does it occur during the course duration? We can answer this using `%within%` which will return `TRUE` if a date falls within an interval.



```
conference <- as_date('2017-04-15')
conference %within% course_interval
## [1] TRUE
```

11.3.2.1 How many invoices were settled within due date?

Let us use `%within%` to count the number of invoices that were settled within the due date. We will do this by:

- creating an interval for the invoice and due date
- check if the payment date falls within the above interval

```
transact |>
  mutate(
    inv_due_interval = interval(Invoice, Due),
    overlaps         = Payment %within% inv_due_interval
  ) |>
  select(Due, Payment, overlaps)
```

```
# A tibble: 2,466 x 3
  Due      Payment overlaps
<date>    <date>    <lgl>
1 2017-04-12 2017-04-12 TRUE
2 2017-04-12 2017-04-12 TRUE
3 2017-04-12 2017-04-12 TRUE
4 2017-04-12 2017-04-12 TRUE
5 2017-04-12 2017-04-12 TRUE
6 2017-04-12 2017-04-12 TRUE
7 2017-04-12 2017-04-12 TRUE
8 2017-04-12 2017-04-12 TRUE
9 2017-04-12 2017-04-12 TRUE
10 2017-04-12 2017-04-12 TRUE
```

```

1 2013-02-01 2013-01-15 TRUE
2 2013-02-25 2013-03-03 FALSE
3 2013-08-02 2013-07-08 TRUE
4 2013-03-12 2013-03-17 FALSE
5 2012-11-24 2012-11-28 FALSE
6 2012-02-26 2012-02-22 TRUE
7 2013-09-12 2013-09-09 TRUE
8 2013-01-15 2013-01-12 TRUE
9 2012-06-13 2012-07-01 FALSE
10 2013-07-31 2013-07-26 TRUE
# i 2,456 more rows

```

11.3.3 Duration

Duration is timespan measured in seconds. To create a duration object, use `duration()`. The timespan can be anything from seconds to years but it will be represented as seconds. Let us begin by creating a duration object where the timespan is in seconds.

```
duration(50, "seconds")
```

```
[1] "50s"
```

Another way to specify the above timespan is shown below:

```
duration(second = 50)
```

```
[1] "50s"
```

As you can see, the output is same in both the cases. Let us increase the timespan to 60 seconds and see what happens.

```
duration(second = 60)
```

```
[1] "60s (~1 minutes)"
```

Although the timespan is primarily measured in seconds, it also shows `~1 minutes` in the brackets. As the length of the timespan increases i.e. the number becomes large, it is represented using larger units such as hours and days. In the below examples, as the number of seconds increases, you can observe larger units being used to represent the timespan.

```
# minutes
duration(minute = 50)
```

```
[1] "3000s (~50 minutes)"
```

```
duration(minute = 60)
```

```
[1] "3600s (~1 hours)"
```

```
# hours
duration(hour = 23)
```

```
[1] "82800s (~23 hours)"
```

```
duration(hour = 24)
```

```
[1] "86400s (~1 days)"
```

The following helper functions can be used to create duration objects as well.

```
# default
dseconds()
```

```
[1] "1s"
```

```
dminutes()
```

```
[1] "60s (~1 minutes)"
```

```
# seconds
duration(second = 59)
```

```
[1] "59s"
```

```
dseconds(59)
```

```
[1] "59s"
```

```
# minutes
duration(minute = 50)
```

```
[1] "3000s (~50 minutes)"
```

```
dminutes(50)
```

```
[1] "3000s (~50 minutes)"
```

```
# hours
duration(hour = 36)
```

```
[1] "129600s (~1.5 days)"
```

```
dhours(36)
```

```
[1] "129600s (~1.5 days)"
```

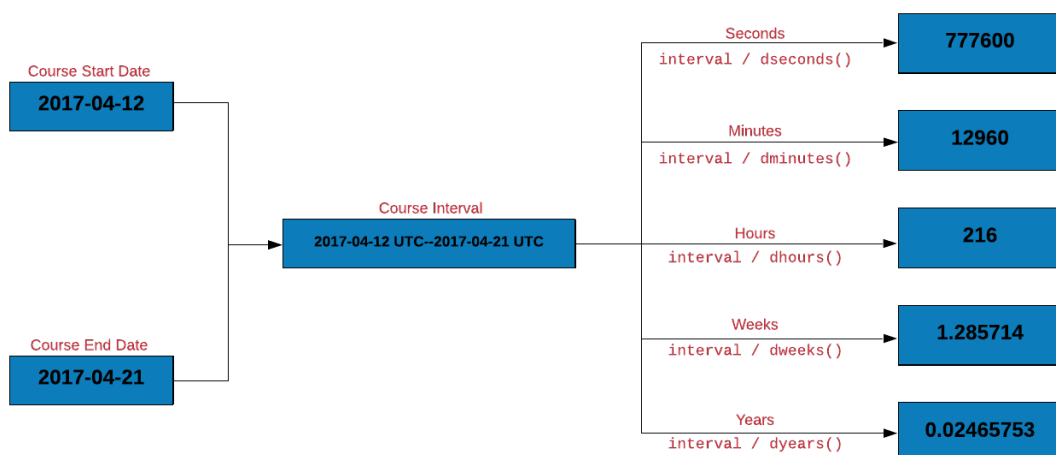
```
# weeks
duration(week = 56)
```

```
[1] "33868800s (~1.07 years)"
```

```
dweeks(56)
```

```
[1] "33868800s (~1.07 years)"
```

Let us use the above helper functions to get the course length in different units.



```
# course length in seconds
course_interval / dseconds()
## [1] 777600

# course length in minutes
course_interval / dminutes()
## [1] 12960

# course length in hours
course_interval / dhours()
## [1] 216

# course length in weeks
course_interval / dweeks()
## [1] 1.285714

# course length in years
course_interval / dyears()
## [1] 0.02464066
```

11.3.4 Period

A **period** is a timespan defined in units such as years, months, and days. In the below examples, we use `period()` to represent timespan using different units.

```
# second
period(5, "second")
```

```
[1] "5S"
```

```
period(second = 5)
```

```
[1] "5S"
```

```
# minute & second
period(c(3, 5), c("minute", "second"))
```

```
[1] "3M 5S"
```

```
period(minute = 3, second = 5)
```

```
[1] "3M 5S"
```

```
# hour, minute & second
period(c(1, 3, 5), c("hour", "minute", "second"))
```

```
[1] "1H 3M 5S"
```

```
period(hour = 1, minute = 3, second = 5)
```

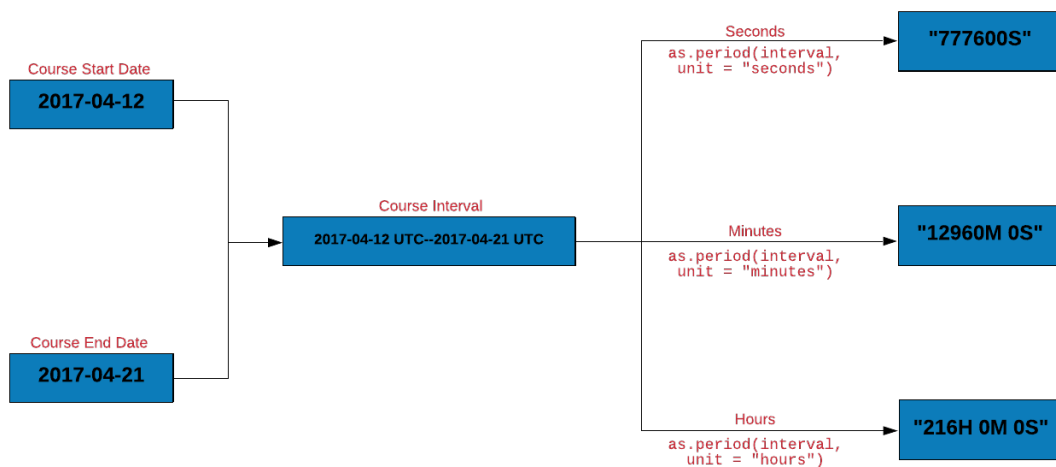
```
[1] "1H 3M 5S"
```

```
# day, hour, minute & second
period(c(3, 1, 3, 5), c("day", "hour", "minute", "second"))
```

```
[1] "3d 1H 3M 5S"
```

```
period(day = 3, hour = 1, minute = 3, second = 5)
```

```
[1] "3d 1H 3M 5S"
```



Let us get the course length in different units using `as.period()`.

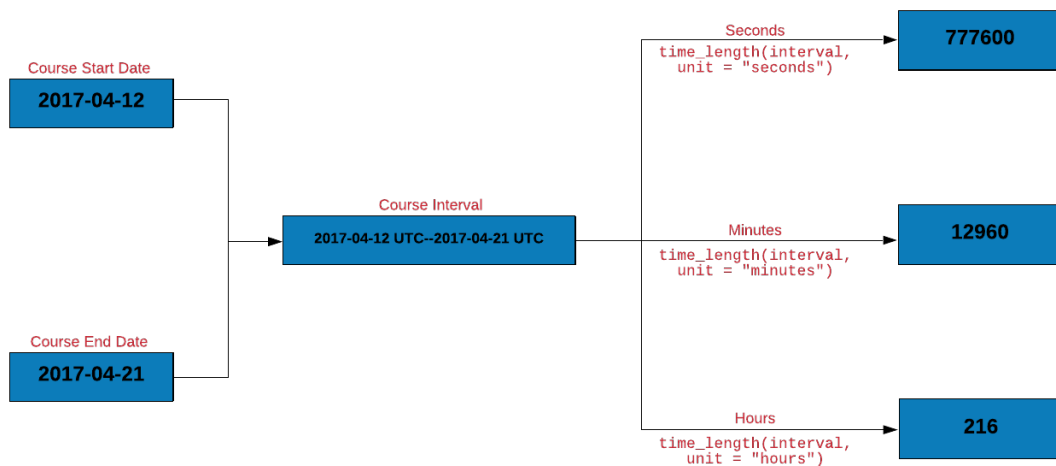
```
# course length in second
as.period(course_interval, unit = "seconds")
## [1] "777600S"

# course length in hours and minutes
```

```
as.period(course_interval, unit = "minutes")
## [1] "12960M 0S"

# course length in hours, minutes and seconds
as.period(course_interval, unit = "hours")
## [1] "216H 0M 0S"
```

`time_length()` computes the exact length of a timespan i.e. duration, interval or period. Let us use `time_length()` to compute the length of the course in different units.



```
# course length in seconds
time_length(course_interval, unit = "seconds")
## [1] 777600

# course length in minutes
time_length(course_interval, unit = "minutes")
## [1] 12960

# course length in hours
time_length(course_interval, unit = "hours")
## [1] 216
```

11.4 Others

In this section, we will learn to round date/time to the nearest unit and roll back dates.

11.4.1 Rounding Dates

We will explore functions for rounding dates

- to the nearest value using `round_dates()`
- down using `floor_date()`
- up using `ceiling_date()`

The unit for rounding can be any of the following:

- second
- minute
- hour
- day
- week
- month
- bimonth
- quarter
- season
- halfyear
- and year

We will look at a few examples using `round_date()` and you will then practice using the other two functions.

```
# minute
round_date(release_date, unit = "minute")
```

```
[1] "2019-12-12 08:05:00 UTC"
```

```
round_date(release_date, unit = "mins")
```

```
[1] "2019-12-12 08:05:00 UTC"
```

```
round_date(release_date, unit = "5 mins")
```

```
[1] "2019-12-12 08:05:00 UTC"
```

```
# hour
round_date(release_date, unit = "hour")
```

```
[1] "2019-12-12 08:00:00 UTC"
```

```
# day  
round_date(release_date, unit = "day")
```

```
[1] "2019-12-12 UTC"
```

11.4.2 Rollback

Use `rollback()` if you want to change the date to the last day of the previous month or the first day of the month.

```
rollback(release_date)
```

```
[1] "2019-11-30 08:05:03 UTC"
```

To change the date to the first day of the month, use the `roll_to_first` argument and set it to `TRUE`.

```
rollback(release_date, roll_to_first = TRUE)
```

```
[1] "2019-12-01 08:05:03 UTC"
```

11.4.3 Your Turn

- round up the `transact` due dates to weeks
- round down the `transact` due dates to days
- rollback the `transact` due dates to the beginning of the month

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(lubridate)
library(readr)
if (!file.exists("transact_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/transact_sample.csv",
    "transact_sample.csv"
  )
}
transact <- read_csv("transact_sample.csv", show_col_types = FALSE)
transact |>
  mutate(days_to_pay = Payment - Invoice) |>
  select(Invoice, Due, Payment, days_to_pay) |>
  head()
```

12 Categorical Data Fundamentals

i Continued in part B

This chapter was split for readability. Reordering, recoding, lumping and visualization continue in @forcats-manipulation.

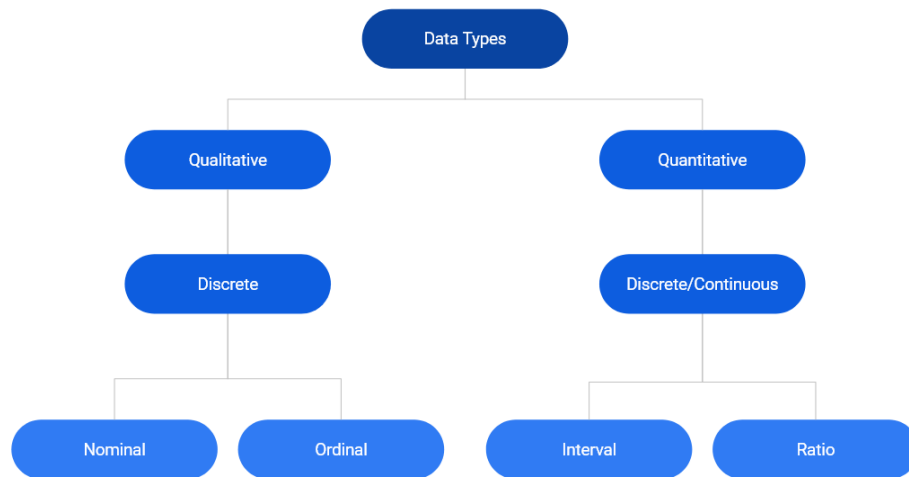
This chapter is the first of two on categorical data. Here we cover what factors are, how R stores them, frequency tabulation and base R pitfalls. @forcats-manipulation continues with reordering, recoding, lumping and ggplot2 visualization.

12.1 Introduction

Handling categorical/qualitative data is integral to data analysis. Almost every data science project involves working with categorical data and students should know how to store, summarize, visualize & manipulate such data. Working with categorical data is different from working with numbers or text. In this chapter, we will understand categorical data and explore the rich set of functions (built-in & through packages) provided by R for working with such data. The word categorical is used interchangeably with qualitative.

12.1.1 Data Types

Before we begin our deep dive on categorical data, let us get a quick overview of different data types.



In the chart above, we can see that data can be primarily classified into **qualitative** or **quantitative**. Qualitative data consists of labels or names. Quantitative data, on the other hand, consists of numbers and indicate how much or how many. This brings us to the next level of classification:

- discrete
- continuous

In the chart, we can observe that qualitative data is always **discrete** where as quantitative data may be **discrete** or **continuous**. Qualitative data is further classified into

- nominal
- ordinal

First, we will understand discrete and continuous data, and then proceed to explore nominal and ordinal data.

Discrete Data



Discrete data arises in situations where counting is involved. It can take on only a finite number of values and cannot be divided into smaller parts. For example, let us consider the number of students in a class. We can have 5 or 10 students but not 5.5 students (we can't have half a student).

Continuous Data



Continuous data arises in situations where measuring is involved. It can take any numeric value in a specified range and can be divided into smaller parts and still have meaning. Examples include money, temperature, length, volume etc.

12.1.2 Categorical Data

Since our interest is in categorical data, we will spend more time understanding the different types of categorical data through various examples. Let us begin by formally defining categorical data:

- it is always discrete
- it may be divided into groups
- consists of names or labels
- takes on limited & fixed number of possible values
- arises in situation when counting is involved
- analysis generally involves the use of data tables

12.1.3 Dichotomous

A categorical variable that can take on exactly two values is termed as **binary** or **dichotomous** variable.



yes



no

12.1.4 Polychotomous

Categorical variables with more than two possible values are called **polychotomous** variables.



yes



maybe



no

12.1.5 Ordinal



In ordinal data, the categories can be ordered or ranked. Examples include

- socio-economic status
- education level
- income level
- satisfaction rating

While we can rank the categories, we cannot assign a value to them. For example, in satisfaction ranking, we cannot say that like is twice as positive as dislike i.e. we are unable to say how much they differ from each other. While the order or rank of data is meaningful, the difference between two pieces of data cannot be measured/determined or are meaningless. Ordinal data provide information about relative comparisons, but not the magnitude of the differences.

12.1.6 Nominal



Nominal data do not have an intrinsic order and cannot be ordered or measured. Examples include

- blood group
- gender
- religion
- color

Categorical data are sometimes coded with numbers, with those numbers replacing names. Although such numbers might appear to be quantitative, they are actually categorical data. When they do take numerical values, those numbers do not have any mathematical meaning. Examples include months expressed in numbers.

12.1.7 Summary

- Data can be qualitative or quantitative.
- Qualitative data is always discrete.
- Dichotomous data consists of only 2 groups/levels.
- Polychotomous data consists of more than 2 groups/levels.
- Nominal data do not have an intrinsic order.
- In ordinal data
 - categories can be ordered or ranked
 - the difference between the categories cannot be determined

12.1.8 Your Turn...

1. State whether the following are quantitative or qualitative

- Age
- Gender
- Annual Sales
- Weight
- Education Level
- Vehicle Type
- Height
- Ice Cream Flavor
- Job/Work Experience
- Blood Group
- Color
- Mode of Payment

2. State whether the following are dichotomous or polychotomous

- Heads / Tails
- Blood Group

- Transportation Mode
- Rich / Poor
- Democrat / Republican
- Education Level
- Gender
- Pass / Fail
- Yes / No
- Positive / Negative

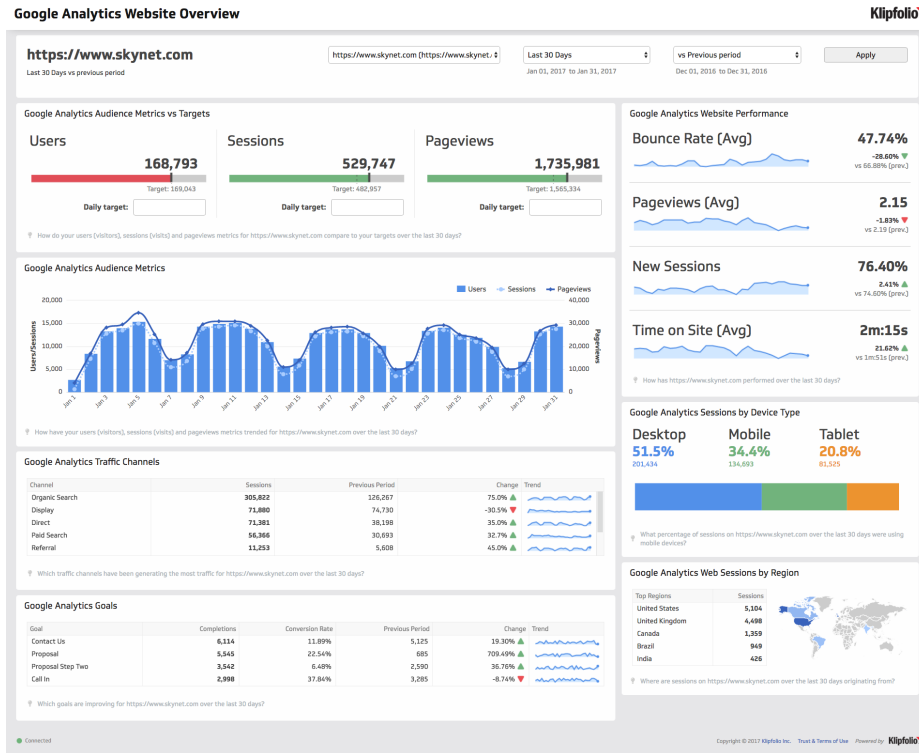
3. State whether the following are nominal or ordinal

- STD Codes
- Prepaid / Postpaid
- Hotel Ratings
- Smart Phone Brands
- Student Grades
- Cellular Operator
- ISPs (Internet Service Providers)
- Occupation
- Bank Account Number
- Class of Travel
- Military Rank

12.2 Case Study

As is the practice, throughout this chapter, we will work on a case study related to an e-commerce firm. As most of you would already be aware, a lot of data is captured when you go on the internet by the websites you browse as well as by third party cookies. Data collected is then used to display ads as well as to feed to recommendation algorithms.

The data used in the case study represents the basic information that is captured when users visit any website. It closely resembles real world data for an e-commerce store. We will try to generate insights about the visitors to be used by an imaginary marketing team for better targeting and promotion. The case study data set can be imported using the RStudio IDE or R code.



12.2.1 Data

The data set is available in both **CSV** & **RDS** formats.

CSV

If you want to specify the data types while reading the data, use the `readr` package. We have explored how to import data into R in a previous [chapter](#). We will read a subset of columns from the data set (it has 20 columns) which will cover both nominal and ordinal data types. To import the data, we will use the `read_csv()` function. The first input is the name of the data set, `analytics.csv`. Ensure that the name is enclosed in single/double quotes.

```
read_csv("analytics_raw.csv",
  col_types = cols_only(device = col_factor(levels = c("Desktop", "Tablet", "Mobile"),
    gender = col_factor(levels = c("female", "male", "NA")),
    user_rating = col_factor(levels = c("1", "2", "3", "4", "5"),
      ordered = TRUE)))
```

```
# A tibble: 244,398 x 3
  device gender user_rating
<fct>   <fct>   <ord>
1 Desktop female 4
2 Mobile  NA    5
```

```

3 Desktop NA      4
4 Desktop NA      5
5 Desktop NA      4
6 Mobile  NA      4
7 Desktop NA      4
8 Desktop NA      4
9 Desktop female  5
10 Desktop NA     4
# i 244,388 more rows

```

Since we are specifying the column data types while importing the data, we will use the `col_types` argument to list out the data types. As we are reading in a subset of the columns and not all of them, we will use the `cols_only()` function indicating that only the columns specified must be read in and not all of them.

Categorical data and the levels/groups are specified using the `col_factor()` function. Use the `levels` argument to specify the levels/groups and the `ordered` argument to indicate if the data is ordinal. By default, it is set to `FALSE`, change this to `TRUE` if the column is ordinal.

RDS

The `.rds` file can be read using `readRDS()`.

```

data <- readRDS('analytics.rds')
head(data)

```

```

# A tibble: 6 x 19
  device os      browser user_type channel gender frequency recency page_depth
  <fct>  <fct>   <fct>  <fct>    <fct>  <fct>    <dbl>  <dbl>    <dbl>
1 Desktop Windows Chrome  New Visi~ Organi~ female      1      0      1
2 Mobile  iOS    Safari  Returnin~ Organi~ <NA>        3      1      1
3 Desktop Chrome ~ Chrome  New Visi~ Direct  <NA>        1      0      5
4 Desktop Macinto~ Chrome  Returnin~ Organi~ <NA>        2      0      1
5 Desktop Macinto~ Chrome  Returnin~ Referr~ <NA>        5      8      1
6 Mobile  Android Chrome  New Visi~ Organi~ <NA>        1      0      5
# i 10 more variables: hour_of_day <chr>, age <dbl>, duration <dbl>,
#   landing_page <fct>, exit_page <fct>, country <fct>, quantity <dbl>,
#   revenue <dbl>, purchase_flag <lgl>, user_rating <dbl>

```

12.2.2 Data Dictionary

Column	Description
device	Device used to browse the website

os	Operating system of the device
browser	Browser used to visit the website
user__type	New or returning visitor
channel	Source of traffic
gender	Gender of the visitor
frequency	Count of visits to the website
recency	Number of days since last visit
page__depth	Number of website pages browsed
hour__of__day	Hour of day
age	Age of the visitor
duration	Time spent on the website (in seconds)
landing__page	Page on which visitor landed
exit__page	Page on which visitor exited
country	Country of origin
city	City of the visitor
quantity	Number of units purchased
revenue	Total revenue
purchase__flag	Whether the visitor checked out?
user__rating	Website UI rating given by visitor

12.3 Factors

In this very important section, we will learn how R

- stores categorical data
- checks if given data is categorical
- converts other data types to factor
- handles missing values in categorical data
- specifies the orders of the categories/levels
- stores ordinal data

12.3.1 Introduction

In R, categorical data is stored as **factor**. Before we explore the **factor** family of functions, let us generate the sample data we will use in this module. We will generate the **device** column from the case study data set using the **sample()** function. We provide the following inputs to generate the data:

- values from which the data must be generated
- the size of the sample
- indicate if the values must be repeated (TRUE/FALSE)

```
device <- sample(c("Desktop", "Mobile", "Tablet"), size = 25, replace = TRUE)
device
```

```
[1] "Mobile" "Desktop" "Desktop" "Desktop" "Mobile" "Desktop" "Tablet"
[8] "Tablet" "Tablet" "Desktop" "Mobile" "Desktop" "Tablet" "Tablet"
[15] "Desktop" "Mobile" "Tablet" "Desktop" "Mobile" "Mobile" "Tablet"
[22] "Tablet" "Desktop" "Mobile" "Desktop"
```

12.3.2 Membership Testing

Great! We have successfully generated the sample data and along the way learnt a new R function for sampling. First, let us check if the sample is a **factor** using the membership function `is.factor()`.

```
is.factor(device)
```

```
[1] FALSE
```

Membership testing functions always have the prefix `is_` and return only logical values. If the object is a member of the specified class, they return **TRUE** else **FALSE**. Since our sample data is not stored as a factor, R has returned **FALSE**.

12.3.3 Coercion

Let us try to coerce it into **factor** using the coercion function `as.factor()`.

```
as.factor(device)
```

```
[1] Mobile Desktop Desktop Desktop Mobile Desktop Tablet Tablet Tablet
[10] Desktop Mobile Desktop Tablet Tablet Desktop Mobile Tablet Desktop
[19] Mobile Mobile Tablet Tablet Desktop Mobile Desktop
Levels: Desktop Mobile Tablet
```

Do you spot any difference in the output? In the last line, it displays the levels or categories of the variable. Don't worry if you didn't spot it. We are just getting started and you will pick it up by the end of this section. Another function that can be used to coerce data into factor is `as_factor()` from the [forcats](#) package.

```
as_factor(device)
```

```
[1] Mobile Desktop Desktop Desktop Mobile Desktop Tablet Tablet Tablet
[10] Desktop Mobile Desktop Tablet Tablet Desktop Mobile Tablet Desktop
[19] Mobile Mobile Tablet Tablet Desktop Mobile Desktop
Levels: Mobile Desktop Tablet
```

Did you notice any difference between these two functions? Focus on the last line of the output where the levels are displayed. Now observe the order of the levels. `as.factor()` displays levels in the alphabetical order whereas `as_factor()` displays them in order of appearance in the data. **Mobile**, followed by **Tablet**, and then **Desktop**. If you look at the data, they appear in the same order.

12.3.4 Factor Function

If you want finer control while creating factors, use the `factor()` function. `as.factor()` should suffice in most cases but use `factor()` when you want to:

- specify levels
- modify labels
- include NA as a level/category
- create ordered factors
- specify order of levels

The first input is a vector, usually a numeric or character vector with a small number of unique values. In our example, it is a character vector of length 25 (i.e. 25 values) but 3 unique values.

```
factor(device)
```

```
[1] Mobile Desktop Desktop Desktop Mobile Desktop Tablet Tablet Tablet
[10] Desktop Mobile Desktop Tablet Tablet Desktop Mobile Tablet Desktop
[19] Mobile Mobile Tablet Tablet Desktop Mobile Desktop
Levels: Desktop Mobile Tablet
```

If you want to specify the levels or categories, use the `levels` argument.

```
factor(device, levels = c("Desktop", "Mobile", "Tablet"))
```

```
[1] Mobile Desktop Desktop Desktop Mobile Desktop Tablet Tablet Tablet
[10] Desktop Mobile Desktop Tablet Tablet Desktop Mobile Tablet Desktop
[19] Mobile Mobile Tablet Tablet Desktop Mobile Desktop
Levels: Desktop Mobile Tablet
```

Levels not specified will be replaced by NA. Let us specify only **Desktop** and **Mobile** as the levels in the device column and see what happens.

```
factor(device, levels = c("Desktop", "Mobile"))
```

```
[1] Mobile Desktop Desktop Desktop Mobile Desktop <NA> <NA> <NA>
[10] Desktop Mobile Desktop <NA> <NA> Desktop Mobile <NA> Desktop
[19] Mobile Mobile <NA> <NA> Desktop Mobile Desktop
Levels: Desktop Mobile
```

As you can see, **Tablet** has been replaced by NA.

12.3.5 Modify Labels

You can change the labels of the levels using the `labels` argument. The labels must be in the same order as the levels. We will modify the labels to **Desk**, **Mob** & **Tab** for **Desktop**, **Mobile** & **Tablet** respectively.

```
factor(device,
  levels = c("Desktop", "Mobile", "Tablet"),
  labels = c("Desk", "Mob", "Tab"))
```

```
[1] Mob Desk Desk Desk Mob Desk Tab Tab Tab Desk Mob Desk Tab Tab Desk
[16] Mob Tab Desk Mob Mob Tab Tab Desk Mob Desk
Levels: Desk Mob Tab
```

You can see that not only the values but the levels are also modified.

12.3.6 Missing Values

Let us regenerate the device column but include some missing values (NA) deliberately to see how `factor()` handles them.

```
# sample with missing values
device <- sample(c("Desktop", "Mobile", "Tablet", NA), size = 25, replace = TRUE)
device
```

```
[1] "Mobile" "Desktop" NA "Tablet" NA NA "Mobile"
[8] "Desktop" NA NA "Desktop" "Tablet" "Desktop" NA
[15] NA "Desktop" "Tablet" "Desktop" "Desktop" "Mobile" "Desktop"
[22] "Desktop" "Mobile" NA "Desktop"
```

```
# store as categorical data
factor(device)
```

```
[1] Mobile Desktop <NA> Tablet <NA> <NA> Mobile Desktop <NA>
[10] <NA> Desktop Tablet Desktop <NA> <NA> Desktop Tablet Desktop
[19] Desktop Mobile Desktop Desktop Mobile <NA> Desktop
Levels: Desktop Mobile Tablet
```

NA is not shown as one of the levels. Why does this happen? By default, it will ignore them. If you look at the arguments of the `factor()` function, the `exclude` argument is set to `NA` by default i.e. NA is excluded automatically. What should we do to ensure that NA is also treated as a level? In order to treat NA as a level, set the `exclude` argument to `NULL`.

```
factor(device, exclude = NULL)
```

```
[1] Mobile Desktop <NA> Tablet <NA> <NA> Mobile Desktop <NA>
[10] <NA> Desktop Tablet Desktop <NA> <NA> Desktop Tablet Desktop
[19] Desktop Mobile Desktop Desktop Mobile <NA> Desktop
Levels: Desktop Mobile Tablet <NA>
```

As you can see, NA is displayed as one of the levels in the data.

12.3.7 Ordered Factors

So far, we have been looking at nominal data. Let us now explore how R handles ordered data. We will generate a new data set of satisfaction ratings to use in this section. Satisfaction ratings are widely used to measure a customer's satisfaction with an organization, service or a product.

```
rating <- sample(c("Dislike", "Neutral", "Like"), size = 25, replace = TRUE)
rating
```

```
[1] "Like" "Neutral" "Neutral" "Neutral" "Dislike" "Neutral" "Dislike"
[8] "Neutral" "Neutral" "Neutral" "Like" "Like" "Dislike" "Dislike"
[15] "Neutral" "Neutral" "Dislike" "Like" "Dislike" "Like" "Like"
[22] "Dislike" "Dislike" "Dislike" "Neutral"
```

It consists of three values Dislike, Neutral & Like in that order. You can see that there is an intrinsic order here. Like is better than neutral which in turn is better than dislike. While we can order them, we can't quantify the difference between them. We can't say neutral is so many times better than dislike.

Membership Testing

As we did earlier, let us check if the data is ordered using the membership function `is.ordered()`.

```
is.ordered(rating)
```

```
[1] FALSE
```

R returns `FALSE` as the variable `rating` is not ordered. Let us use `as.ordered()` to coerce it into an ordered factor.

```
as.ordered(rating)
```

```
[1] Like    Neutral Neutral Neutral Dislike Neutral Dislike Neutral Neutral
[10] Neutral Like    Like    Dislike Dislike Neutral Neutral Dislike Like
[19] Dislike Like    Like    Dislike Dislike Dislike Neutral
Levels: Dislike < Like < Neutral
```

Look at the last line where the levels are displayed. In case of ordered factors, you will see a `<` between the labels. This is used to indicate the order of the levels. Now `rating` is both an ordered but the order of the levels is not correct. It should be `Dislike < Neutral < Like` but is displayed in order of appearance in the data. Let us use the `factor()` function since we need more control over how the levels are ranked and set the `ordered` argument to `TRUE`.

```
factor(rating, ordered = TRUE)
```

```
[1] Like    Neutral Neutral Neutral Dislike Neutral Dislike Neutral Neutral
[10] Neutral Like    Like    Dislike Dislike Neutral Neutral Dislike Like
[19] Dislike Like    Like    Dislike Dislike Dislike Neutral
Levels: Dislike < Like < Neutral
```

The ranking of the levels has not changed and is still the same. Why is this happening? If you observe carefully, the ranking follows the alphabetical order (**D**esktop, **M**obile, **T**able). The `factor()` function uses the same order for the levels.

12.3.8 Modify Order of Levels

To change the order/ranking of the levels, we need to specify it using the `levels` argument. Let us do that in the next example.

```
factor(rating, levels = c("Dislike", "Neutral", "Like"), ordered = TRUE)
```

```
[1] Like    Neutral Neutral Neutral Dislike Neutral Dislike Neutral Neutral
[10] Neutral Like    Like    Dislike Dislike Neutral Neutral Dislike Like
[19] Dislike Like    Like    Dislike Dislike Dislike Neutral
Levels: Dislike < Neutral < Like
```

Now, you can see that the levels are ranked correctly. The `ordered()` function can also be used to create ordered factors. Let us recreate the previous example using the `ordered()` function.

```
ordered(rating, levels = c("Dislike", "Neutral", "Like"))
```

```
[1] Like    Neutral Neutral Neutral Dislike Neutral Dislike Neutral Neutral
[10] Neutral Like    Like    Dislike Dislike Neutral Neutral Dislike Like
[19] Dislike Like    Like    Dislike Dislike Dislike Neutral
Levels: Dislike < Neutral < Like
```

You can specify levels, modify labels and handle missing values using the `ordered()` function as well.

12.3.9 Key Functions

Function	Description
<code>is.factor()</code>	Identify
<code>is.ordered()</code>	
<code>as.factor()</code>	Convert
<code>as_factor()</code>	
<code>as.ordered()</code>	
<code>factor()</code>	Create
<code>ordered()</code>	

12.3.10 Summary

- R uses **factor** to handle categorical data.
- Use `as.factor()` or `as_factor()` to coerce other data types to **factor**.
- Use `is.factor()` or `is.ordered()` to identify factor & ordered factor respectively.
- Use `factor()` to

- specify labels
 - modify labels
 - handle missing data
 - create ordered factors
 - specify order of levels
- Use `ordered()` to create ordered factors.

12.3.11 Your Turn...

Use `analytics_raw.rds` data set to answer the below questions.

1. Check whether the below variables are factor
 - `device`
 - `page_depth`
 - `landing_page`
2. Coerce the following variables to type factor
 - `device`
 - `os`
 - `browser`
 - `user_type`
 - `channel`
 - `gender`
 - `landing_page`
 - `exit_page`
 - `city`
 - `country`
 - `user_type`
3. Use only the following levels in the `gender` column:
 - `male`
 - `female`
4. Include `NA` as a level in the `gender` column.
5. Change label of `NA` to `missing` in the `gender` column.
6. Change the labels of the levels in `user_type` column to
 - `New`
 - `Returning`
7. Check if the `user_rating` column is ordered. If not, coerce it to type ordered factor.

12.4 Summarize

12.4.1 Introduction

Categorical data cannot be summarized in the same way as numeric data. It does not make sense to look at range, standard deviation etc. since data consists of a few distinct values only. So how do we summarize such data? We can look at

- count/frequency
- proportion
- cumulative frequency
- cross table
- contingency table etc.

In this section, we will explore the above ways of summarizing categorical data. We will also spend some time learning about tables as you will be using them extensively while working with categorical data. R has many packages for tabulating data and we list and explore all of them in the last section of this chapter.

12.4.2 Number of Categories

From our case study, we want to know the number of devices used to browse the website, the name of the devices and the proportion of traffic they drive to our website. Let us begin with the number of devices. To view the number of groups/categories in a categorical variable, use `nlevels()`.

```
nlevels(data$device)
```

```
[1] 3
```

There are 3 categories of devices used by the visitors to browse the website. This can also be used for data sanitization i.e. as an analyst you know that there are only 3 valid categories of device into which any visitor can be classified into. If you see more than 3 categories, you might want to check if there are any issues in data collection or processing. Now that we know there are 3 categories of devices, let us check if they are valid. The `levels()` function will return the labels of the groups.

12.4.3 Category Names

Knowing the number of levels is useful but not sufficient. `levels()` is one of the most useful functions when it comes to dealing with categorical data.

```
levels(data$device)
```

```
[1] "Desktop" "Mobile"  "Tablet"
```

Other functions that you can use include `unique()` and `fct_unique()`. Both these functions will return the unique names/labels along with the levels while `levels()` returns the labels of the levels.

```
unique(data$device)
```

```
[1] Desktop Mobile  Tablet
Levels: Desktop Mobile Tablet
```

```
fct_unique(data$device)
```

```
[1] Desktop Mobile  Tablet
Levels: Desktop Mobile Tablet
```

12.4.4 Names & Counts

So we have checked the number of devices and their names. Let us now examine their distribution i.e. count/frequency. `table()` and `summary()` will display the levels and their counts while `fct_count()` will return a tibble with 2 columns (level & count). It is extremely useful for further data processing or visualization (using `ggplot2`).

```
table(data$device)
```

```
Desktop  Mobile  Tablet
 177282   63482   3634
```

```
fct_count(data$device)
```

```
# A tibble: 3 x 2
  f          n
  <fct>    <int>
1 Desktop 177282
2 Mobile  63482
3 Tablet  3634
```

```
summary(data$device)
```

Desktop	Mobile	Tablet
177282	63482	3634

12.4.5 Tables

In the previous section, we used the `table()` function to tabulate categorical data. We will recreate the tabulation for device and store it in a new variable `tab`.

```
tab <- table(data$device)
tab
```

Desktop	Mobile	Tablet
177282	63482	3634

What does this function return? It is not a `vector`, `list`, `data.frame` or `matrix`. Let us use the `class()` function to check the class of the object returned by `table()`. It returns an object of the class **table**. This is a new type of object. Let us spend some time understanding tables as they are useful for organizing and summarizing categorical data. **table** is also the most used object when it comes to dealing with categorical data.

The `table()` function returns the counts of the categories but let us say we want to view the proportion or percentage instead of counts i.e. the proportion or percentage of traffic driven to our website by the different devices. The `proportions()` or `prop.table()` function comes in handy in such cases. It takes a table object as input (`tab` in our case).

```
prop.table(tab)
```

Desktop	Mobile	Tablet
0.72538237	0.25974844	0.01486919

```
proportions(tab)
```

Desktop	Mobile	Tablet
0.72538237	0.25974844	0.01486919

To get the percentages, multiply the output by 100. Use the `round()` function to round the decimal places according to your requirements.

```
proportions(tab) * 100
```

```
Desktop    Mobile    Tablet
72.538237  25.974844  1.486919
```

```
round(proportions(tab) * 100, 2)
```

```
Desktop    Mobile    Tablet
72.54      25.97      1.49
```

So far, we have used `table()` to tabulate a single categorical variable. It can be used for a lot more than just tabulating data. We can examine the relationship between two categorical variables as well as create multidimensional tables. Let us look at the relationship between **gender** and **device** in our case study. Does gender affect the type of device used? To answer this, we will create a two way or cross table. In the `table()` function, we can specify multiple variables by separating them with a comma.

```
tab2 <- table(data$gender, data$device)
tab2
```

```
      Desktop Mobile Tablet
female  32803   7268   494
male    46418  14503   696
<NA>    98061  41711  2444
```

Keep in mind that the order of the variables matter. Rows represent the first variable while column represents the second.

```
table(data$device, data$gender)
```

```
      female male <NA>
Desktop 32803 46418 98061
Mobile  7268 14503 41711
Tablet  494  696  2444
```

The `proportions()` function works with two way tables as well.

```
proportions(tab2)
```

	Desktop	Mobile	Tablet
female	0.134219593	0.029738378	0.002021293
male	0.189927904	0.059341729	0.002847814
<NA>	0.401234871	0.170668336	0.010000082

```
proportions(tab2) * 100
```

	Desktop	Mobile	Tablet
female	13.4219593	2.9738378	0.2021293
male	18.9927904	5.9341729	0.2847814
<NA>	40.1234871	17.0668336	1.0000082

We would like to introduce another function at this point of time, `margin.table()`. What does this function do? It computes the marginal frequencies i.e. the sum of the rows or columns. It takes a **table** object as input. The `margin` argument allows us to specify whether we want the sum of rows or columns. 1 indicates rows and 2 indicates columns.

```
margin.table(tab2, 1) # sum of rows
```

female	male	<NA>
40565	61617	142216

```
margin.table(tab2, 2) # sum of columns
```

Desktop	Mobile	Tablet
177282	63482	3634

If the `margin` argument is `NULL` (which it is by default), the function returns the sum of all cells of the table.

```
margin.table(tab2)
```

```
[1] 244398
```

`table()` does not display row or column labels. It does display the group labels though. Let us revisit the output from `tab2`. You can observe that while it includes the group labels, the row and column labels are missing. The output from the `dimnames()` function shows the group labels of the variables but the row & column labels are absent.

```
dimnames(tab2)
```

```
[[1]]  
[1] "female" "male"   NA  
  
[[2]]  
[1] "Desktop" "Mobile"  "Tablet"
```

```
names(tab2)
```

```
NULL
```

```
names(dimnames(tab2))
```

```
[1] "" ""
```

The output from `names(dimnames(tab2))` is also empty. Let us add the variable names as the row & column labels to `tab2`.

```
names(dimnames(tab2)) <- c("Gender", "Device")  
tab2
```

	Device		
Gender	Desktop	Mobile	Tablet
female	32803	7268	494
male	46418	14503	696
<NA>	98061	41711	2444

Now look at the output from `tab2` and you can observe the difference. The same is also visible when we run `dimnames(tab2)`.

```
dimnames(tab2)
```

```
$Gender  
[1] "female" "male"   NA  
  
$Device  
[1] "Desktop" "Mobile"  "Tablet"
```

To add margin totals to the table, use `addmargins()`. Like `proportions()` and `margin.table()`, it also takes a `table` object as the input.

```
addmargins(tab2)
```

	Device			
Gender	Desktop	Mobile	Tablet	Sum
female	32803	7268	494	40565
male	46418	14503	696	61617
<NA>	98061	41711	2444	142216
Sum	177282	63482	3634	244398

`rowSums()` returns the row total while `colSums()` returns the column total. They are similar to `margin.table()`.

```
rowSums(tab2)
```

female	male	<NA>
40565	61617	142216

```
colSums(tab2)
```

Desktop	Mobile	Tablet
177282	63482	3634

`xtabs()` is another way of creating multidimensional tables in R. In comparison to `table()`, it

- uses formula notation for input
- the `data` argument ensures variable names are referenced instead of using `$` i.e. `data$variable`
- displays row & column labels by default

```
tabx <- xtabs(~gender+device, data = data)
tabx
```

	device		
gender	Desktop	Mobile	Tablet
female	32803	7268	494
male	46418	14503	696
<NA>	98061	41711	2444

The following functions work with `xtabs()` as well

- `proportions()`
- `margin.table()`
- `addmargins()`

```
proportions(tabx)
```

	device		
gender	Desktop	Mobile	Tablet
female	0.134219593	0.029738378	0.002021293
male	0.189927904	0.059341729	0.002847814
<NA>	0.401234871	0.170668336	0.010000082

```
margin.table(tabx, 1)
```

gender			
	female	male	<NA>
	40565	61617	142216

```
margin.table(tabx, 2)
```

device			
	Desktop	Mobile	Tablet
	177282	63482	3634

```
addmargins(tabx)
```

	device			
gender	Desktop	Mobile	Tablet	Sum
female	32803	7268	494	40565
male	46418	14503	696	61617
<NA>	98061	41711	2444	142216
Sum	177282	63482	3634	244398

So far, we have been working with one or two dimensional tables. Both the `table()` and `xtabs()` functions are capable of creating multidimensional tables. Keep in mind that multidimensional tables are complex and it becomes increasingly difficult to understand or interpret them.

```
tab3 <- xtabs(~gender+device+channel, data = data)
tab3
```

, , channel = (Other)

	device		
gender	Desktop	Mobile	Tablet
female	786	258	0
male	1063	507	19
<NA>	2173	1186	81

, , channel = Affiliates

	device		
gender	Desktop	Mobile	Tablet
female	1314	60	0
male	1714	169	0
<NA>	3518	548	65

, , channel = Direct

	device		
gender	Desktop	Mobile	Tablet
female	4785	977	59
male	7010	2381	95
<NA>	15824	8292	430

, , channel = Display

	device		
gender	Desktop	Mobile	Tablet
female	123	753	104
male	210	491	73
<NA>	554	911	156

, , channel = Organic Search

	device		
gender	Desktop	Mobile	Tablet
female	17109	4480	282
male	25016	9563	448
<NA>	54071	27223	1476

, , channel = Paid Search

	device		
gender	Desktop	Mobile	Tablet
female	645	230	22
male	887	478	26

```

      <NA>      1274      782      51

, , channel = Referral

      device
gender  Desktop Mobile Tablet
female   7387      74       0
male     9251     185       0
<NA>    18052     615      51

, , channel = Social

```

```

      device
gender  Desktop Mobile Tablet
female   654     436      27
male     1267     729      35
<NA>    2595    2154     134

```

ftable stands for flat tables and is useful for printing attractive tables. It makes it easy to read and interpret multidimensional tables. In the next example, we will use `f`table() to print the tables we have created in the previous examples and compare the outputs.

```
f
```

table(tabx)

```

      device Desktop Mobile Tablet
gender
female      32803    7268    494
male        46418   14503    696
NA           98061   41711   2444

```

```
f
```

table(tab2)

```

      Device Desktop Mobile Tablet
Gender
female      32803    7268    494
male        46418   14503    696
NA           98061   41711   2444

```

```
f
```

table(tab3)

```

      channel (Other) Affiliates Direct Display Organic Search Paid Search Referr
gender device
female Desktop      786      1314    4785    123      17109      645    73

```

	Mobile	258	60	977	753	4480	230	
	Tablet	0	0	59	104	282	22	
male	Desktop	1063	1714	7010	210	25016	887	92
	Mobile	507	169	2381	491	9563	478	1
	Tablet	19	0	95	73	448	26	
NA	Desktop	2173	3518	15824	554	54071	1274	180
	Mobile	1186	548	8292	911	27223	782	6
	Tablet	81	65	430	156	1476	51	

By default, missing values (NAs) are excluded from tables. Let us modify the gender data from our case study a bit and see how the `table()` function deals with missing values. We won't explicitly specify `NA` as a level while recreating the gender data.

```
gen <- as.factor(as.character(data$gender))
table(gen)
```

```
gen
female  male
 40565  61617
```

As you can see, `table()` excludes missing values while tabulating the data. In order to ensure that missing values are also counted, we can use the `useNA` argument. It can take two values:

- `ifany`
- `always`

In the first case, it will show `NA` as a level and the count only if there are missing values in the data. In the second case, it will always show `NA` as a level irrespective of whether there are missing values in the data or not.

```
table(gen, useNA = "ifany")
```

```
gen
female  male  <NA>
 40565  61617 142216
```

```
table(data$device, useNA = "always")
```

```
Desktop  Mobile  Tablet  <NA>
 177282   63482   3634    0
```

In this final section on tables, we will learn how to select/access the different parts of a `table`. We will use `[]` operator to select rows and columns of a table (it is similar to selecting data from a `data.frame`). Below are a few examples:

- select first row

```
tab2[1, ]
```

Desktop	Mobile	Tablet
32803	7268	494

- select first column

```
tab2[, 1]
```

female	male	<NA>
32803	46418	98061

- select first two rows

```
tab2[1:2, ]
```

	Device		
Gender	Desktop	Mobile	Tablet
female	32803	7268	494
male	46418	14503	696

- select first two columns

```
tab2[, 1:2]
```

	Device		
Gender	Desktop	Mobile	
female	32803	7268	
male	46418	14503	
<NA>	98061	41711	

- select nth row

```
tab2[2, ]
```

Desktop	Mobile	Tablet
46418	14503	696

- select nth column

```
tab2[, 2]
```

```
female  male  <NA>  
7268   14503  41711
```

- select row by group label

```
tab2["female", ]
```

```
Desktop  Mobile  Tablet  
32803    7268    494
```

- select column by group label

```
tab2[, "Mobile"]
```

```
female  male  <NA>  
7268   14503  41711
```

Before we end this section, let us learn how to test if an object is of class table using `is.table()`.

```
is.table(tab2)
```

```
[1] TRUE
```

Next, we will look at different R packages for two way/contingency tables.

12.4.6 Contingency Table

For cross tables with output similar to SAS or SPSS, use any of the below:

- `CrossTable()` from the `gmodels` package
- `ds_cross_table()` from the `descriptr` package

```
gmodels::CrossTable(data$device, data$gender)
```

Cell Contents	

	N
	Chi-square contribution
	N / Row Total
	N / Col Total
	N / Table Total

Total Observations in Table: 102182

	data\$gender		
data\$device	female	male	Row Total
----- ----- ----- -----			
Desktop	32803	46418	79221
	58.228	38.334	
	0.414	0.586	0.775
	0.809	0.753	
	0.321	0.454	
----- ----- ----- -----			
Mobile	7268	14503	21771
	218.694	143.975	
	0.334	0.666	0.213
	0.179	0.235	
	0.071	0.142	
----- ----- ----- -----			
Tablet	494	696	1190
	0.986	0.649	
	0.415	0.585	0.012
	0.012	0.011	
	0.005	0.007	
----- ----- ----- -----			
Column Total	40565	61617	102182
	0.397	0.603	
----- ----- ----- -----			

```
descriptr::ds_cross_table(data, device, gender)
```

Cell Contents

Frequency
Percent
Row Pct
Col Pct

Total Observations: 244398

		gender			
device	female	male	NA	Row Total	
Desktop	32803	46418	98061	177282	
	0.134	0.19	0.401		
	0.19	0.26	0.55	0.73	
	0.81	0.75	0.69		
Mobile	7268	14503	41711	63482	
	0.03	0.059	0.171		
	0.11	0.23	0.66	0.26	
	0.18	0.24	0.29		
Tablet	494	696	2444	3634	
	0.002	0.003	0.01		
	0.14	0.19	0.67	0.01	
	0.01	0.01	0.02		
Column Total	40565	61617	142216	244398	
	0.166	0.252	0.582		

We list and explore different R packages for summarizing categorical data in the last section of this chapter.

12.4.7 Key Functions

Function	Description
<code>nlevels()</code>	Number of Levels
<code>levels()</code>	Levels / Groups / Categories
<code>unique()</code>	
<code>fct_unique()</code>	
<code>table()</code>	Count / Frequency
<code>summary()</code>	
<code>fct_count()</code>	
<code>xtabs()</code>	Two Way / Multi Way Tables
<code>ftable()</code>	
<code>gmodels::CrossTable()</code>	
<code>descriptr::ds_cross_table()</code>	

12.4.8 Your Turn...

1. Display the number of levels in

- `browser`
- `channel`
- `landing_page`
- `exit_page`

2. Display the categories in

- `os`
- `channel`
- `browser`
- `gender`
- `user_type`

3. Display the count/frequency of

- `channel`
- `user_type`

4. Examine the distribution of the following and summarize your observations:

- `channel` by `user_type`
- `device` by `purchase_flag`
- `channel` by `device`
- `channel` by `purchase_flag`
- `user_type` by `purchase_flag`

13 Factor Manipulation with forcats

This chapter continues @categorical-data-in-r. We reorder, relevel, recode, collapse and lump factors with **forcats**, and visualize them with **ggplot2** only.

13.1 Data Manipulation

13.1.1 Introduction

In this section, our focus will be on handling the levels of a categorical variable, and exploring the **forcats** package for the same. We will basically look at 3 key operations or transformations we would like to do when it comes to factors which are:

- change value of levels
- add or remove levels
- change order of levels

Before we start working with the value of the levels, let us take a quick look at some of the functions we used in the previous sections. We will store the source of traffic as **channel** instead of referring to the column in the **data.frame** every time.

```
channel <- data$channel
```

Let us go back to the function we used for tabulating data, **fct_count()**. If you observe the result, it is in the same order as displayed by **levels()**.

```
fct_count(channel)
```

```
# A tibble: 8 x 2
  f                n
  <fct>          <int>
1 (Other)        6073
2 Affiliates      7388
3 Direct        39853
4 Display        3375
5 Organic Search 139668
6 Paid Search     4395
7 Referral       35615
8 Social         8031
```

If you want to sort the results by the count i.e. most common level comes at the top, use the `sort` argument.

Channel	Count		Channel	Count
(Other)	6073		Organic Search	139668
Affiliates	7388		Direct	39853
Direct	39853		Referral	35615
Display	3375		Social	8031
Organic Search	139668		Affiliates	7388
Paid Search	4395		(Other)	6073
Referral	35615		Paid Search	4395
Social	8031		Display	3375

```
fct_count(channel, sort = TRUE)
```

```
# A tibble: 8 x 2
  f           n
<fct>     <int>
1 Organic Search 139668
2 Direct        39853
3 Referral      35615
4 Social         8031
5 Affiliates     7388
6 (Other)        6073
7 Paid Search   4395
8 Display       3375
```

If you want to view the proportion along with the count, set the `prop` argument to `TRUE`.

```
fct_count(channel, prop = TRUE)
```

```
# A tibble: 8 x 3
  f           n     p
<fct>     <int> <dbl>
1 (Other)    6073 0.0248
2 Affiliates 7388 0.0302
3 Direct    39853 0.163
4 Display   3375 0.0138
5 Organic Search 139668 0.571
6 Paid Search 4395 0.0180
7 Referral  35615 0.146
8 Social     8031 0.0329
```

One of the important steps in data preparation/sanitization is to check if the levels are valid i.e. only levels which should be present in the data are actually present. `fct_match()` can be used to check validity of levels. It returns a logical vector if the level is present and an error if not.

```
channel |>
  fct_match("Social") |>
  table()
```

```
FALSE  TRUE
236367 8031
```

13.1.2 Your Turn...

1. Display the count/frequency of the following variables in the descending order
 - device
 - landing_page
 - exit_page
2. Check if `laptop` is a level in the `device` column.

13.1.3 Change Value of Levels

In this section, we will learn how to change the value of the levels. In order to keep it interesting, we will state an objective from our case study and then map it into a function from the `forcats` package.

Combine both Paid & Organic Search into a single level, Search

Channel	Count		Channel	Count
(Other)	6073		(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375		Display	3375
Organic Search	139668	→ Search = c(Paid, Organic)	Search	144063
Paid Search	4395		Referral	35615
Referral	35615		Social	8031
Social	8031			

In this case, we want to change the value of two levels, *Paid Search* & *Organic Search* and give them the common value *Search*. You can also look at it as collapsing two levels into one. There are two functions we can use here:

- `fct_collapse()`
- `fct_recode()`

Let us look at `fct_collapse()` first. After specifying the categorical variable, we specify the new value followed by a character vector of the existing values. Remember, the new value is not enclosed in quotes (single or double) but the existing values must be a **character** vector.

```
channel |>
  fct_collapse(Search = c("Paid Search", "Organic Search")) |>
  fct_count()
```

```
# A tibble: 7 x 2
  f         n
  <fct>   <int>
1 (Other)  6073
2 Affiliates 7388
3 Direct   39853
4 Display  3375
5 Search  144063
6 Referral 35615
7 Social   8031
```

In the case of `fct_recode()`, each value being changed must be specified in a new line. Similar to `fct_collapse()`, the new value is not enclosed in quotes but the existing values must be.

Channel	Count		Channel	Count
(Other)	6073	Search = "Paid Search" Search = "Organic Search"	(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375		Display	3375
Organic Search	139668		Search	144063
Paid Search	4395		Referral	35615
Referral	35615		Social	8031
Social	8031			

```
channel |>
  fct_recode(
    Search = "Paid Search",
    Search = "Organic Search"
  ) |>
  fct_count()
```

```
# A tibble: 7 x 2
  f           n
  <fct>     <int>
1 (Other)    6073
2 Affiliates 7388
3 Direct    39853
4 Display    3375
5 Search   144063
6 Referral  35615
7 Social     8031
```

The `dplyr` and `car` packages also have recode functions.

Retain only those channels which have driven a minimum traffic of 5000 to the website

Channel	Count		Channel	Count
(Other)	6073	Lump < 5000	(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375		Organic Search	139668
Organic Search	139668		Referral	35615
Paid Search	4395		Social	8031
Referral	35615		Other	7770
Social	8031			

Instead of having all the channels, we desire to retain only those channels which have driven at least 5000 visits to the website. What about the rest of the channels which have driven less than 5000? We will recategorize them as **Other**. Keep in mind that we already have a **(Other)** level in our data. `fct_lump_min()` will lump together all levels which do not have a minimum count specified. In our case study, only **Display** drives less than 5000 visits and it will be categorized into **Other**.

```
channel |>
  fct_lump_min(min = 5000) |>
  fct_count()
```

```
# A tibble: 7 x 2
  f           n
  <fct>       <int>
1 (Other)     6073
2 Affiliates  7388
3 Direct     39853
4 Organic Search 139668
5 Referral    35615
6 Social      8031
7 Other       7770
```

Retain only top 3 referring channels and categorize rest into Other

Channel	Count		Channel	Count
(Other)	6073	Retain top 3 Channels →	Direct	39853
Affiliates	7388		Organic Search	139668
Direct	39853		Referral	35615
Display	3375		Other	29262
Organic Search	139668			
Paid Search	4395			
Referral	35615			
Social	8031			

Suppose you decide to retain only the top 3 channels in terms of the traffic driven to the website. In our case study, these are **Direct**, **Organic Search** and **Referral**. We want to retain these 3 levels and categorize the rest as **Other**. `fct_lump_n()` will retain top `n` levels by count/frequency and lump the rest into **Other**.

```
channel |>
  fct_lump_n(n = 3) |>
  fct_count()
```

```
# A tibble: 4 x 2
  f           n
  <fct>       <int>
1 Direct     39853
```

```
2 Organic Search 139668
3 Referral      35615
4 Other         29262
```

In our case, n is 3 and hence the top 3 channels in terms of traffic driven are retained while the rest are lumped into **Other**.

Retain only those channels which have driven at least 2% of the overall traffic

Channel	Count		Channel	Count
(Other)	6073	Lump < 2% Traffic →	(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375		Organic Search	139668
Organic Search	139668		Referral	35615
Paid Search	4395		Social	8031
Referral	35615		Other	7770
Social	8031			

In the second scenario above, we retained channels based on minimum traffic driven by them to the website. The criteria was count of visits. If you want to specify the criteria as a percentage or proportion instead of count, use `fct_lump_prop()`. The criteria is a value between **0** and **1**. In our case study, we want to retain channels that have driven at least **2%** of the overall traffic. Hence, we have specified the criteria as **0.02**.

```
channel |>
  fct_lump_prop(prop = 0.02) |>
  fct_count()
```

```
# A tibble: 7 x 2
  f          n
<fct>    <int>
1 (Other)    6073
2 Affiliates  7388
3 Direct    39853
4 Organic Search 139668
5 Referral   35615
6 Social     8031
7 Other     7770
```

As you can see, only **Display** drives less than **2%** of overall traffic and has been lumped into **Other**.

Retain the following channels and merge the rest into Other

- Organic Search
- Direct
- Referral

In the previous scenarios, we have been retaining or lumping channels based on some criteria like count or percentage of traffic driven to the website. In this scenario, we want to retain certain levels by specifying their labels and combine the rest into **Other**. While we can use `fct_collapse()` or `fct_recode()`, a more appropriate function would be `fct_other()`. We will do a comparison of the three functions in a short while.

Channel	Count		Channel	Count
(Other)	6073	Retain specific Channels →	Direct	39853
Affiliates	7388		Organic Search	139668
Direct	39853		Referral	35615
Display	3375		Other	29262
Organic Search	139668			
Paid Search	4395			
Referral	35615			
Social	8031			

`fct_other()` has two arguments, **keep** and **drop**. **keep** is used when we know the levels we want to retain and **drop** is used when we know the levels we want to drop. In this scenario, we know the levels we want to retain and hence we will use the **keep** argument and specify them. **Organic Search**, **Direct** and **Referral** will be retained while the rest of the channels will be lumped into **Other**.

```
# channels to be retained
retain <- c("Organic Search", "Direct", "Referral")

channel |>
  fct_other(keep = retain) |> #<<
  fct_count()
```

```
# A tibble: 4 x 2
  f           n
<fct>     <int>
1 Direct   39853
```

2 Organic Search 139668
 3 Referral 35615
 4 Other 29262

Merge the following channels into Other and retain rest of them:

- Display
- Paid Search

Channel	Count		Channel	Count
(Other)	6073	Drop specific channels →	(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375		Organic Search	139668
Organic Search	139668		Referral	35615
Paid Search	4395		Social	8031
Referral	35615		Other	7770
Social	8031			

In this scenario, we know the levels we want to drop and hence we will use the **drop** argument and specify them. **Display** and **Paid Search** will be lumped into **Other** while the rest of the channels will be retained.

```
# channels to be dropped
dropped <- c("Display", "Paid Search")

channel |>
  fct_other(drop = dropped) |>
  fct_count()
```

```
# A tibble: 7 x 2
  f          n
<fct>    <int>
1 (Other)    6073
2 Affiliates  7388
3 Direct    39853
4 Organic Search 139668
5 Referral   35615
6 Social     8031
7 Other     7770
```

In the previous scenario, we said we will compare `fct_other()` with `fct_collapse()` and `fct_recode()`. Let us use the other two functions as well and see the difference.

```
# collapse
channel |>
  fct_collapse(Other = c("(Other)",
                        "Affiliates",
                        "Display",
                        "Paid Search",
                        "Social")) |>
  fct_count()
```

```
# A tibble: 4 x 2
  f           n
<fct>     <int>
1 Other      29262
2 Direct     39853
3 Organic Search 139668
4 Referral   35615
```

```
# recode
channel |>
  fct_recode(Other = "(Other)",
            Other = "Affiliates",
            Other = "Display",
            Other = "Paid Search",
            Other = "Social") |>
  fct_count()
```

```
# A tibble: 4 x 2
  f           n
<fct>     <int>
1 Other      29262
2 Direct     39853
3 Organic Search 139668
4 Referral   35615
```

As you can observe, `fct_other()` requires less typing and is easier to specify.

Anonymize the data set before sharing it with your colleagues

Channel	Count	Anonymize →	Channel	Count
(Other)	6073		ch_1	7388
Affiliates	7388		ch_2	139668
Direct	39853		ch_3	8031
Display	3375		ch_4	4395
Organic Search	139668		ch_5	35615
Paid Search	4395		ch_6	3375
Referral	35615		ch_7	39853
Social	8031		ch_8	6073

Anonymizing data is extremely important when you are sharing sensitive data with others. Here, we want to anonymize the channels which drive traffic to the website so that we can share it with others without divulging the names of the channels. `fct_anon()` allows us to anonymize the levels in the data. Using the `prefix` argument, we can specify the prefix to be used while anonymizing the data.

```
channel |>
  fct_anon(prefix = "ch_") |>
  fct_count()
```

```
# A tibble: 8 x 2
  f         n
<fct> <int>
1 ch_1 139668
2 ch_2  4395
3 ch_3  7388
4 ch_4 35615
5 ch_5  3375
6 ch_6  8031
7 ch_7 39853
8 ch_8  6073
```

13.1.4 Key Functions

Function	Description
<code>'fct_collapse()'</code>	Collapse factor levels
<code>'fct_recode()'</code>	Recode factor levels
<code>'fct_lump_min()'</code>	Lump factor levels with count lesser than specified value
<code>'fct_lump_n()'</code>	Lump all levels except the top n levels

<code>'fct_lump_prop()'</code>	Lump factor levels with count lesser than specified proportion
<code>'fct_lump_lowfreq()'</code>	Lump together least frequent levels
<code>'fct_other()'</code>	Replace levels with Other level
<code>'fct_anon()'</code>	Anonymize factor levels

13.1.5 Your Turn..

- Combine the following levels in `landing_page` into `Account`
 - My Account
 - Register
 - Sign In
 - Your Info
- Combine levels in `landing_page` that drive less than 1000 visits.
- Get top 10 landing and exit pages.
- Get landing pages that drive at least 5% of the total traffic to the website.
- Retain only the following levels in the `browser` column:
 - Chrome
 - Firefox
 - Safari
 - Edge
- Anonymize landing and exit page levels.

13.1.6 Add / Remove Levels

In this small section, we will learn to:

- add new levels
- drop levels
- make missing values explicit

Add a new level, Blog

Channel	Count
(Other)	6073
Affiliates	7388
Direct	39853
Display	3375
Organic Search	139668
Paid Search	4395
Referral	35615
Social	8031

Add new channel

Channel	Count
(Other)	6073
Affiliates	7388
Direct	39853
Display	3375
Organic Search	139668
Paid Search	4395
Referral	35615
Social	8031
Blog	0

`fct_expand()` allows us to add new levels to the data. The label of the new level must be specified after the variable name and must be enclosed in quotes. If the level already exists, it will be ignored. Let us add a new level, Blog.

```
channel |>
  fct_expand("Blog") |> #<<
  levels()
```

```
[1] "(Other)"      "Affiliates"   "Direct"       "Display"
[5] "Organic Search" "Paid Search"  "Referral"     "Social"
[9] "Blog"
```

Drop existing level

Channel	Count
(Other)	6073
Affiliates	7388
Direct	39853
Display	3375
Organic Search	139668
Paid Search	4395
Referral	35615
Social	8031
Blog	0

Drop Blog Channel

Channel	Count
(Other)	6073
Affiliates	7388
Direct	39853
Display	3375
Organic Search	139668
Paid Search	4395
Referral	35615
Social	8031

On the other hand, `fct_drop()` will drop levels which have no values i.e. unused levels. If you want to drop only specific levels, use the `only` argument and specify the name of the level in quotes. Let us drop the new level we added in the previous example.

```
channel |>
  fct_expand("Blog") |>
  fct_drop() |> #<<
  levels()
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Display"
[5] "Organic Search" "Paid Search"   "Referral"      "Social"
```

Make missing values explicit

Gender	Count		Gender	Count
female	40565	$\xrightarrow{\text{<NA>}}$ (Missing)	female	40565
male	61617		male	61617
<NA>	142216		(Missing)	142216

In our data set, the gender column has many missing values, and in R, missing values are represented by `NA`. Suppose you are sharing the data or analysis with someone who is not an R user, and does not know what `NA` represents. In such a scenario, we can use the `fct_na_value_to_level()` function to make the missing values in the gender column explicit i.e. it will appear as `(Missing)` instead of `NA`. This will help non R users to understand that there are missing values in the data.

```
data |>
  pull(gender) |>
  fct_na_value_to_level(level = "(Missing)") |> #<<
  fct_count()
```

```
# A tibble: 3 x 2
  f         n
<fct>   <int>
1 female  40565
2 male    61617
3 (Missing) 142216
```

13.1.7 Key Functions

Function	Description
<code>'fct_expand()'</code>	Add additional levels to a factor
<code>'fct_drop()'</code>	Drop unused factor levels
<code>'fct_na_value_to_level()'</code>	Make missing values explicit

13.1.8 Change Order of Levels

In this last part of this section, we will learn how to change the order of the levels. We will look at the following scenarios from our case study:

We want to make

- Organic Search the first level
- Referral the third level
- Display the last level

Organic Search is the first level

Channel	Count		Channel	Count
(Other)	6073		Organic Search	139668
Affiliates	7388		(Other)	6073
Direct	39853		Affiliates	7388
Display	3375	Organic Search is first level →	Direct	39853
Organic Search	139668		Display	3375
Paid Search	4395		Paid Search	4395
Referral	35615		Referral	35615
Social	8031		Social	8031

In this scenario, we want the levels to appear in a certain order. In the first case, we want **Organic Search** to be the first level. `fct_relevel()` allows us to manually reorder the levels. To move a level to the beginning, specify the label (it must be enclosed in quotes).

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"   "Direct"       "Display"
[5] "Organic Search" "Paid Search"  "Referral"     "Social"
```

```
channel |>
  fct_relevel("Organic Search") |> #<<
  levels()
```

```
[1] "Organic Search" "(Other)"      "Affiliates"    "Direct"
[5] "Display"        "Paid Search"   "Referral"      "Social"
```

Referral is the third level

Channel	Count		Channel	Count
(Other)	6073	Referral is the third level →	(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Referral	35615
Display	3375		Direct	39853
Organic Search	139668		Display	3375
Paid Search	4395		Organic Search	139668
Referral	35615		Paid Search	4395
Social	8031		Social	8031

The **after** argument is useful when we want to move the level to the end or anywhere between the beginning and end. In the second case, we want **Referral** to be the third level. After specifying the label, use the **after** argument and specify the level after which **Referral** should appear. Since we want to move it to the third position, we will set the value of **after** to 2 i.e. **Referral** should come after the second position.

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Display"
[5] "Organic Search" "Paid Search"   "Referral"      "Social"
```

```
channel |>
  fct_relevel("Referral", after = 2) |> #<<
  levels()
```

```
[1] "(Other)"      "Affiliates"    "Referral"      "Direct"
[5] "Display"      "Organic Search" "Paid Search"   "Social"
```

Display is the last level

Channel	Count		Channel	Count
(Other)	6073		(Other)	6073
Affiliates	7388		Affiliates	7388
Direct	39853		Direct	39853
Display	3375	→ Display is the last level	Organic Search	139668
Organic Search	139668		Paid Search	4395
Paid Search	4395		Referral	35615
Referral	35615		Social	8031
Social	8031		Display	3375

In this last case, we want to move **Display** to the end. If you know the number of levels, you can specify a value here. In our data, there are eight channels i.e. eight levels, so we can set the value of **after** to 7. What happens when we do not know the number of levels or if they tend to vary? In such cases, to move a level to the end, set the value of **after** to **Inf**.

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Display"
[5] "Organic Search" "Paid Search"   "Referral"      "Social"
```

```
channel |>
  fct_relevel("Display", after = Inf) |> #<<
  levels()
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Organic Search"
[5] "Paid Search"   "Referral"      "Social"        "Display"
```

Let us now look at a scenario where we want to order the levels by

- frequency (largest to smallest)
- order of appearance (in data)

Order levels by frequency

Channel	Count		Channel	Count
(Other)	6073	$\xrightarrow{\text{fct_infreq(channel)}}$	Organic Search	139668
Affiliates	7388		Direct	39853
Direct	39853		Referral	35615
Display	3375		Social	8031
Organic Search	139668		Affiliates	7388
Paid Search	4395		(Other)	6073
Referral	35615		Paid Search	4395
Social	8031		Display	3375

In the first case, the levels with the most frequency should appear at the top. `fct_infreq()` will order the levels by their frequency.

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"   "Direct"       "Display"
[5] "Organic Search" "Paid Search"  "Referral"     "Social"
```

```
# order levels by frequency
channel |>
  fct_infreq() |> #<<
  levels()
```

```
[1] "Organic Search" "Direct"       "Referral"     "Social"
[5] "Affiliates"    "(Other)"     "Paid Search"  "Display"
```

Order levels by appearance

Channel	Count		Channel	Count
(Other)	6073	$\xrightarrow{\text{fct_inorder(channel)}}$	Organic Search	139668
Affiliates	7388		Direct	39853
Direct	39853		Referral	35615
Display	3375		Affiliates	7388
Organic Search	139668		(Other)	6073
Paid Search	4395		Social	8031
Referral	35615		Display	3375
Social	8031		Paid Search	4395

In the second case, the order of the levels should be the same as the order of their appearance in the data. `fct_inorder()` will order the levels according to the order in which they appear in the data.

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Display"
[5] "Organic Search" "Paid Search"   "Referral"      "Social"
```

```
# order levels in order of appearance
channel |>
  fct_inorder() |> #<<
  levels()
```

```
[1] "Organic Search" "Direct"        "Referral"      "Affiliates"
[5] "(Other)"        "Social"        "Display"       "Paid Search"
```

Reverse the order of the levels

Channel	Count		Channel	Count
(Other)	6073	$\xrightarrow{\text{fct_rev(channel)}}$	Social	8031
Affiliates	7388		Referral	35615
Direct	39853		Paid Search	4395
Display	3375		Organic Search	139668
Organic Search	139668		Display	3375
Paid Search	4395		Direct	39853
Referral	35615		Affiliates	7388
Social	8031		(Other)	6073

The order of the levels can be reversed using `fct_rev()`.

```
levels(channel)
```

```
[1] "(Other)"      "Affiliates"    "Direct"        "Display"
[5] "Organic Search" "Paid Search"   "Referral"      "Social"
```

```
# reverse order of levels
channel |>
  fct_rev() |> #<<
  levels()
```

```
[1] "Social"          "Referral"        "Paid Search"     "Organic Search"
[5] "Display"         "Direct"          "Affiliates"      "(Other)"
```

Randomly shuffle the order of the levels

Channel	Count		Channel	Count
(Other)	6073	$\xrightarrow{\text{fct_shuffle(channel)}}$	Direct	39853
Affiliates	7388		(Other)	6073
Direct	39853		Organic Search	139668
Display	3375		Social	8031
Organic Search	139668		Affiliates	7388
Paid Search	4395		Referral	35615
Referral	35615		Display	3375
Social	8031		Paid Search	4395

The order of the levels can be randomly shuffled using `fct_shuffle()`.

```
levels(channel)
```

```
[1] "(Other)"          "Affiliates"      "Direct"          "Display"
[5] "Organic Search"  "Paid Search"     "Referral"        "Social"
```

```
# randomly shuffle order of levels
channel |>
  fct_shuffle() |> #<<
  levels()
```

```
[1] "Organic Search" "Direct"          "Paid Search"     "Social"
[5] "Affiliates"    "(Other)"         "Referral"        "Display"
```

13.1.9 Key Functions

Function	Description
<code>'fct_relevel()'</code>	Reorder factor levels
<code>'fct_shift()'</code>	Shift factor levels
<code>'fct_infreq()'</code>	Reorder factor levels by frequency
<code>'fct_rev()'</code>	Reverse order of factor levels
<code>'fct_inorder()'</code>	Reorder factor levels by first appearance

13.1.10 Your Turn...

9. Make `Home` first level in the `landing_page` column.
10. Make `Apparel` second level in the `landing_page` column.
11. Make `Specials` last level in the `landing_page` column.
12. Order the levels in the browser by frequency:
13. Order the levels in landing page by appearance:
14. Shuffle the levels in `os`
15. Reverse the levels in browser

13.2 Data Visualization

13.2.1 Introduction

In this section, we will learn to visualize categorical data. We will look at the following type of plots:

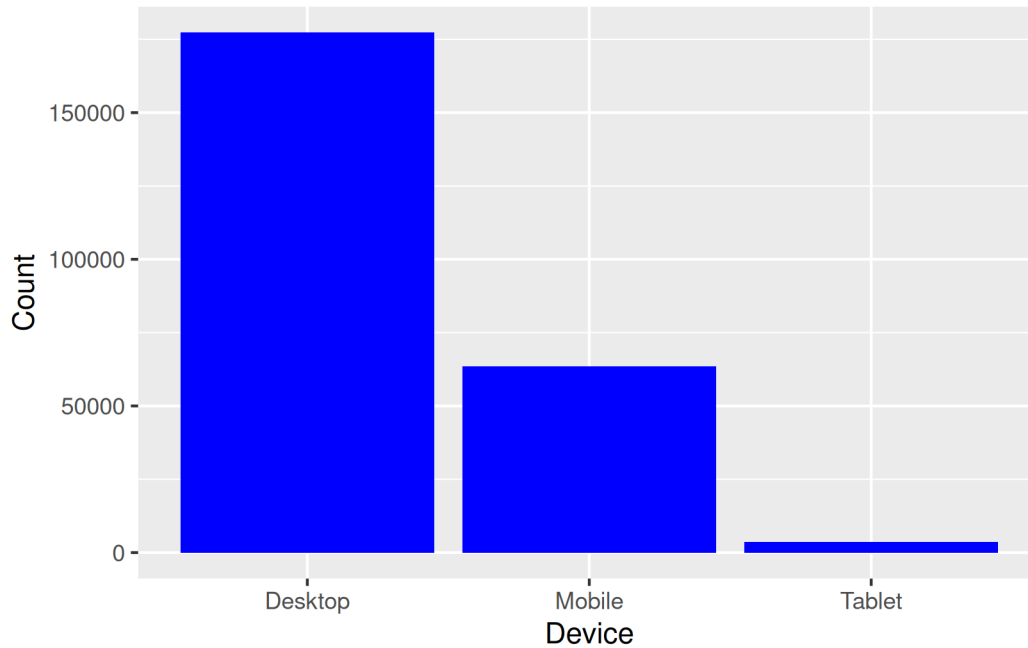
- univariate bar plot
- bivariate bar plot
 - grouped
 - stacked
 - proportional
- mosaic plot
- pie chart
- donut chart

We will be using [ggplot2](#) in this chapter. So you should know the basics of data visualization with `ggplot2`. If you are new or have never used `ggplot2`, do not worry. We have several [tutorials](#) and an [ebook](#) on `ggplot2`, you can go through them first and then come back to this section.

13.2.2 Bar Plot

Bar charts provide a visual representation of categorical data. The bars can be plotted either vertically or horizontally. The categories/groups appear along the horizontal X axis and the height of the bar represents a measured value.

```
ggplot(data) +  
  geom_bar(aes(x = device), fill = "blue") +  
  xlab("Device") + ylab("Count")
```

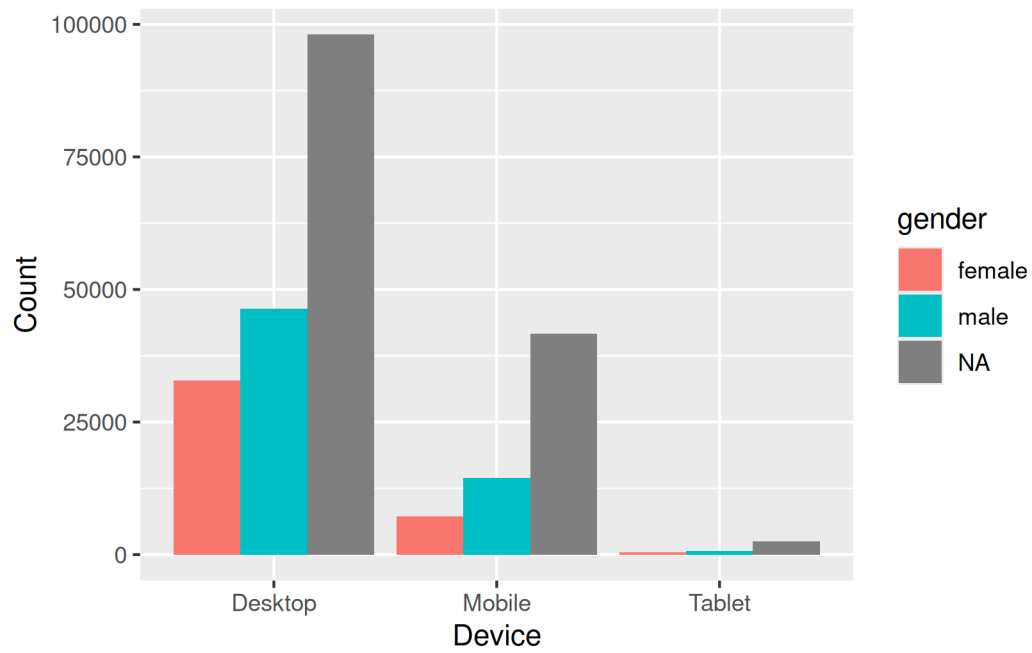


In the above example, the bars represent the count/frequency of the categories. If the bars represent continuous data, the value could be *mean* or *sum* of the variable being represented.

13.2.3 Grouped Bar Plot

A grouped bar chart plots values for two levels of a categorical variable instead of one. You should use grouped bar chart when making comparisons across different categories of data. Use it when you want to look at how the second category variable changes within each level of the first and vice versa.

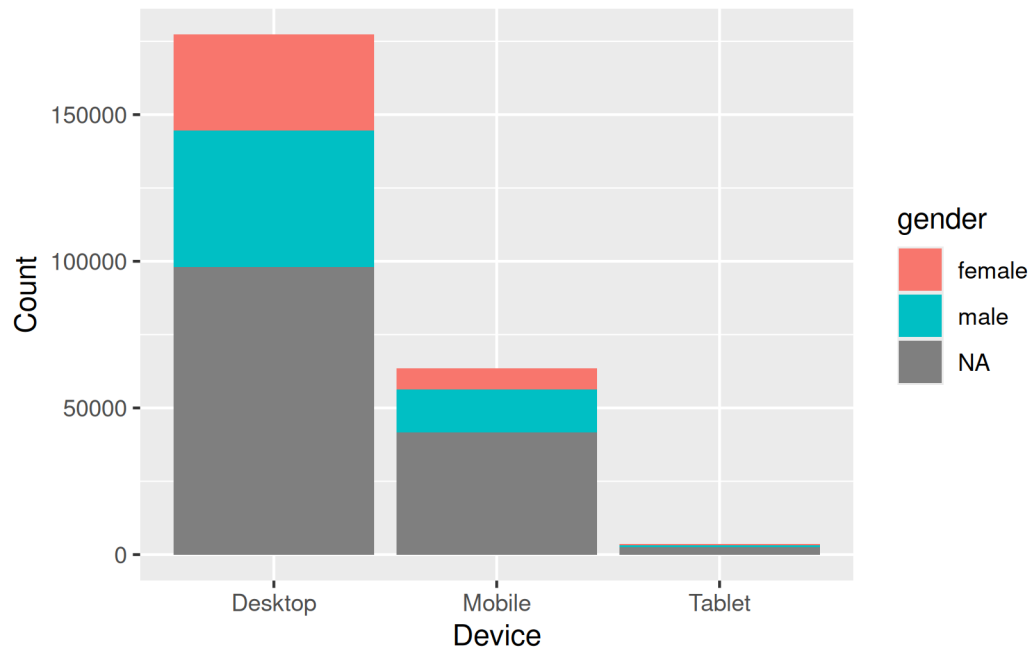
```
ggplot(data) +  
  geom_bar(aes(x = device, fill = gender), position = "dodge") +  
  xlab("Device") + ylab("Count")
```



13.2.4 Stacked Bar Plot

In stacked bar plots, the bars are stacked on top of each other instead of placing them next to each other. Use stacked bar plots while looking at cumulative value.

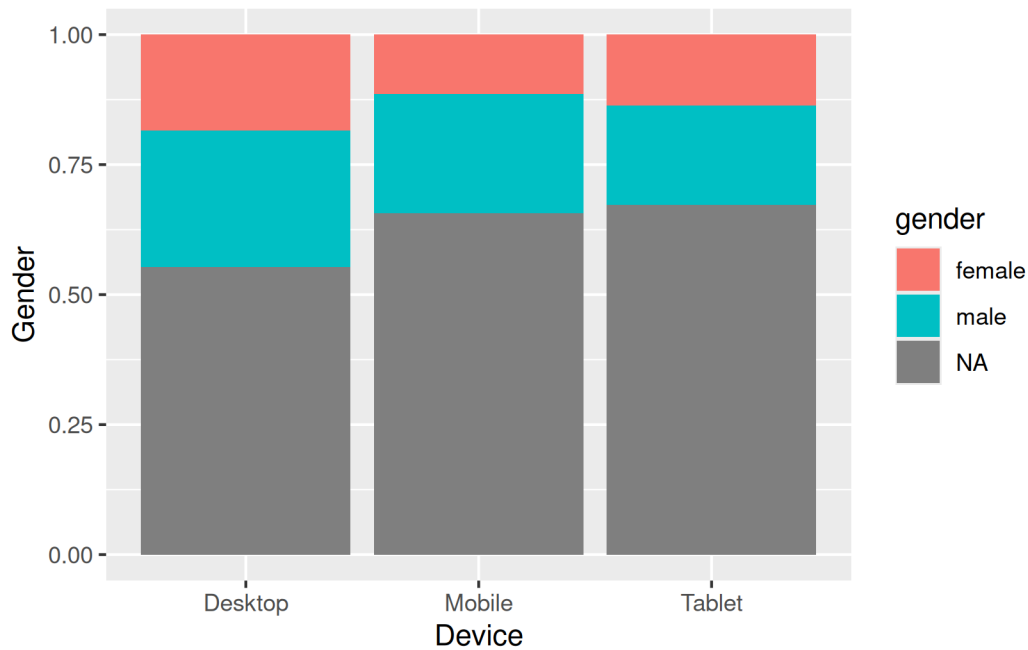
```
ggplot(data) +  
  geom_bar(aes(x = device, fill = gender)) +  
  xlab("Device") + ylab("Count")
```



13.2.5 Proportional Bar Plot

Also known as percent stacked plot, the height of all bars in this plot are the same. The distribution of the second categorical variable is scaled to 1 or 100. The length of each bar is determined by its share in the category. Use this when you want to concurrently observe each of several variables as they fluctuate and as their percentage ratio's change.

```
data |>
  select(device, gender) |>
  table() |>
  tibble::as_tibble() |>
  ggplot(aes(x = device, y = n, fill = gender)) +
  geom_bar(stat = "identity", position = "fill") +
  xlab("Device") + ylab("Gender")
```



13.2.6 Mosaic Plot

A mosaic plot is a graphical representation of a two way table or contingency table. It was introduced by Hartigan & Kleiner and is divided into rectangles. Proportions on horizontal axis represents the number of observations for each level of the X variable. The vertical length of each rectangle is proportional to the proportion of Y variable in each level of X variable.

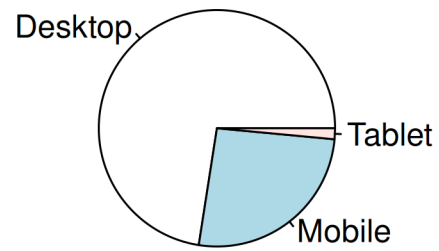
Mosaic plots need the **ggmosaic** package (archived on CRAN) and are out of scope here — they are covered in the [ggplot2 ebook](#).

13.2.7 Pie Chart

Pie chart is a circular chart, divided into slices to show relevant sizes of data. It shows the distribution of the different levels of a categorical variable as a circle is divided into radial slices. Each level corresponds with a single slice of the circle and size indicates the proportion of the level. Use it when comparing each group's contribution to the whole as opposed to comparing groups to each other.

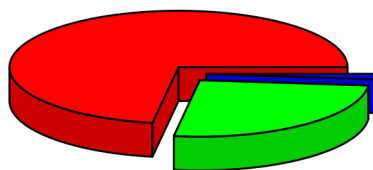
Base R

```
data |>
  pull(device) |>
  table() |>
  pie()
```



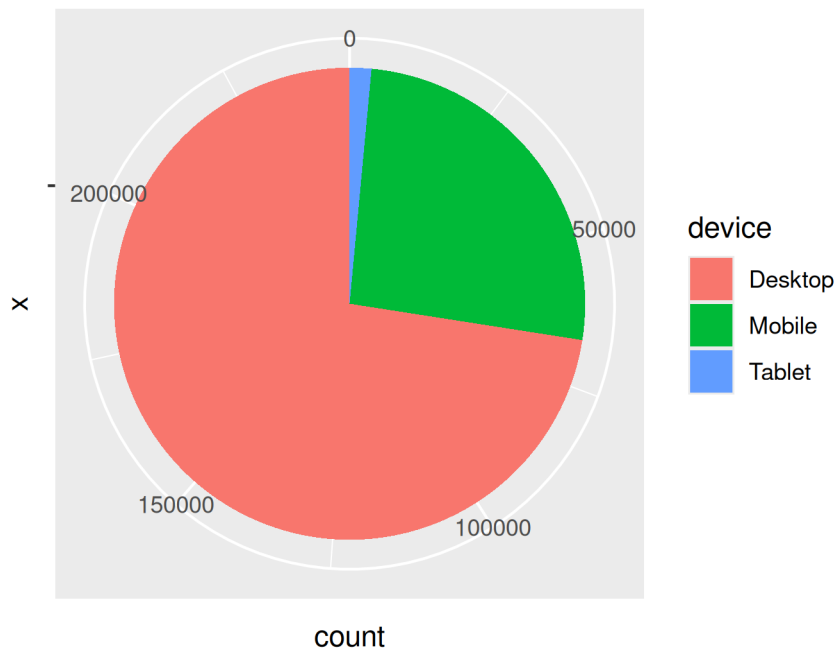
3D Pie Chart

```
data |>  
  pull(device) |>  
  table() |>  
  pie3D(explode = 0.1)
```



ggplot2

```
data |>
  pull(device) |>
  fct_count() |>
  rename(device = f, count = n) |>
  ggplot() +
  geom_bar(aes(x = "", y = count, fill = device), width = 1, stat = "identity") +
  coord_polar("y", start = 0)
```

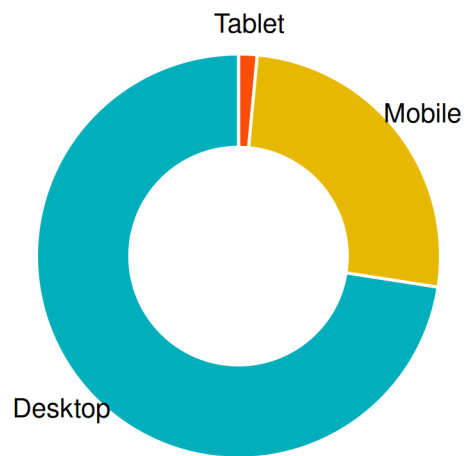


13.2.8 Donut Chart

Donut chart is a variation of the pie chart. It has a round hole in the middle which makes it look like a donut. The focus is on the length of the arcs and not the proportions of the slices. Blank spaces inside donut chart can be used to display information inside it.

```
data |>
  pull(device) |>
  fct_count() |>
  rename(device = f, count = n) |>
  ggdonutchart("count", label = "device", fill = "device", color = "white",
    palette = c("#00AFBB", "#E7B800", "#FC4E07"))
```

device Desktop Mobile Tablet



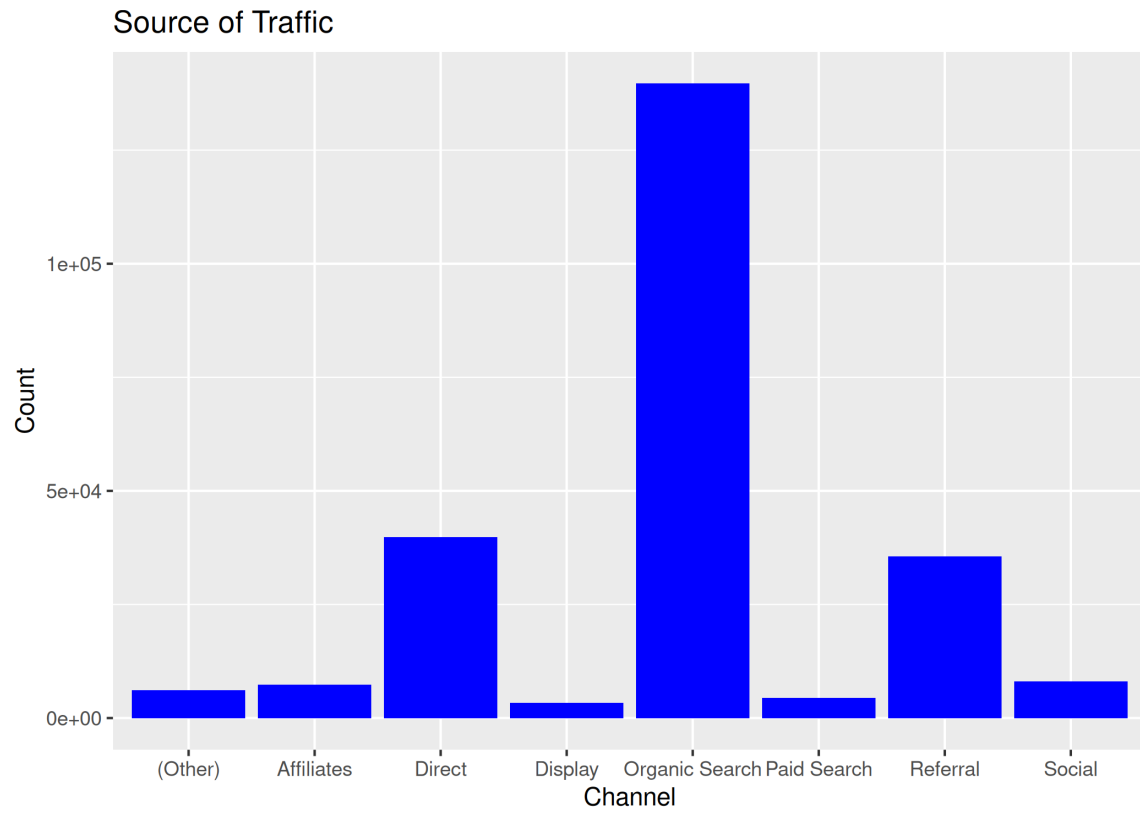
13.2.9 Summary

- Bar charts provide a visual representation of categorical data.
- Use grouped bar chart to make comparison against different categories of data.
- Use stacked bar chart while looking at cumulative data.
- Use proportional bar chart when you want to concurrently observe each of the several variables as they fluctuate.
- Use mosaic plot to discover associations between two variables.
- Use pie chart and donut chart when comparing each group's contribution to the whole.

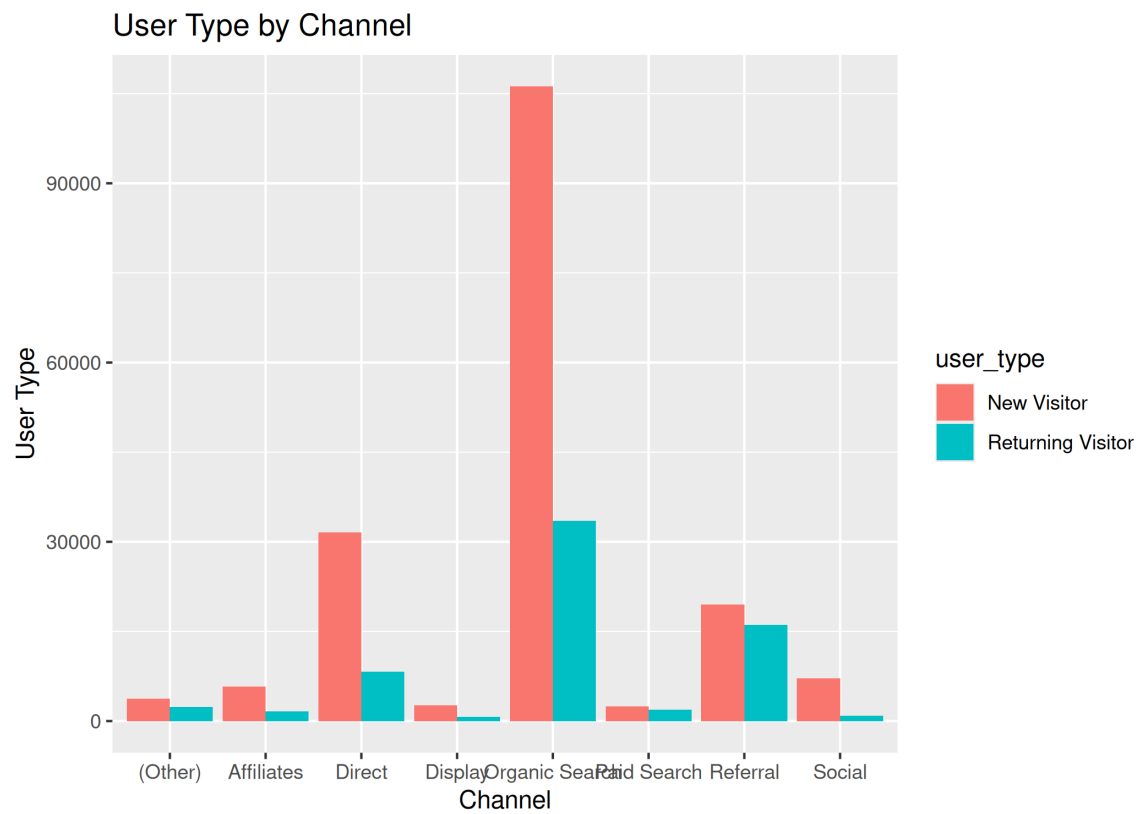
13.2.10 Your Turn...

Generate all the below plots:

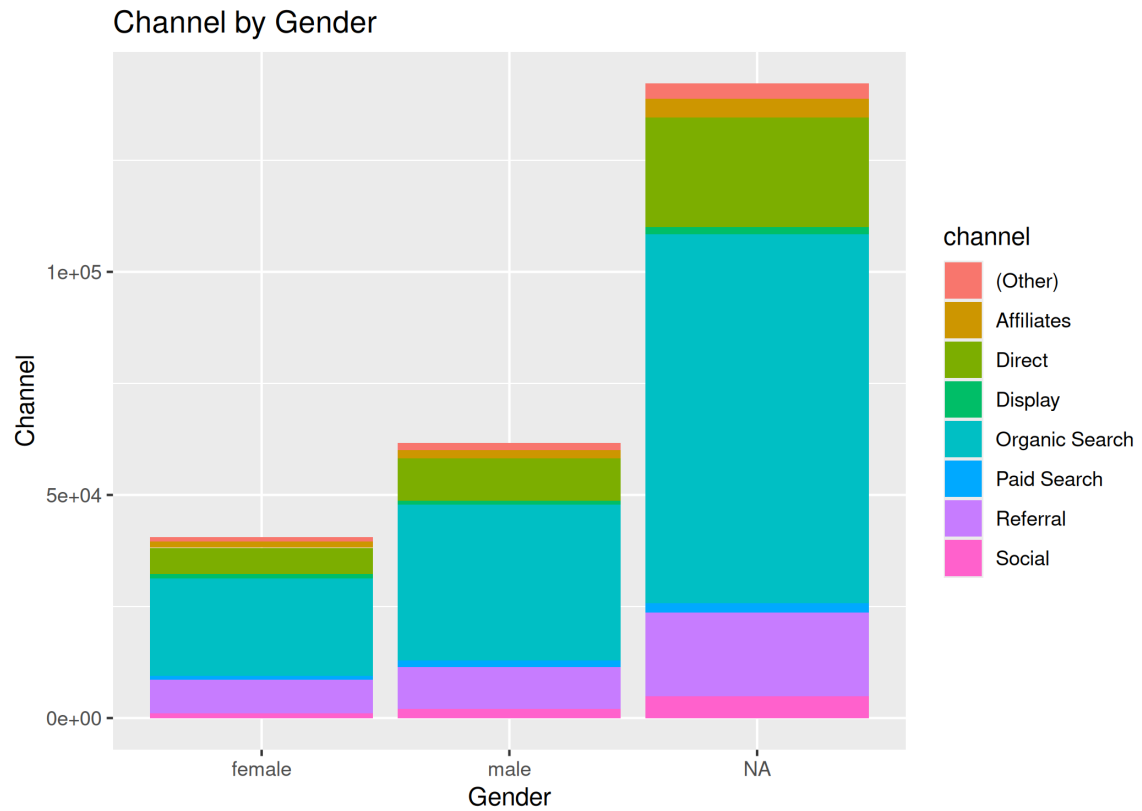
1. Bar plot of `channel`



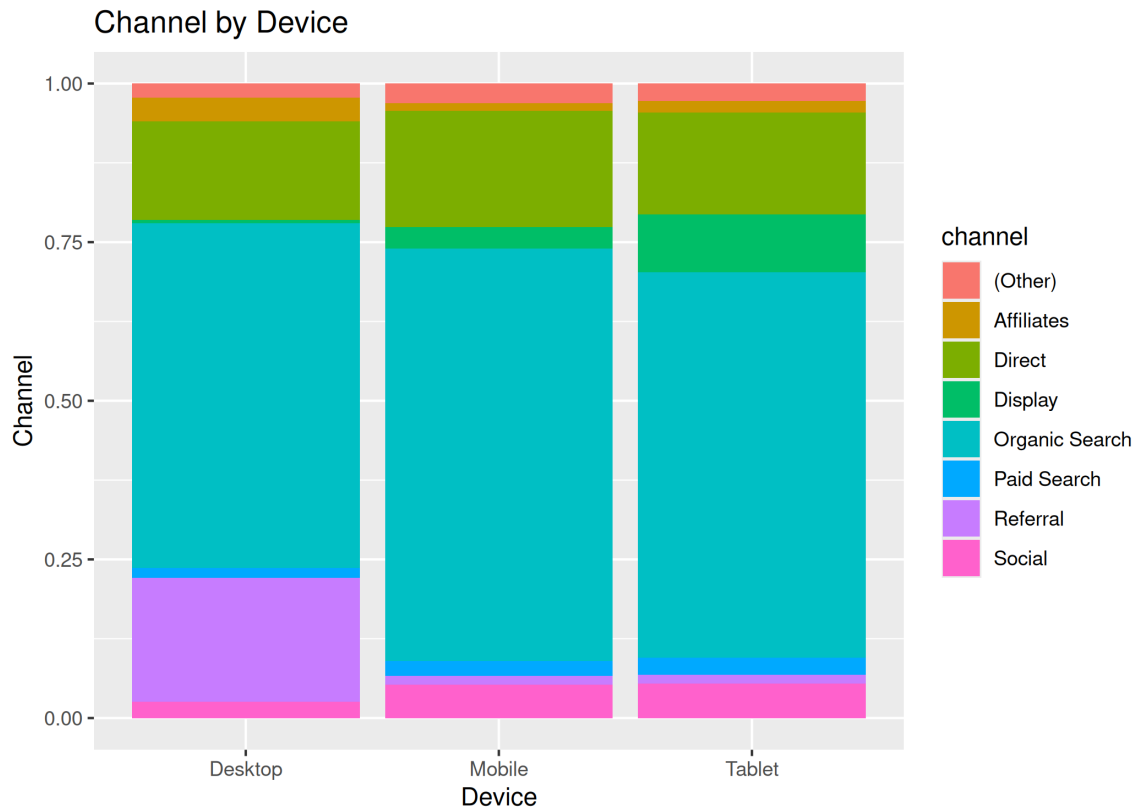
2. Display grouped bar plot of `user_type` by `channel`



3. Display stacked bar plot of `channel` by `gender`



4. Display proportional bar plot of channel by device

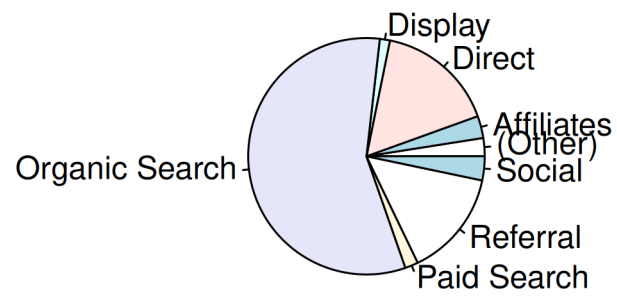


5. Display mosaic plot of `device` by `channel`

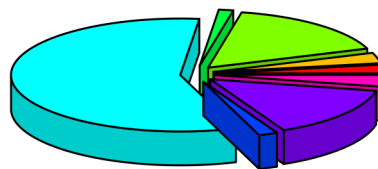
Mosaic plots need the **ggmosaic** package (archived on CRAN) and are out of scope here — they are covered in the [ggplot2 ebook](#).

6. Display pie or donut chart of `channel`

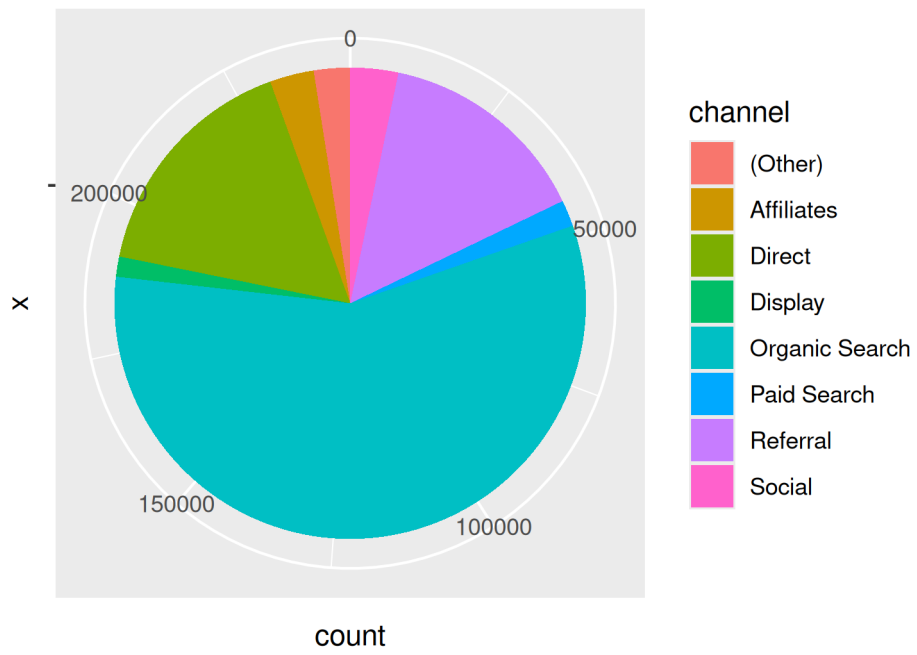
6.1 Pie Chart



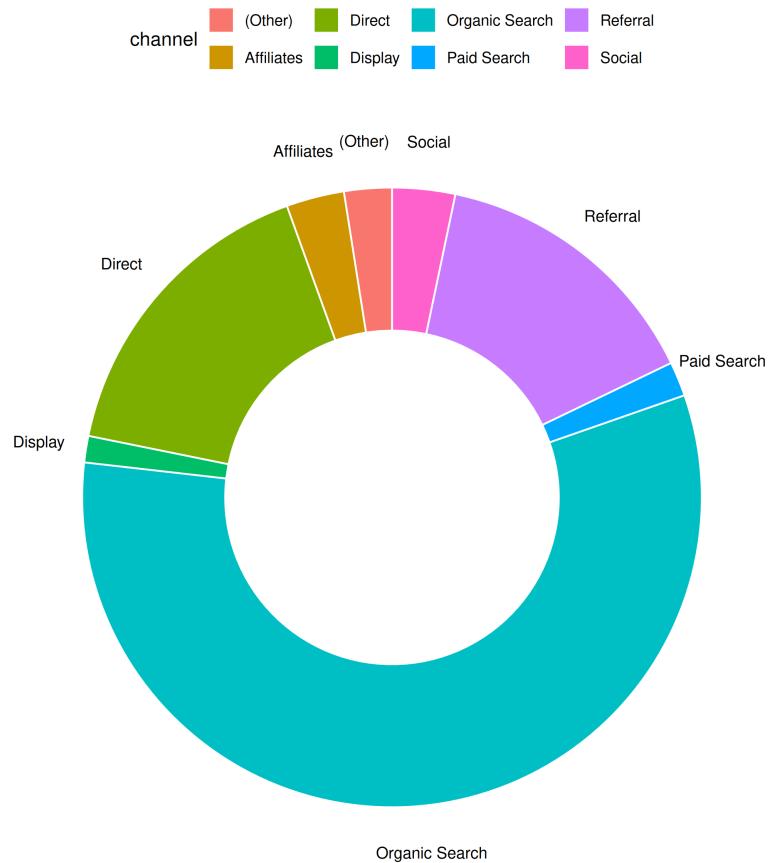
6.2 3D Pie Chart



6.3 Pie Chart (ggplot2)



6.4 Donut Chart



13.3 Useful R Packages

Below are a list of functions from different R packages for summarizing categorical data.

13.3.1 Summarize Variables

```
Hmisc::describe(data$device)
```

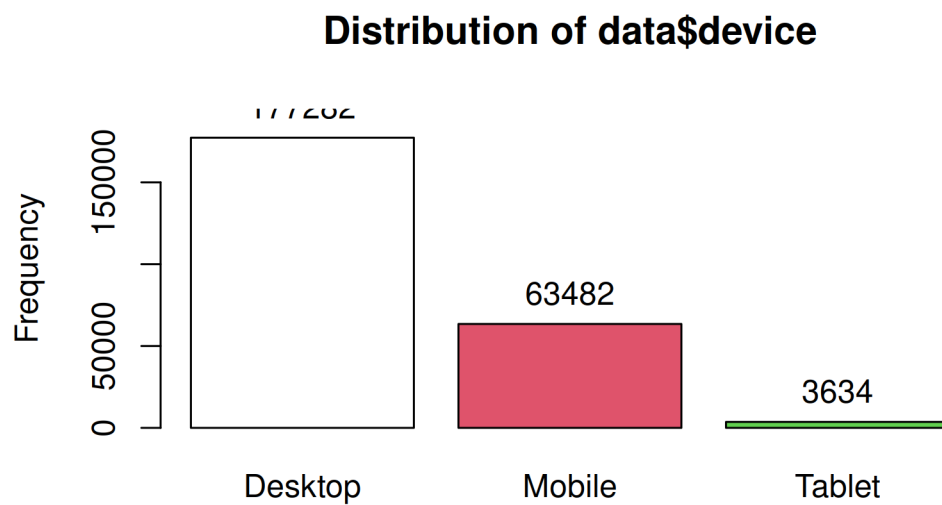
```
data$device
  n missing distinct
244398      0         3

Value      Desktop  Mobile  Tablet
Frequency 177282    63482   3634
Proportion 0.725    0.260   0.015
```

```
janitor::tabyl(data$device)
```

```
data$device      n      percent
Desktop 177282 0.72538237
Mobile   63482 0.25974844
Tablet   3634 0.01486919
```

```
epiDisplay::tab1(data$device)
```



```
data$device :
      Frequency Percent Cum. percent
Desktop   177282    72.5         72.5
Mobile    63482    26.0         98.5
Tablet     3634     1.5        100.0
Total    244398   100.0        100.0
```

```
summarytools::freq(data$device)
```

```
Frequencies
data$device
Type: Factor
```

Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
------	---------	--------------	---------	--------------

Desktop	177282	72.54	72.54	72.54	72.54
Mobile	63482	25.97	98.51	25.97	98.51
Tablet	3634	1.49	100.00	1.49	100.00
<NA>	0			0.00	100.00
Total	244398	100.00	100.00	100.00	100.00

```
questionr::freq(data$device)
```

```

      n    % val%
Desktop 177282 72.5 72.5
Mobile  63482 26.0 26.0
Tablet   3634  1.5  1.5

```

```
descriptr::ds_freq_table(data, device)
```

Variable: device				
Levels	Frequency	Cum Frequency	Percent	Cum Percent
Desktop	177282	177282	72.54	72.54
Mobile	63482	240764	25.97	98.51
Tablet	3634	244398	1.49	100
Total	244398	-	100.00	-

13.3.2 Summarize Data Set

```
minilytics <- select(data, device, browser, os)
psych::describe(minilytics)
```

```

      vars      n mean  sd median trimmed mad min max range  skew kurtosis
device*    1 244398 1.29 0.49      1    1.22 0.0   1   3     2   1.31     0.58
browser*   2 244398 8.22 4.84      6    7.14 0.0   1  26    25   1.80     1.43
os*        3 244398 8.80 4.47      8    8.99 8.9   1  16    15  -0.08      -
1.36

      se
device* 0.00
browser* 0.01
os*      0.01

```

```
skimr::skim(minilytics)
```

Table 13.4: Data summary

Name	minilytics
Number of rows	244398
Number of columns	3
Column type frequency: factor	3
Group variables	None

Variable type: factor

skim_vari- able	n_miss- ing	com- plete_rate	or- dered	n_uniquetop_counts
device	0	1	FALSE	3 Des: 177282, Mob: 63482, Tab: 3634
browser	0	1	FALSE	26 Chr: 192176, Saf: 35323, Fir: 6420, Edg: 3787
os	0	1	FALSE	16 Win: 91223, Mac: 66171, And: 39055, iOS: 27866

```
summarytools::dfSummary(minilytics)
```

Data Frame Summary

minilytics

Dimensions: 244398 x 3

Duplicates: 244317

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid
1	device [factor]	1. Desktop 2. Mobile 3. Tablet	177282 (72.5%) 63482 (26.0%) 3634 (1.5%)	IIIIIIIIIIIIIIIIII IIIII	244398 (100.0%)
2	browser [factor]	1. Amazon Silk 2. Android Browser 3. Android Webview 4. APKPure 5. BlackBerry	168 (0.1%) 101 (0.0%) 1215 (0.5%) 1 (0.0%) 7 (0.0%)		244398 (100.0%)

		6. Chrome	192176 (78.6%)	IIIIIIIIIIIIIIIIII	
		7. Coc Coc	45 (0.0%)		
		8. Edge	3787 (1.5%)		
		9. Firefox	6420 (2.6%)		
		10. Internet Explorer	700 (0.3%)		
		[16 others]	39778 (16.3%)	III	
3	os	1. (not set)	179 (0.1%)		244398
	[factor]	2. Android	39055 (16.0%)	III	(100.0%)
		3. BlackBerry	16 (0.0%)		
		4. Chrome OS	14038 (5.7%)	I	
		5. Firefox OS	4 (0.0%)		
		6. iOS	27866 (11.4%)	II	
		7. Linux	5781 (2.4%)		
		8. Macintosh	66171 (27.1%)	IIIIII	
		9. OS/2	3 (0.0%)		
		10. Playstation 4	15 (0.0%)		
		[6 others]	91270 (37.3%)	IIIIIIII	

```
tableone::CreateTableOne(vars = c("device", "os"), data = minilytics)
```

	Overall
n	244398
device (%)	
Desktop	177282 (72.5)
Mobile	63482 (26.0)
Tablet	3634 (1.5)
os (%)	
(not set)	179 (0.1)
Android	39055 (16.0)
BlackBerry	16 (0.0)
Chrome OS	14038 (5.7)
Firefox OS	4 (0.0)
iOS	27866 (11.4)
Linux	5781 (2.4)
Macintosh	66171 (27.1)
OS/2	3 (0.0)
Playstation 4	15 (0.0)
Playstation Vita	1 (0.0)
Samsung	5 (0.0)
Tizen	20 (0.0)
Windows	91223 (37.3)
Windows Phone	13 (0.0)

```
desctable::desctable(minilytics)
```

		N	%
1	device	244398	NA
2	device: Desktop	177282	7.253824e+01
3	device: Mobile	63482	2.597484e+01
4	device: Tablet	3634	1.486919e+00
5	browser	244398	NA
6	browser: Amazon Silk	168	6.874033e-02
7	browser: Android Browser	101	4.132603e-02
8	browser: Android Webview	1215	4.971399e-01
9	browser: APKPure	1	4.091687e-04
10	browser: BlackBerry	7	2.864181e-03
11	browser: Chrome	192176	7.863239e+01
12	browser: Coc Coc	45	1.841259e-02
13	browser: Edge	3787	1.549522e+00
14	browser: Firefox	6420	2.626863e+00
15	browser: Internet Explorer	700	2.864181e-01
16	browser: Maxthon	5	2.045843e-03
17	browser: Mozilla Compatible Agent	13	5.319192e-03
18	browser: MRCHROME	1	4.091687e-04
19	browser: Opera	1145	4.684981e-01
20	browser: Opera Mini	52	2.127677e-02
21	browser: Playstation 4	15	6.137530e-03
22	browser: Playstation Vita Browser	1	4.091687e-04
23	browser: Puffin	18	7.365036e-03
24	browser: Safari	35323	1.445306e+01
25	browser: Safari (in-app)	658	2.692330e-01
26	browser: Samsung Internet	2065	8.449333e-01
27	browser: SeaMonkey	3	1.227506e-03
28	browser: Seznam	1	4.091687e-04
29	browser: UC Browser	174	7.119535e-02
30	browser: User-Agent:Mozilla	28	1.145672e-02
31	browser: YaBrowser	276	1.129305e-01
32	os	244398	NA
33	os: (not set)	179	7.324119e-02
34	os: Android	39055	1.598008e+01
35	os: BlackBerry	16	6.546698e-03
36	os: Chrome OS	14038	5.743910e+00
37	os: Firefox OS	4	1.636675e-03
38	os: iOS	27866	1.140189e+01
39	os: Linux	5781	2.365404e+00
40	os: Macintosh	66171	2.707510e+01

41	os: OS/2	3	1.227506e-03
42	os: Playstation 4	15	6.137530e-03
43	os: Playstation Vita	1	4.091687e-04
44	os: Samsung	5	2.045843e-03
45	os: Tizen	20	8.183373e-03
46	os: Windows	91223	3.732559e+01
47	os: Windows Phone	13	5.319192e-03
48	os: Xbox	8	3.273349e-03

Try it live

 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(forcats)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
    "ecom_sample.csv"
  )
}
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)
ecom |>
  mutate(referrer = fct_infreq(referrer)) |>
  count(referrer)
```

14 Exercise Solutions

14.1 Introduction

Try every **Your Turn** before reading its solution — the struggle is the lesson. Worked solutions below are seeded for @dplyr-basics; solutions for the remaining chapters are open stubs. To contribute one, open an issue or pull request with the chapter, the question, and a runnable chunk in this book's style (`|>-first`, same data as the chapter).

We will use the following R packages:

```
library(dplyr)
library(readr)
```

14.2 dplyr Basics

```
ecom <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),
      purchase = col_logical(), n_pages = col_double(), n_visit = col_double(),
      duration = col_double(), order_value = col_double(), order_items = col_double()
    )
  )
```

Q1. What is the average number of pages visited by purchasers and non-purchasers?

```
ecom |>
  summarise(mean_pages = mean(n_pages), .by = purchase)
```

```
# A tibble: 2 x 2
  purchase mean_pages
  <lg1>      <dbl>
1 FALSE      4.79
2 TRUE      15.8
```

Purchasers browse far more pages (~16 vs ~5) — the signal behind the AOV case study.

Q2. What is the average time on site for purchasers vs non-purchasers?

```
ecom |>
  summarise(mean_time = mean(duration), .by = purchase)
```

```
# A tibble: 2 x 2
  purchase mean_time
  <lgl>         <dbl>
1 FALSE         355.
2 TRUE          359.
```

Time on site barely differs (~355 vs ~359 seconds): page depth, not duration, separates buyers here.

Q3. What is the average number of pages visited by purchasers and non-purchasers using mobile?

```
ecom |>
  filter(device == "mobile") |>
  summarise(mean_pages = mean(n_pages), .by = purchase)
```

```
# A tibble: 2 x 2
  purchase mean_pages
  <lgl>         <dbl>
1 FALSE         4.99
2 TRUE         16.2
```

14.3 Open stubs

Solutions for these chapters are not yet written — contributions welcome (see above):

- @joining-tables-in-r-dplyr: no **Your Turn** block; propose join puzzles on `customer/order`
- @dplyr-helper-functions: sampling/slicing drills on `ecom`
- @r-pipe-magrittr: rewrite-a-chain exercises (`%>%` → `|>`, `_`, `\(x)`)
- @tibbles-in-r: tibble vs `data.frame` edge cases
- @tidying-data-with-tidyr: the three **Your Turn** prompts in that chapter
- @strings-in-r: URL/email parsing variants on `mockstring`
- @date-and-time-in-r: the nine **Your Turn** blocks (intervals, TZ/DST, formats)
- @categorical-data-in-r: the seven **Your Turn** blocks (lumping, reordering, recoding)

A Arrow, Parquet and DuckDB

A.1 Introduction

Everything so far fit in memory as CSV. Two tools cover the next scale, and both speak the dplyr you already know:

- **Parquet** (via **arrow**) — a compressed columnar file format. The same data takes far less disk and reads much faster than CSV.
- **DuckDB** (via **duckplyr**) — an in-process analytical database. dplyr verbs run inside it, so data larger than memory stays on disk until `collect()`.

Both are optional: install them with `install.packages(c("arrow", "duckdb", "duckplyr"))`. The chunks below run only when the packages are present (`eval = requireNamespace(...)`), so the book still builds without them. Browser execution of parquet is intentionally deferred until WebR binary sizes are checked (see the P2 plan).

A.2 Parquet with arrow

Write `ecom` to parquet and read it back. The round-trip lands in a temporary directory, so nothing litters the project:

```
library(dplyr)
library(readr)
library(arrow)

ecom <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),
      purchase = col_logical(), n_pages = col_double(), n_visit = col_double(),
      duration = col_double(), order_value = col_double(), order_items = col_double()
    )
  )

pq <- file.path(tempdir(), "ecom.parquet")
cs <- file.path(tempdir(), "ecom.csv")
write_parquet(ecom, pq)
```

```
write_csv(ecom, cs)

# same rows, roughly half the bytes
c(csv_bytes = file.size(cs), parquet_bytes = file.size(pq))
```

```
csv_bytes parquet_bytes
32287      14878
```

`read_parquet()` returns a tibble, so every verb from `@dplyr-basics` works unchanged:

```
ecom_pq <- read_parquet(pq)
class(ecom_pq)
```

```
[1] "spec_tbl_df" "tbl_df"      "tbl"        "data.frame"
```

```
ecom_pq |>
  filter(purchase) |>
  summarise(revenue = sum(order_value), orders = n(), .by = device)
```

```
# A tibble: 3 x 3
  device revenue orders
  <fct>    <dbl>  <int>
1 tablet   51321     36
2 mobile   51504     36
3 laptop   56531     31
```

A.3 DuckDB with duckplyr

`duckdb_tibble()` (`duckplyr` $\geq$ 1.0) moves a data frame into DuckDB and hands back a lazy table: dplyr verbs build the query, nothing runs until `collect()`:

```
library(duckplyr)

ecom_duck <- duckdb_tibble(ecom)
class(ecom_duck)
```

```
[1] "duckplyr_df" "tbl_df"      "tbl"        "data.frame"
```

```
ecom_duck |>
  filter(purchase) |>
  summarise(revenue = sum(order_value), orders = n(), .by = device) |>
  mutate(aov = revenue / orders) |>
  collect()
```

```
# A tibble: 3 x 4
  device revenue orders  aov
* <fct>   <dbl>  <int> <dbl>
1 laptop   56531     31 1824.
2 tablet   51321     36 1426.
3 mobile   51504     36 1431.
```

The numbers match the Chapter @dplyr-basics AOV exactly — the engine changed, the answers did not. Reach for this pattern when a single CSV no longer fits comfortably in memory: keep the data in DuckDB (or parquet on disk), push the verbs down, and `collect()` only the summary.

B Legacy magrittr Operators

This book teaches the base R pipe `|>` exclusively (see `@r-pipe-magrittr` for the one comparison table). This appendix is a graveyard: a read-only map for the magrittr operators you will meet in inherited code. Do not use them in new code.

Inherited operator	Meaning	Modern equivalent (this book)
<code>x %>% f()</code>	Forward <code>x</code> into first argument of <code>f</code>	<code>x %>% f()</code>
<code>x %\$% f(col)</code>	Expose columns of <code>x</code> to <code>f</code>	<code>x %>% pull(col) %>% f()</code>
<code>x %<>% f()</code>	Forward and assign back to <code>x</code>	<code>x <- x %>% f()</code>
<code>x %>% extract(i) / extract2(i)</code>	Subset by position or name	<code>x[i] / x[[i]]</code> , or <code>pull(x, col) / pluck(x, i)</code>
<code>x %>% use_series(col)</code>	Extract <code>col</code> as vector	<code>pull(x, col)</code> or <code>x\$col</code>

The pipe chapter's data-extraction section covers `pull()/pluck()` with runnable examples; the legacy note there explains why `%$%`, `%<>%`, `extract()/extract2()` and `use_series()` stay out of the main text.

C From pandas to dplyr

C.1 Introduction

Switching from Python? The ideas transfer one-to-one; only the verbs change. `pandas` snippets below are static reference (never executed by this book); the `dplyr` side runs on the ecommerce data from @dplyr-basics.

```
library(dplyr)
library(readr)

ecom <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/web.csv',
    col_types = cols_only(device = col_factor(levels = c("laptop", "tablet", "mobile")),
      referrer = col_factor(levels = c("bing", "direct", "social", "yahoo", "google")),
      purchase = col_logical(), n_pages = col_double(), n_visit = col_double(),
      duration = col_double(), order_value = col_double(), order_items = col_double()
    )
  )
```

C.2 Verb map

Task	pandas	dplyr (this book)
Read CSV	<code>pd.read_csv("web.csv")</code>	<code>read_csv("web.csv")</code>
First rows	<code>df.head(10)</code>	<code>head(ecom, 10) /</code> <code>slice_head(ecom, n = 10)</code>
Filter rows	<code>df[df.purchase]</code> or <code>df.query("purchase")</code>	<code>filter(ecom, purchase)</code>
Pick columns	<code>df[["device", "order_value"]]</code>	<code>select(ecom, device, order_value)</code>
Sort	<code>df.sort_values("n_pages")</code>	<code>arrange(ecom, n_pages)</code>
New column	<code>df.assign(aov=df.revenue/df.orders)</code>	<code>mutate(ecom4, aov = revenue / orders)</code>
Group + aggregate	<code>df.groupby("device").agg(revenue=sum)</code>	<code>summarise_by(device, revenue = sum(order_value), .by = device)</code>

Task	pandas	dplyr (this book)
Join	<code>pd.merge(customer, order, on="id", how="left")</code>	<code>left_join(customer, order, by = join_by(id))</code>
Drop missing	<code>df.dropna()</code>	<code>drop_na(df)</code>
Reshape long/wide	<code>df.melt(...)</code> / <code>df.pivot(...)</code>	<code>pivot_longer()</code> / <code>pivot_wider()</code>
Strings	<code>s.str.contains(pat)</code> , <code>s.str.extract(pat)</code>	<code>str_detect()</code> , <code>str_extract()</code>
Datetimes	<code>pd.to_datetime(s)</code> , <code>s.dt.year</code>	<code>ymd()</code> , <code>year()</code> , <code>month()</code> , <code>day()</code>
Categories	<code>astype("category")</code> , <code>cat.reorder_categories</code>	<code>factor()</code> , <code>fct_relevel()</code> , <code>fct_lump_*</code>

C.3 Worked pair: grouped means

pandas:

```
ecom.groupby("purchase")["n_pages"].mean()
```

dplyr (runs here):

```
ecom |>
  summarise(mean_pages = mean(n_pages), .by = purchase)
```

```
# A tibble: 2 x 2
  purchase mean_pages
  <lgl>         <dbl>
1 FALSE         4.79
2 TRUE         15.8
```

Same split-apply-combine, same answer. The rest of the grammar maps the same way: chain with `|>` where pandas chains with `.`, and reach for Chapters [@dplyr-basics](#)–[@categorical-data-in-r](#) for the full drill.

Try it live

💡 Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via [WebR](#) — no R installation needed. The runtime downloads once, on this page only.

```
library(dplyr)
library(readr)
if (!file.exists("ecom_sample.csv")) {
  download.file(
    "https://wrangle-r.rsquaredacademy.com/data-webr/ecom_sample.csv",
    "ecom_sample.csv"
  )
}
ecom <- read_csv("ecom_sample.csv", show_col_types = FALSE)
ecom |>
  summarise(mean_pages = mean(n_pages), .by = purchase)
```